

NO. 349f7bae12lk8406 | 2026-05-16 13:48:11

- 题目：网络流量检测管理系统的设计与开发
- 作者：郭子谊
- 检测所属单位：-

📄 论文字符数：89446 📄 论文页数：89 📊 表格数量：16 🖼️ 图片数量：10

检测结果



0.36%

全文总相似比 (复写率+自引率+他引率+专业术语)

相似结果详情

0.3%	0.0%	0.06%	0.0%
复写率	自引率	他引率	专业术语

其他指标

去除引用本人文献相似率：0.36% 去除专业术语相似率：0.36% 自写率：99.64%

典型相似文章：无

检测范围 | 1989-01-01 ~ 2026-05-16

- | | | |
|-----------------|------------------|-------------------|
| ■ 中文科技期刊论文全文数据库 | ■ 博士/硕士学位论文全文数据库 | ■ 外文特色文献数据全库 |
| ■ 中文主要报纸全文数据库 | ■ 中国专利特色数据库 | ■ 中国主要会议论文特色数据库 |
| ■ 古籍文献/图书资源 | ■ IPUB原创作品 | ■ 互联网数据资源/互联网文档资源 |
| ■ 港澳台文献资源 | ■ 年鉴资源 | ■ 维普优先出版论文全文数据库 |

相似片段

相似片段：

12	11	1
总相似片段	相似片段	引用片段

检测来源：

期刊 1	综合 0	外文 0
博硕 0	互联网 11	

引用文献汇总

引用文献来源：1

序号	引用文献	引用字符数	引用率	来源
1	【WebSocket通信技巧】：Apache HttpClient实现实时交互指南	43	0.06%	互联网

相似文献汇总 (当前只展示10条数据,全部详情请查看片段对照报告)

相似文献来源：11

序号	相似文献	相似字符数	相似率	来源
1	正向代理与反向代理：深入解析与实例讲解代理服务器是一种	60	0.08%	互联网
2	网络防火墙系统设计研究绪论,软件工程硕士论文_学术堂	50	0.07%	互联网
3	面向青少年特殊应用的心理健康咨询系统的分析与设计 - 豆丁网	49	0.07%	互联网
4	大学生毕业论文-7785	44	0.06%	互联网
5	【WebSocket通信技巧】：Apache HttpClient实现实时交互指南	43	0.06%	互联网
6	大学生毕业论文-10303	36	0.05%	互联网
7	一套某计算机接口物理特性测试系统的开发研究_论文定制中心	36	0.05%	互联网
8	基于卷积神经网络的入侵检测技术的研究 - 豆丁网	34	0.05%	互联网
9	浅谈利用网络服务器在机房内控制学生上网行为 王亮 - 中小企业管理与科技(上旬刊) - 2014	32	0.04%	期刊
10	代理技术 重磅，代理服务器背后的故事(正向、反向代理	29	0.04%	互联网

文字标注

■ 自写片段 ■ 复写片段 ■ 引用片段 ■ 专业术语 ■ 自引片段



重慶理工大學

毕业设计（论文）

题目 网络流量检测管理系统的设计与开发

学 院 计算机科学与工程学院

专 业 计算机科学与技术

班 级 122030701

学生姓名 郭子谊 学号 11912060207

指导教师 范伟 职称 教授

时 间 2026年06月

目 录

摘 要	I
Abstract	II
1 绪论	1
1.1 研究背景与意义	1
1.1.1 网络代理在开发调试与安全观测中的作用	1
1.1.2 传统代理工具在可视化管理方面的不足	1
1.1.3 本课题的工程实践意义	2
1.2 国内外研究现状	2
1.2.1 网络代理与抓包工具发展现状	2
1.2.2 Web 化运维控制台发展现状	3
1.2.3 对象存储在流量留存中的应用	3
1.3 研究目标与主要内容	4
1.3.1 系统建设目标	4
1.3.2 主要研究内容	4
1.3.3 论文拟解决的问题	5
1.4 论文组织结构	5
2 系统相关技术	6

2.1 HTTP/HTTPS 代理相关技术	6
2.1.1 HTTP 正向代理原理	6
2.1.2 HTTPS CONNECT 隧道原理	6
2.1.3 MITM 中间人代理原理	7
2.1.4 HTTP/1.1 长连接与分块传输	7
2.2 后端开发相关技术	8
2.2.1 Go 网络编程与 net/http	8
2.2.2 Go 并发控制技术	8
2.2.3 MinIO 对象存储技术	9
2.3 前端开发相关技术	9
2.3.1 Vue 3 与 Vite	9
2.3.2 Vue Router 与 Pinia	10
2.3.3 图表与虚拟滚动技术	10
2.4 前后端通信与部署相关技术	10
2.4.1 REST 配置接口	10
2.4.2 WebSocket 实时推送	11
2.4.3 前端嵌入式部署	11
2.5 设计模式相关技术	12
2.5.1 观察者模式（发布-订阅模式）	12
2.5.2 责任链模式	12
2.5.3 单例模式	12
2.5.4 适配器模式	13
3 系统需求分析	13
3.1 系统总体需求	13
3.1.1 用户角色与使用场景	13
3.1.2 功能性需求	14
3.1.3 非功能性需求	15
3.2 后端功能需求	15
3.2.1 代理转发需求	15
.....	

3.2.2 规则路由需求 16

3.2.3 访问控制需求 16

3.2.4 流量监控需求 16

3.2.5 抓包与对象存储需求 17

3.2.6 配置管理需求 17

3.3 前端功能需求 17

3.3.1 登录与代理核心连接 17

3.3.2 概览仪表盘 18

3.3.3 连接与历史连接管理 18

3.3.4 日志管理 18

3.3.5 MITM 抓包管理 18

3.3.6 路由、安全、存储和高级配置 19

3.4 数据与接口需求 19

3.4.1 配置数据模型 19

3.4.2 WebSocket 消息模型 20

3.4.3 存储数据模型 20

3.5 可行性分析 21

3.5.1 技术可行性 21

3.5.2 经济可行性 21

3.5.3 操作可行性 21

4 系统总体设计 22

4.1 系统总体架构设计 22

4.1.1 整体架构 22

4.1.2 前后端协作流程 23

4.1.3 运行时数据流 23

4.2 后端模块划分 24

4.2.1 代理核心模块 25

4.2.2 配置管理模块 26

4.2.3 路由模块 26

.....

4.2.4 访问控制模块	26
4.2.5 流量监控与连接管理模块	27
4.2.6 MITM 抓包与存储模块	27
4.2.7 WebSocket 控制模块	28
4.3 前端模块划分	28
4.3.1 路由与布局模块	29
4.3.2 状态管理模块	29
4.3.3 页面展示模块	30
4.4 关键数据结构设计	30
4.4.1 代理上下文 Pcontext	31
4.4.2 连接信息 ConnectionInfo	31
4.4.3 规则配置结构	32
4.4.4 抓包交换结构	32
4.5 系统安全设计	32
4.5.1 控制服务访问控制	33
4.5.2 MITM 使用边界	33
4.5.3 代理自环防护	33
5 系统详细设计与实现	34
5.1 后端启动流程实现	34
5.1.1 主程序初始化	34
5.1.2 代理监听服务	35
5.1.3 前端资源嵌入	35
5.2 HTTP 代理实现	36
5.2.1 普通 HTTP 转发流程	36
5.2.2 HTTP-MITM 引擎实现	37
5.2.3 WebSocket 与流式响应处理	37
5.3 HTTPS 与 CONNECT 实现	38
5.3.1 CONNECT 隧道模式	38
5.3.2 MITM 模式	38
.....	

5.3.3 证书签发实现 39

5.4 路由引擎实现 39

5.4.1 出站拨号器设计 39

5.4.2 路由规则匹配 40

5.4.3 路由热重载 40

5.4.4 自环检测 41

5.5 访问控制实现 41

5.5.1 目标访问控制 41

5.5.2 来源 IP 封禁 42

5.5.3 拦截日志与连接关闭 42

5.6 流量统计与连接管理实现 42

5.6.1 HTTP 明文流量统计 42

5.6.2 隧道流量统计 43

5.6.3 连接生命周期管理 43

5.6.4 用户流量聚合 44

5.7 MITM 抓包与 MinIO 存储实现 44

5.7.1 抓包快照生成 44

5.7.2 请求体和响应体捕获 45

5.7.3 大文件与 chunked 上传处理 45

5.7.4 下载接口实现 46

5.8 WebSocket 控制服务实现 46

5.8.1 连接认证与订阅 46

5.8.2 实时数据推送器 47

5.8.3 批量发送与背压处理 47

5.9 前端控制台实现 47

5.9.1 登录页实现 47

5.9.2 仪表盘布局实现 48

5.9.3 概览页实现 48

5.9.4 连接页与历史连接页实现 49

.....

5.9.5 日志页实现	49
5.9.6 MITM 页面实现	49
5.9.7 配置页面实现	50
5.10 异常处理与可靠性实现	50
5.10.1 连接关闭处理	50
5.10.2 请求体关闭处理	51
5.10.3 存储异常降级	51
5.11 设计模式在系统中的应用	52
5.11.1 观察者模式（发布-订阅模式）的应用	52
5.11.2 责任链模式的应用	52
5.11.3 单例模式的应用	52
5.11.4 适配器模式的应用	53
6 系统测试与结果分析	53
6.1 测试环境与测试方法	54
6.1.1 测试环境	54
6.1.2 测试方法	55
6.2 单元测试设计	56
6.2.1 HTTP 解析器测试	56
6.2.2 配置管理测试	56
6.2.3 访问控制测试	56
6.2.4 MinIO 存储测试	57
6.2.5 goroutine 泄露检查测试	57
6.3 功能测试设计	57
6.3.1 HTTP 代理功能测试	57
6.3.2 HTTPS 隧道功能测试	57
6.3.3 MITM 抓包功能测试	58
6.3.4 路由配置功能测试	58
6.3.5 访问控制功能测试	58
6.3.6 前端页面功能测试	58
.....	

6.4 性能测试设计 59

6.4.1 5MB 文件上传下载测试 59

6.4.2 高并发压力测试 59

6.4.3 pprof 性能分析 60

6.5 测试结果分析 60

6.5.1 功能完整性分析 60

6.5.2 性能与资源占用分析 61

6.5.3 问题与改进分析 61

7 总结与展望 62

7.1 工作总结 62

7.1.1 系统完成情况 62

7.1.2 论文工作总结 63

7.2 系统特色 64

7.2.1 代理核心与控制台一体化 64

7.2.2 实时监控与可操作连接表 64

7.2.3 流式抓包与对象存储结合 64

7.3 不足之处 65

7.3.1 安全认证仍需完善 65

7.3.2 协议覆盖仍可扩展 65

7.3.3 数据持久化能力仍可增强 66

7.4 展望 66

7.4.1 多用户与权限管理 66

7.4.2 更丰富的规则体系 66

7.4.3 分布式代理节点管理 66

致谢 67

参考文献 67

附录 68

附录 A 配置文件示例 68

附录 B 主要接口说明 71

.....

附录 C WebSocket 消息格式	73
---------------------------	----

附录 D 测试与构建命令	75
--------------------	----

摘 要

随着 Web 服务、接口联调和网络安全观测需求的增加，代理系统不仅需要完成请求转发，还需要具备实时监控、规则配置、访问控制和抓包留存等管理能力。本文设计并实现了一套基于 Go 与 Vue 的可视化网络代理管理系统，面向开发调试、网络观测和课程演示等场景，构建从代理接入、运行观测、策略干预到数据留存的完整工作流程。系统后端以 Go 语言为核心，基于 HTTP/HTTPS 代理机制实现普通 HTTP 正向代理、HTTPS CONNECT 隧道、授权环境下的 MITM 明文解析、规则路由、访问控制、连接管理、实时流量统计和抓包正文存储等功能；通过请求、响应和 CONNECT 责任链组织转发、路由、统计、存储捕获和安全检查等处理逻辑。前端采用 Vue 3、Vite、Pinia、Vue Router、Chart.js 和虚拟滚动技术，完成登录、概览、连接、历史连接、日志、MITM 抓包、路由配置、安全策略、用户监控、存储配置和高级设置等页面。系统通过 REST 接口完成配置读取与保存，通过 WebSocket 按主题批量推送流量、连接、日志、拦截记录、抓包记录和用户流量数据，并使用 MinIO 对请求体和响应体进行流式存储，降低大文件抓包对代理进程和浏览器端内存的影响。测试部分围绕 HTTP 解析器、配置管理、访问控制、MinIO 存储、goroutine 泄露检查、HTTP/HTTPS 转发、MITM 抓包、KnownSize 与 Chunked 传输以及压力基准进行验证。结果表明，系统能够较好地完成代理转发、实时观测、策略配置和抓包留存等核心任务，在开发调试与毕业设计演示场景下具有一定的可用性和工程实践价值。

关键词：网络代理；Go；Vue；WebSocket；MITM；对象存储；访问控制

Abstract

With the increasing demand for Web service integration, interface debugging, and network security observation, a proxy system is expected not only to forward requests but also to provide real-time monitoring, rule configuration, access control, and traffic retention. This thesis designs and implements a visual network proxy management system based on Go and Vue. The system targets development debugging, network observation, and graduation project demonstration scenarios, and builds a complete workflow from proxy access and runtime observation to policy intervention and data retention. The backend is implemented in Go. It supports ordinary HTTP forward proxying, HTTPS CONNECT tunneling, authorized MITM inspection, rule-based routing, access control, connection management, real-time traffic statistics, and request and response body storage. Request, response, and CONNECT processing chains are used to organize forwarding, routing, traffic accounting, storage capture, and security checks. The frontend is developed with Vue 3, Vite, Pinia, Vue Router, Chart.js, and virtual scrolling, providing pages for login, overview, active connections, historical connections, logs, MITM records, routing rules, security policies, user monitoring, storage settings, and advanced configuration. REST APIs are used for configuration reading and persistence,

while WebSocket topics are used to push traffic, connections, logs, interception records, MITM exchanges, and user traffic in batches. MinIO is introduced to store request and response bodies in a streaming manner, which reduces memory pressure on the proxy process and browser when large files are captured. The system is evaluated through tests for HTTP parsing, configuration management, access control, MinIO storage, goroutine leak detection, HTTP/HTTPS forwarding, MITM capture, known-size and chunked transmission, and benchmark-based stress scenarios. The results show that the system can complete core tasks such as proxy forwarding, real-time observation, policy configuration, and capture retention, and it has practical value for development debugging and graduation project demonstration.

Key words: network proxy, Go, Vue, WebSocket, MITM, object storage, access control

1 绪论

1.1 研究背景与意义

1.1.1 网络代理在开发调试与安全观测中的作用

随着 Web 应用、移动应用和接口服务的规模不断扩大,客户端与服务端之间的通信链路已经从简单的页面访问扩展为接口调用、文件上传下载、长连接推送、第三方服务访问和多环境联调等多种形态。HTTP 协议以请求和响应为基本交互单元,不同 HTTP 版本共享统一的语义体系,请求方法、状态码和头部字段共同描述客户端意图与服务端结果^[1]。在这一链路中,网络代理位于客户端与目标服务之间,既可以完成请求转发,也可以对连接状态、请求头、响应头、数据体大小和错误信息进行集中观测。

对于 HTTPS 场景,客户端通常使用 CONNECT 方法要求代理建立到目标主机的隧道,隧道建立成功后代理在两个方向上转发字节流^[1]。这使代理工具能够在不破坏端到端加密的前提下完成访问转发;而在授权调试场景中,MITM 模式又可以通过本地证书将隧道内流量解析为明文 HTTP 请求和响应,从而辅助接口调试、故障定位和安全策略验证。WebSocket 则以单个 TCP 连接支持网页与服务端之间的双向通信,适合实时状态推送和在线协作等场景^[2]。因此,一个完整的代理管理系统不仅要处理普通 HTTP 请求,还要处理 CONNECT 隧道、MITM 明文流和长连接流量。

本课题所实现的系统正是围绕上述需求展开。后端 CoreHttpServer 同时作为 HTTP 服务端和转发客户端工作,入口处根据请求方法区分普通 HTTP 请求和 CONNECT 请求;配置、路由、访问控制、流量统计、MinIO 存储和 WebSocket 控制服务在主程序中按顺序初始化。该设计使系统既能承担网络代理的基础转发职责,也能为后续的连接列表、流量曲线、日志检索、抓包详情和用户流量统计提供统一数据来源。

1.1.2 传统代理工具在可视化管理方面的不足

传统代理或抓包工具往往侧重某一类能力,例如命令行代理强调轻量转发,抓包工具强调请求响应查看,网类系统强调路由、认证和策略控制。这些工具在真实工程中各有价值,但对于毕业设计这样的综合实践场景,仅具备单点功能难以完整展示代理系统从请求进入、策略判断、路由转发、流量统计到前端呈现的全过程。尤其是在高频连接、长连接和大文件传输场景下,如果缺少统一的可视化界面,使用者很难直观判断系统当前负载、连接状态和异常来源。

本系统针对这一问题，将代理核心和 Web 控制台整合为一个整体。后端控制服务提供 `/api/config`、`/api/stats`、`/api/storage/download` 和 `/start` 等接口，既负责配置读写，也负责 WebSocket 实时推送；前端则包含概览、连接、历史连接、日志、MITM、路由配置、高级配置、存储配置、安全策略和用户监控等页面。与单纯命令行程序相比，这种设计更适合进行功能演示、问题复现和运行状态分析。

从工程实践角度看，可视化管理的意义不只是展示数据，还在于把操作入口与反馈结果放在同一系统中。例如用户在安全策略页面配置来源 IP 封禁后，后端访问控制模块可以热重载规则并关闭已命中的活动连接；用户在连接页关闭单条连接后，后端通过连接表中的回调释放对应会话；用户在 MITM 页面查看请求响应详情时，可以继续通过对对象存储下载请求体或响应体。上述闭环有助于体现代理系统的完整管理能力。

1.1.3 本课题的工程实践意义

本课题具有较强的工程综合性。后端部分基于 Go 语言实现，使用 `net/http` 提供的服务端、客户端和传输层能力完成代理监听与请求转发，Go 标准库文档也明确提供了 HTTP 客户端和服务端实现^[3]。项目代码中，`main.go` 创建代理核心并启动 HTTP 代理端口、HTTPS 代理端口和控制服务端口；`CoreHttpServer` 维护请求处理器、响应处理器、CONNECT 处理器、路由器、访问控制器、MinIO 客户端和连接表。围绕这些结构，系统需要处理 TLS、连接劫持、HTTP 解析、并发统计、通道推送和流式数据复制等问题。

前端部分基于 Vue 3、Vite、Pinia、Vue Router、Chart.js 和虚拟滚动技术实现。Vue 官方文档将 Vue 定义为构建用户界面的 JavaScript 框架，并强调其声明式、组件化和响应式模型^[4]。本项目的前端路由将登录页和控制台页面分离，控制台页面包含概览、连接、日志、MITM、路由配置、存储配置和安全策略等子页面；Pinia Store 负责维护 WebSocket 连接状态、实时流量、连接快照、日志、抓包记录、拦截日志和用户流量列表。前后端结合后，可以较完整地体现现代 Web 管理台的组件化开发方式。

从软件工程训练角度看，本课题覆盖了需求分析、架构设计、模块划分、接口设计、异常处理、测试验证和部署构建等多个环节。系统既有明确的网络协议基础，也有可运行的前后端实现；既关注普通请求转发，也关注流量观测、访问控制和大对象存储等工程问题。因此，该课题不仅能够展示开发语言和框架的使用能力，还能体现对系统边界、运行状态和可靠性问题的综合分析能力。

1.2 国内外研究现状

1.2.1 网络代理与抓包工具发展现状

从技术形态看，网络代理主要包括正向代理、反向代理、透明代理、调试抓包代理和网关代理等类型。正向代理更关注客户端侧访问转发与出口控制，反向代理更关注服务端入口负载、路由和保护，抓包工具则强调请求响应的可观察性。HTTP 标准中对方法语义、CONNECT 隧道和连接行为的定义，为这些工具提供了共同的协议基础^[1]。在 HTTPS 普遍使用的背景下，代理工具通常需要在隧道透传和授权 MITM 解密之间做出明确区分。

成熟代理工具已经能够支持请求查看、规则匹配、证书安装、响应修改、限速、脚本扩展和日志分析等能力，但这些工具往往功能复杂、学习成本较高，或者侧重调试而不强调课程项目中的实现透明度。本系统没有以复刻商业工具为目标，而是选择围绕毕业设计可解释、可演示、可验证的核心链路展开：HTTP 请求由责任链处理，CONNECT 请求由策略选择器决定透传或 MITM，访问控制与路由规则通过配置热重载生效，运行数据通过 WebSocket 统一推送到前端。

因此，本课题在研究现状中的定位是工程化实现与可视化整合。它并不追求覆盖所有协议和所有高级抓包功能，而是重点展示代理核心、规则系统、实时监控、抓包留存和前端控制台之间的协同关系。通过这种定位，论文后续章节可以把每一项需求直接对应到代码模块和运行流程，避免停留在工具使用层面。

1.2.2 Web 化运维控制台发展现状

随着系统运行数据规模增加，运维和调试工具逐渐从命令行输出转向 Web 化控制台。Web 控制台通常包含实时仪表盘、配置管理、日志查询、连接列表和告警信息等页面，能够降低使用者理解运行状态的成本。对于代理系统而言，连接和流量具有明显的实时性，如果只依赖周期性刷新 REST 接口，容易出现延迟较高、请求频繁和前端状态不同步等问题。

WebSocket 协议提供了基于单个 TCP 连接的双向通信能力，可替代轮询式通信，适用于网页与服务端之间的实时消息交换^[2]。本系统后端控制服务在 /start 路径升级 WebSocket 连接，前端连接成功后发送订阅消息，声明需要接收的 traffic、connections、logs、mitm_detail、intercept_logs 和 user_traffic 等主题。后端 Hub 根据订阅主题批量推送数据，前端 Store 接收后再通知对应页面刷新。

项目的前端页面也体现了 Web 化控制台的发展方向。概览页展示最近 60 秒上传和下载速率曲线、安全策略摘要、路由策略摘要和用户流量摘要；连接页和历史连接页支持筛选、排序、展开和关闭连接；日志页支持等级和类别过滤；MITM 页面支持搜索、排序、展开 Header 与下载 Body。与单页数据展示相比，这种多页面控制台更符合代理管理系统的日常使用方式。

1.2.3 对象存储在流量留存中的应用

在抓包和流量审计场景中，请求体与响应体可能远大于头部和元数据。如果系统将所有 Body 直接保存在内存中，不仅会增加代理进程的内存压力，也会导致前端 WebSocket 消息过大，影响实时界面稳定性。更合理的方式是将头部、状态码、耗时、大小和对象 Key 作为元数据推送到前端，而将真正的大对象写入独立存储系统。

MinIO Go Client SDK 提供面向 Amazon S3 兼容对象存储的访问接口，并包含对象上传、下载、状态查询和预签名 URL 等对象操作能力^[5]。本系统后端使用 MinIO 保存 MITM 模式下捕获的请求体和响应体。已知 Content-Length 的数据可以通过流式上传写入对象存储，未知长度或 chunked 数据先写入临时文件，在获得实际大小后再上传。这样既能保留抓包正文，又避免将大文件完整加载到内存中。

这种设计与本课题的可视化目标相匹配。前端 MITM 页面主要展示会话 ID、父会话 ID、方法、Host、URL、状态码、耗时、请求大小、响应大小和上传状态；当用户确实需要正文内容时，再通过 /api/storage/download 根据对象 Key 获取下载链接或经控制服务代理下载。由此，系统在实时性、内存占用和数据留存之间取得了较好的平衡。

1.3 研究目标与主要内容

1.3.1 系统建设目标

本课题的总体目标是设计并实现一套基于 Go 与 Vue 的可视化网络代理管理系统，使其既能够完成网络代理的基本转发，又能够通过 Web 控制台展示系统运行状态并提供配置管理能力。后端需要支持普通 HTTP 代理、HTTPS CONNECT 隧道和授权 MITM 解析；前端需要支持代理核心连接、实时概览、连接列表、日志查看、MITM 抓包、安全策略、路由配置、存储配置和高级参数配置。

具体而言，系统首先应具备稳定的代理核心。普通 HTTP 请求应能够清理代理头部后转发到目标服务；CONNECT 请

求应能够建立客户端到目标服务之间的双向隧道；MITM 模式应能够在证书可信且授权使用的前提下解析隧道内 HTTP 请求与响应。其次，系统应具备实时观测能力，包括全局上传下载速率、活动连接、历史连接、日志批次、拦截记录、抓包交换记录和用户 IP 到 Host 维度的流量统计。

此外，系统还应具备可配置和可验证的特点。配置项包括代理端口、HTTPS 端口、MITM 开关、路由规则、代理节点、访问控制规则、来源 IP 封禁、公网 IP 和 MinIO 连接信息。保存配置后，后端应能对路由和访问控制进行热重载。测试方面，系统应能够通过单元测试、集成测试、压力测试和 pprof 分析验证主要功能与资源占用情况。

1.3.2 主要研究内容

本文的研究内容首先包括网络代理核心设计。代理核心以 CoreHttpServer 为中心，围绕请求责任链、响应责任链和 CONNECT 处理链组织功能。普通 HTTP 请求通过 MyHttpHandle 转发，CONNECT 请求通过 MyHttpsHandle 处理，MITM 模式下则通过协议嗅探、TLS 握手、HTTP/1.1 请求读取和响应写回完成明文代理处理。该部分是系统所有监控、抓包和策略控制能力的基础。

其次，本文研究规则路由与访问控制设计。路由模块通过代理节点和路由规则决定出站连接使用直连还是二级 HTTP 代理；访问控制模块支持域名后缀、域名关键词、IP 精确匹配和来源 IP 封禁。两类规则都需要支持配置热重载，以便用户在 Web 控制台修改配置后影响后续请求。系统还需要进行代理自环检测，避免把请求重新转发回自身监听端口。

再次，本文研究实时监控与对象存储设计。流量统计模块在请求体和响应体读写过程中更新全局计数、连接计数和用户流量计数；WebSocket Hub 将流量、连接、日志、MITM 交换记录、拦截日志和用户流量按主题批量推送到前端；MinIO 存储模块通过 Body 包装器在主转发链路读取数据的同时进行对象上传。前端控制台则负责将这些运行数据组织为图表、列表、详情面板和配置表单。

1.3.3 论文拟解决的问题

本文拟解决的第一个问题是如何在同一后端程序中统一处理普通 HTTP 请求、CONNECT 隧道和 MITM 明文请求。三类流量在协议形态、连接生命周期和可观测粒度上存在差异：普通 HTTP 请求可以直接读取请求和响应，CONNECT 隧道只能统计加密字节流，MITM 模式则需要在授权前提下重新建立 TLS 会话并解析 HTTP 消息。论文后续将说明系统如何通过处理链和上下文对象统一组织这些流程。

第二个问题是如何在代理转发过程中完成准确且低侵入的流量统计。代理不能为了统计而提前读取完整 Body，否则会影响流式转发和大文件场景。因此，系统采用 Reader 包装方式，在请求体和响应体被正常读取时同步累计字节数，并将统计结果写入全局流量、连接表和用户 Host 聚合结构。该方式能够保持主链路转发语义，同时提供前端实时展示所需数据。

第三个问题是如何在不阻塞主转发链路的前提下留存抓包正文。系统需要把请求体和响应体保存到 MinIO，但上传失败不应导致代理整体不可用。为此，系统将抓包元数据与 Body 对象分离，使用 Pipe 和上传协程处理已知长度数据，使用临时文件处理未知长度数据，并在存储不可用时保留请求响应元数据。最后，论文还将讨论如何将高频运行数据稳定推送到前端，以及如何设计可热更新的路由与访问控制规则。

1.4 论文组织结构

本文共分为七个主要章节，并在正文之后保留致谢、参考文献和附录。第 1 章为绪论，说明课题背景、研究意

义、国内外相关现状、建设目标、主要研究内容和论文结构。第 2 章介绍系统相关技术，包括 HTTP/HTTPS 代理基础、Go 后端技术、Vue 前端技术、REST 与 WebSocket 通信方式以及本文涉及的设计模式。

第 3 章进行系统需求分析，从用户角色、功能性需求、非功能性需求、后端功能、前端功能、数据与接口需求以及可行性等方面明确系统边界。第 4 章给出系统总体设计，说明整体架构、前后端协作流程、后端模块划分、前端模块划分、关键数据结构和安全设计。第 5 章展开系统详细设计与实现，重点分析代理核心、HTTP 与 HTTPS 处理、MITM、路由、访问控制、流量统计、MinIO 存储、WebSocket 控制服务和前端控制台实现。

第 6 章为系统测试与结果分析，围绕测试环境、单元测试、功能测试、性能测试和资源占用分析说明系统验证过程。第 7 章总结系统完成情况和论文工作，归纳系统特色，分析仍然存在的不足，并提出多用户权限、更丰富规则体系和分布式代理节点管理等后续改进方向。

2 系统相关技术

2.1 HTTP/HTTPS 代理相关技术

2.1.1 HTTP 正向代理原理

HTTP 正向代理位于客户端与目标服务器之间，客户端将带有完整目标地址的请求发送给代理服务器，代理服务器再以客户端身份向目标服务器发起请求并返回响应。HTTP 语义规范将请求方法、目标资源、头部字段、状态码和响应内容统一组织为请求响应模型，这为代理服务器解析、修改和转发消息提供了基础^[1]。在正向代理场景中，代理服务器不仅承担网络转发角色，还可以在请求进入和响应返回两个阶段记录连接状态、头部大小、数据体大小和异常信息。

本系统的普通 HTTP 入口由 `CoreHttpServer.ServeHTTP` 根据请求方法分发到 `MyHttpHandle`。当请求不是 `CONNECT` 时，系统会先创建 `Pcontext` 上下文并登记连接信息，再执行请求责任链。若访问控制等处理器没有提前返回响应，则 `RemoveProxyHeaders` 会清理 `Proxy-Connection`、`Proxy-Authenticate`、`Proxy-Authorization` 等代理头部，并通过 `Pcontext.RoundTrip` 调用底层 `http.Transport` 访问目标服务器。响应返回后，系统继续执行响应责任链，将统计包装、抓包记录和响应头写回组织在同一转发流程中。

HTTP 正向代理还需要兼顾透明性与可观测性。透明性要求代理尽量保持原始请求语义，例如不应把服务端使用的 `RequestURI` 字段直接带入客户端请求；可观测性则要求在不提前读取完整请求体的情况下统计流量。本系统采用 `Reader` 包装方式，在请求体或响应体被正常读取时累加字节数，使流量统计与实际转发过程同步发生。这种实现方式适合文件上传、下载等流式场景，避免因统计逻辑破坏 HTTP 请求响应链路。

2.1.2 HTTPS CONNECT 隧道原理

HTTPS 正向代理通常使用 `CONNECT` 方法建立隧道。客户端向代理发送形如 `CONNECT host:port HTTP/1.1` 的请求，代理与目标服务器建立 TCP 连接后向客户端返回连接建立响应，随后客户端与目标服务器之间的 TLS 握手和应用数据都在该隧道中传输^[1]。在普通隧道模式下，代理只能看到加密后的字节流，无法直接解析 HTTP 方法、路径、头部和响应状态。

本系统的隧道处理由 `MyHttpsHandle` 完成。该函数首先使用 `http.Hijacker` 从 Go 的 HTTP 服务端接管底层连接，然后根据配置和 `CONNECT` 处理链选择透传、MITM 或拒绝策略。在透传模式下，系统调用路由拨号器连接目标主机，向客户端写入 `HTTP/1.0 200 Connection established`，再通过双向复制完成客户端连接与目标连接之间

的数据转发。为了便于前端展示，系统还把隧道登记为连接表中的 TUNNEL 或 HTTPS-Tunnel 记录，并用包装后的连接统计上行和下行字节数。

隧道代理的关键特点是连接生命周期长、数据边界不再等同于单个 HTTP 请求。系统在该模式下不生成明文抓包记录，而是把流量统计粒度限定在隧道整体层面。这样既符合 HTTPS 端到端加密的基本语义，也为用户在控制台观察连接建立、流量变化和连接关闭提供了必要信息。

2.1.3 MITM 中间人代理原理

MITM 模式是在授权调试环境下对 HTTPS 流量进行明文观测的一种代理方式。其基本过程是：客户端信任本地代理生成的 CA 证书，代理在客户端侧为目标域名动态签发证书并终止 TLS 连接，同时在服务端侧继续与真实目标服务器建立连接。这样，代理可以在中间位置读取明文 HTTP 请求和响应。由于该模式会突破客户端到服务端之间的加密边界，必须限定在用户明确授权的开发调试、课程演示或内部测试环境中使用。

本系统在 `https.go` 中通过 `TLSConfigFromCA` 和 `signer.SignHost` 为目标主机生成证书配置。`MyHttpsHandle` 在 MITM 分支中先向客户端返回连接建立响应，然后用首字节嗅探判断隧道内数据是 TLS 握手还是明文 HTTP。当识别为 TLS 流量时，系统调用 `tls.Server` 与客户端完成本地 TLS 握手；随后通过自定义 `httpparser.RequestReader` 从解密后的连接中读取 HTTP/1.1 请求。该设计使系统既能处理 HTTPS-MITM，也能在 `HttpMitmNoTunnel` 等配置下处理不经过 CONNECT 的 HTTP-MITM 流量。

MITM 模式为系统提供了更丰富的观测能力。每个被解析出的子请求会创建独立会话，并通过 `StartCapture`、`CaptureRequest` 和响应体关闭时触发的 `SendExchange` 生成抓包交换记录。请求体和响应体在满足条件时会被进一步包装为 `MinIO` 捕获读取器，前端 MITM 页面只接收方法、URL、Host、头部、状态码、耗时、大小和对象 Key 等元数据。这样既保留了明文分析能力，又避免将大体积正文直接推送到浏览器。

2.1.4 HTTP/1.1 长连接与分块传输

HTTP/1.1 支持连接复用，同一条 TCP 连接上可以连续发送多个请求。对于代理服务器而言，这意味着不能只处理服务端标准库已经解析出的第一个请求，还要在 Hijack 后继续从连接中读取后续请求。分块传输、SSE 和 WebSocket 等场景进一步增加了处理复杂度：分块响应没有固定的 `Content-Length`，SSE 需要及时刷新，WebSocket 在升级成功后不再按照普通 HTTP 响应体处理。

本系统针对这些情况实现了专门的处理逻辑。在 HTTP-MITM 与 HTTPS-MITM 流程中，`httpparser.RequestReader` 负责从连接流中循环读取请求；在 HTTP 引擎模式下，`NewRequestReaderWithBufio` 会接收 Hijack 时 `bufio.Reader` 中已经缓存的数据，避免丢失首个请求之后的有效字节。对于响应体被过滤器修改的情况，系统主动删除原有 `Content-Length` 并设置 `Transfer-Encoding: chunked`，再通过 `newChunkedWriter` 写回分块响应。

WebSocket 和 SSE 的处理也体现了代理系统对协议边界的区分。系统通过请求或响应头判断是否存在 `Connection: Upgrade` 与 `Upgrade: websocket`，一旦识别为 WebSocket 升级响应，就跳过 MITM Body 捕获，并使用底层 `io.ReadWriter` 进入双向转发。对于 `text/event-stream` 或分块响应，系统使用带刷新能力的 `Writer`，减少缓冲导致的实时性下降。这些处理保证了代理在面对长连接和流式响应时仍能保持可用。

2.2 后端开发相关技术

2.2.1 Go 网络编程与 net/http

Go 标准库 net/http 同时提供 HTTP 服务端、客户端和传输层能力，适合实现既能接收代理请求、又能向目标服务器发起请求的中间层程序^[3]。本系统的后端程序以 CoreHttpServer 为核心结构体，对外实现 http.Handler 接口，对内持有 http.Transport、请求处理器、响应处理器、CONNECT 处理器、路由器、访问控制器、MinIO 客户端和连接表。该结构使代理核心既是入口服务端，也是出站请求客户端。

系统启动流程集中在 main.go。程序首先读取 PROXY_MAN_CONFIG_PATH 指定的配置文件，初始化日志收集器、MinIO、路由、访问控制和流量监控，然后启动控制服务、HTTP 代理服务 and HTTPS 代理服务。HTTP 代理端口使用普通 ListenAndServe，HTTPS 代理端口使用 ListenAndServeTLS，并通过 TLSNextProto 固定 HTTP/1.1，以保证 CONNECT 隧道依赖的 Hijack 能力可用。

在出站请求方面，http.Transport 的连接池用于复用目标站点连接。系统设置了连接超时、TLS 握手超时、最大空闲连接数和空闲连接回收时间，并在直连拨号器和二级代理拨号器中分别持有独立 Transport。这样，规则路由命中不同出站节点时可以隔离连接池，避免一个节点的 Keep-Alive 连接被错误复用于另一个节点。

2.2.2 Go 并发控制技术

代理系统天然具有高并发特征：多个客户端连接会同时进入代理核心，连接表、流量计数、日志通道、抓包记录和 WebSocket 推送都需要在并发环境下工作。Go 的 sync 包提供互斥锁、读写锁、条件变量和并发 Map 等基础同步原语，其中 sync.Map 适合多协程读写的连接表或订阅表场景^[6]。本系统在后端连接管理和 WebSocket Hub 中大量使用 sync.Map，在访问控制、配置管理、路由器和用户流量聚合中使用 sync.RWMutex 控制读写临界区。

系统的计数类数据主要使用原子操作。全局上传下载流量由 atomic.Int64 保存，连接会话号和 MITM 交换记录编号通过 atomic.AddInt64 递增，累计拦截次数使用 atomic.Uint64。这类变量更新频率高、操作粒度小，使用原子操作可以减少锁竞争。对于用户 IP 到 Host 的流量统计，系统在获得对应统计对象后分别对上行和下行字段进行原子累加，使频繁统计不会阻塞主转发链路。

通道和 goroutine 用于组织异步推送。LogChan、InterceptLogChan 和 GlobalExchangeChan 分别承载日志、拦截日志和 MITM 交换记录；WebSocket Hub 启动多个推送器，按固定周期或批量阈值把数据推送给订阅客户端。为了避免前端未订阅或消费较慢时阻塞代理流程，部分通道写入采用 select 的非阻塞分支，批量推送器则将多条消息合并后再序列化发送。这种设计在实时性和系统稳定性之间取得了平衡。

2.2.3 MinIO 对象存储技术

抓包系统需要处理请求体和响应体，其中正文可能是图片、压缩包、视频片段或大文件。如果所有正文都直接保存在内存中，代理进程和浏览器端都会面临较大压力。MinIO 提供兼容 Amazon S3 的对象存储能力，并提供 Go 客户端 SDK 进行对象上传、下载和状态管理^[5]。本系统将 MITM 抓包正文保存到 MinIO，对前端只传递对象 Key、大小、上传状态和错误信息。

在实现上，myminio.Client.BuildBodyReader 会把原始 Body 包装为 bodyCaptReader。主转发链路读取 Body 时，数据一方面继续流向目标服务器或客户端，另一方面写入 io.Pipe，上传协程从 Pipe 中读取并写入对象存储。对于已知 Content-Length 的数据，系统可以直接按固定大小调用上传接口；对于未知长度或 chunked 数据，系统先写入临时文件，获得实际大小后再上传，并在结束后清理临时文件。

下载能力通过控制服务中的 `/api/storage/download` 暴露。若配置了 `PublicEndpoint`，系统可以生成面向对象存储的访问地址；若没有公开访问端点，则通过控制服务代理下载对象。该方式把抓包元数据展示和大对象获取分离，既保证 MITM 页面加载轻量，也允许用户在需要时取回完整请求体或响应体。

2.3 前端开发相关技术

2.3.1 Vue 3 与 Vite

Vue 3 是以声明式渲染和组件化为核心的前端框架，适合将复杂管理平台拆分为多个可维护的页面组件^[4]。本系统前端使用 Vue 3 组织登录页、仪表盘布局、概览、连接、历史连接、日志、MITM、路由配置、存储配置、高级配置、安全策略和用户监控等页面。每个页面围绕自身业务状态进行组合式编写，公共状态则交由 Store 统一维护。

Vite 是面向现代 Web 项目的构建工具，提供开发服务器和生产构建能力，其开发阶段基于原生 ES 模块提供快速反馈，生产阶段输出优化后的静态资源^[7]。本项目的 `package.json` 中定义了 `npm run dev`、`npm run build`、`npm run preview` 等脚本，`vite.config.js` 负责前端构建配置。开发阶段可单独运行前端服务调试页面，交付阶段则通过 Makefile 将构建产物复制到 Go 控制服务的嵌入目录。

Vue 与 Vite 的组合使系统前端既便于开发，也便于嵌入式部署。前端页面不直接承担代理转发逻辑，而是专注于配置编辑、实时状态呈现和用户操作。后端通过 REST 与 WebSocket 提供数据，前端通过组件、计算属性和响应式状态将这些数据组织为可交互控制台，从而实现代理核心与可视化管理界面的解耦。

2.3.2 Vue Router 与 Pinia

Vue Router 是 Vue 官方客户端路由方案，可在单页应用中将浏览器地址与页面组件对应起来，并通过 RouterView 渲染当前路由组件^[8]。本项目在 `src/router/index.js` 中使用 `createRouter` 和 `createWebHistory` 创建路由，根路径对应登录页，`/dashboard` 对应控制台布局，其下再划分概览、连接、历史连接、日志、MITM、路由配置、高级配置、存储配置和安全策略子路由。

路由守卫用于保证控制台访问前已经连接代理核心。`router.beforeEach` 会读取 WebSocket Store 中的 `isConnected` 状态，当用户未连接控制服务却访问控制台页面时，系统会重定向回登录页。这种设计把连接状态和页面访问边界结合起来，避免用户在无后端数据的情况下进入空白或失效页面。

Pinia 用于集中管理前端共享状态。Pinia 官方文档将 `state` 作为 Store 的核心部分，并通过函数返回初始状态以适配客户端和服务端环境^[9]。本项目的 `websocket.js` Store 保存 WebSocket 实例、连接状态、订阅配置、流量历史、活动连接、历史连接、日志、MITM 交换记录、拦截日志、用户流量和后端配置。各页面通过订阅回调或响应式引用读取这些状态，减少了跨组件传值和重复请求。

2.3.3 图表与虚拟滚动技术

代理管理平台需要展示实时上传速率、下载速率和累计流量。`Chart.js` 提供基于 Canvas 的图表绘制能力，支持在页面中创建折线图、柱状图等常见可视化图形^[10]。本系统在概览页中注册 `Chart.js` 组件，并通过 `new Chart` 创建最近 60 秒上传和下载速率折线图。WebSocket Store 每收到一次 `traffic` 消息，就把数据写入 `trafficHistory` 并通知概览页更新图表。

连接列表、日志列表和 MITM 抓包列表都可能随着系统运行不断增长。如果直接渲染所有 DOM 节点，浏览器会在排序、筛选和滚动时承受较大压力。`TanStack Virtual` 的 Vue 适配器提供 `useVirtualizer`，用于根据滚动容器

和条目估算尺寸只渲染可视区域附近的元素^[11]。本项目在概览连接简表、详细连接页、历史连接页、日志页、MITM 页面和用户监控页中使用虚拟滚动，以提高大列表展示的流畅度。

图表和虚拟滚动共同服务于实时控制台体验。图表让用户快速判断代理当前负载和流量趋势，虚拟滚动让用户在高频连接和大量日志下仍能进行筛选、排序和展开操作。二者都不改变后端代理逻辑，但直接影响系统可观测性的使用效果，是前端技术选型中的重要部分。

2.4 前后端通信与部署相关技术

2.4.1 REST 配置接口

REST 接口适合处理低频、明确请求响应边界的配置读写操作。本系统控制服务在 `StartControlServer` 中注册 `/api/config`、`/api/stats` 和 `/api/storage/download` 等接口。其中 `/api/config` 支持 GET 与 POST：GET 返回当前代理端口、MITM 开关、路由规则、访问控制、来源 IP 封禁、MinIO 和高级行为配置；POST 接收前端提交的配置，并在保存成功后触发路由和访问控制热重载。

`/api/stats` 用于返回进程级统计数据，例如累计拦截次数。前端 WebSocket 连接成功后会调用 `fetchStats` 初始化统计值，避免页面刷新后只能等待下一批推送。`/api/storage/download` 则根据对象 Key 下载 MITM 捕获的请求体或响应体。通过把配置、统计补偿和对对象下载放在 REST 通道中，系统避免把大对象或低频操作全部塞入 WebSocket 消息。

前端 `api.js` 对这些 HTTP 调用进行了简单封装。`fetchConfig` 获取配置，`updateConfig` 使用 JSON 请求体保存配置，`fetchStats` 获取统计值。配置页面调用 `Store` 中的加载和保存逻辑后，后端会完成配置持久化和热重载，前端再根据返回结果刷新页面状态。该流程清晰地区分了“配置命令”和“实时推送”的边界。

2.4.2 WebSocket 实时推送

WebSocket 协议在单个 TCP 连接上提供双向通信能力，适合浏览器与服务端之间的实时消息交换^[2]。代理管理系统中的流量速率、连接表、日志、拦截记录、MITM 交换详情和用户流量都具有实时性，如果完全依赖轮询接口，会增加请求数量并降低状态同步效率。因此，本系统将 WebSocket 作为运行数据推送的主通道。

后端控制服务在 `/start` 路径升级 WebSocket 连接，并通过查询参数 `token` 进行简单认证。客户端连接成功后发送 `subscribe` 消息，声明需要订阅的主题。Hub 根据订阅信息分别推送 `traffic`、`connections`、`log_batch`、`mitm_exchange_batch`、`intercept_log_batch` 和 `user_traffic`。这种主题化设计使页面可以按需开启或关闭数据流，例如用户监控页打开时订阅用户流量，离开页面后取消订阅。

WebSocket 推送还需要考虑批量化和背压。后端日志推送器按 300 毫秒或 200 条日志批量发送，MITM 与拦截日志推送器也按固定周期或数量阈值合并消息。发送前先序列化一次，再分发给满足订阅条件的客户端；每个连接的写操作由 `writeMu` 保护，避免并发写同一 WebSocket 连接。前端 `Store` 接收消息后按类型写入对应状态，并限制日志、抓包记录和历史连接的缓存数量，防止页面内存无界增长。

2.4.3 前端嵌入式部署

为了方便毕业设计演示和单程序部署，本系统将前端构建产物嵌入 Go 后端。Go 的 `embed` 包支持在编译期把文件或目录嵌入程序，嵌入后的 `embed.FS` 可与 `net/http` 等包协同提供静态文件服务^[12]。本项目在 `proxysocket` `/proxy.go` 中使用 `//go:embed dist` 嵌入前端 `dist` 目录，并通过 `http.FileServer` 对外提供静态资源。

控制服务同时承担静态页面、REST 接口和 WebSocket 服务三类职责。handleStaticFiles 会优先尝试返回真实静态资源；当访问路径没有对应文件且不是带扩展名的静态资源时，返回 index.html，由 Vue Router 的 History 模式接管前端路由。这样用户访问 /dashboard/overview 等路径时，即使直接刷新浏览器，也能回到单页应用入口。

构建流程由 Makefile 串联。build-ui 在前端目录执行 npm run build，copy-dist 将构建产物复制到后端的 proxysocket/dist，build 再编译 Linux 可执行文件。该流程把前端构建和后端编译纳入同一交付步骤，最终产物只需要启动一个后端程序，就能提供代理端口、控制页面、配置接口和实时推送服务。

2.5 设计模式相关技术

2.5.1 观察者模式（发布-订阅模式）

观察者模式用于建立一个对象状态变化与多个依赖对象之间的一对多通知关系，发布-订阅可以看作观察者思想在消息主题上的工程化表达^[13]。本系统的实时推送正是这一思想的应用：后端代理核心、日志模块、抓包模块和用户流量模块不断产生运行数据，WebSocket Hub 根据客户端订阅主题把数据分发给不同前端页面。

在后端，WebSocketHub 保存连接与订阅信息，broadcastToTopic 根据主题判断是否应向某个客户端发送消息。StartTrafficPusher、StartConnectionPusher、StartLogPusher、StartMitmDetailPusher、StartInterceptLogPusher 和 StartUserTrafficPusher 分别对应不同数据源。在前端，Pinia Store 接收消息后再调用页面注册的回调，例如概览页订阅流量和连接，日志页订阅日志，MITM 页面订阅抓包记录。通过这种两层观察者结构，代理核心不需要直接依赖具体页面组件。

2.5.2 责任链模式

责任链模式将多个处理步骤按顺序串联，请求在链上传递，每个处理器可以选择继续传递或提前返回结果^[13]。代理服务器非常适合使用这一模式，因为一次请求可能依次经过访问控制、流量统计、MinIO 捕获、路由选择、抓包快照、响应包装和日志记录等环节。如果把这些逻辑集中写在一个函数中，会导致入口函数过长且难以维护。

本系统在 CoreHttpServer 中维护请求处理器、响应处理器和 CONNECT 处理器三类链。filterRequest 顺序执行请求处理器，当访问控制命中规则时可以直接返回 403 响应；filterResponse 顺序执行响应处理器，用于统计响应头、包装响应体并在关闭时发送 MITM 交换记录；CONNECT 处理器则决定隧道是透传、MITM 还是拒绝。路由、访问控制和流量监控均通过 HookOnReq、HookOnResp 或 DoConnectFunc 注入，不需要修改代理入口的核心分发结构。

2.5.3 单例模式

单例模式强调某类全局状态或服务在系统运行期只有一个统一访问入口^[13]。Go 语言项目中不一定通过传统私有构造函数实现单例，更常见的方式是使用包级变量、初始化函数和显式注入控制全局唯一实例。本系统中，全局流量计数器、日志通道、拦截日志通道、MITM 交换通道、用户流量监控器和 WebSocket Hub 都体现了这一思想。

例如，GlobalTrafficUp 和 GlobalTrafficDown 保存进程级累计流量，所有请求和隧道统计都会写入这两个计数器；GlobalExchangeChan 是 MITM 抓包记录进入控制服务的统一通道；GlobalUserTraffic 维护客户端 IP 到 Host 维度的流量聚合；hub 作为控制服务中的唯一 WebSocket Hub，统一管理前端连接和主题订阅。通过集中入口，前端展示的数据来源保持一致，也便于后续测试和问题定位。

单例式全局状态需要注意并发安全。本系统对高频计数使用原子变量，对复杂集合使用锁或 `sync.Map`，对通道写入采用容量限制和非阻塞分支。这样可以避免全局状态成为并发瓶颈，同时保持运行数据在各模块之间的可访问性。

2.5.4 适配器模式

适配器模式用于把不同接口或不同实现包装为统一接口，使上层逻辑可以用一致方式调用^[13]。本系统的规则路由模块使用 `OutboundDialer` 抽象出站方式，`DirectDialer` 和 `HttpProxyDialer` 分别代表直连和二级 HTTP 代理，但都实现 `Dial`、`Name` 和 `GetTransport`。路由器只需要根据规则返回某个 `OutboundDialer`，不必关心具体出站方式。

`RouterRoundTripper` 也是适配器模式的体现。MITM 明文请求最终仍通过 `RoundTrip` 发起出站访问，而路由选择本身发生在拨号器层。`RouterRoundTripper` 把路由匹配和对应节点的 `Transport` 调用适配为统一的 `RoundTrip` 接口，使 MITM 流量也能复用规则路由能力。这样普通请求、MITM 请求和隧道透传可以在不同层次上共享路由逻辑。

流量统计和 `MinIO` 捕获中的 `Reader` 包装同样体现了适配思想。原始 `io.ReadCloser` 只负责读取数据，系统通过 `reqBodyReader`、`respBodyReader` 和 `bodyCaptReader` 为其增加流量累计、父子连接统计、用户流量聚合和对对象存储上传能力，但对外仍表现为可读取、可关闭的 `Body`。上层转发逻辑继续使用 `io.Copy` 或 `Transport.RoundTrip`，无需感知底层已经叠加了统计和存储功能。

3 系统需求分析

3.1 系统总体需求

3.1.1 用户角色与使用场景

本系统面向的主要使用者包括开发调试人员、课程演示人员以及需要观察网络访问状态的网络管理员。开发调试人员关注请求是否正确到达目标服务、响应状态是否符合预期以及接口调用过程中是否出现超时、重定向或异常；课程演示人员更关注系统是否能够清楚展示代理转发、访问控制、路由选择、抓包留存和前端实时推送之间的完整链路；网络管理员则更关注连接来源、目标主机、流量规模和异常访问拦截情况。因此，系统需求不能只停留在代理转发本身，还需要提供可观测、可配置和可操作的管理能力。

典型使用流程可以概括为三个阶段。第一阶段是接入代理，用户在浏览器、操作系统或测试工具中配置 HTTP 代理地址，使普通 HTTP 请求和 HTTPS CONNECT 请求进入代理核心。第二阶段是观察运行状态，用户通过 Web 控制台连接控制服务，查看全局上传下载速率、活动连接、历史连接、日志、拦截记录和用户流量。第三阶段是按需干预，用户可以启用 MITM、配置路由规则、设置访问控制、封禁来源 IP、断开异常连接，并在授权调试场景中下载请求体或响应体进行分析。

由于代理系统处在客户端与目标服务之间，其使用边界也必须在需求阶段明确。普通 CONNECT 隧道只承担加密字节流转发，符合 HTTPS 端到端加密的基本语义；MITM 模式会终止客户端侧 TLS 连接并读取明文 HTTP 内容，因此只能用于用户明确授权的开发、测试和课程演示环境。系统在功能上提供 MITM 能力，但论文需求分析中将其定位为受控调试能力，而不是默认启用的通用流量解析能力。

3.1.2 功能性需求

功能性需求围绕“代理转发、实时观测、规则控制、抓包留存、配置管理和前端交互”展开。后端需要同时处理 HTTP 请求、HTTPS CONNECT 隧道和 MITM 明文请求；前端需要把后端推送的运行数据组织成可理解的页面；配置接口需要支持低频配置读写，WebSocket 则负责高频运行数据推送。HTTP 协议对方法、状态码、头部字段和 CONNECT 语义的定义，是代理功能需求的协议基础^[1]；WebSocket 协议提供的双向通信能力，则支撑前端实时控制台的数据刷新需求^[2]。

表 3-1 系统功能性需求表

编号	需求项	需求说明
F-01	代理转发	支持普通 HTTP 请求转发、HTTPS CONNECT 隧道透传、HTTP-MITM 和 HTTPS-MITM 明文处理。
F-02	实时监控	统计全局流量、活动连接、历史连接、运行日志、拦截日志和用户 IP 到 Host 维度流量。
F-03	规则路由	支持二级 HTTP 代理节点，按域名后缀、域名关键词和 IP 规则选择出站路径。
F-04	访问控制	支持目标域名或 IP 拦截、来源 IP 封禁、HTTP 与 CONNECT 入口统一拦截。
F-05	MITM 抓包	在授权场景下记录请求、响应、状态码、耗时、头部、大小和正文对象 Key。
F-06	配置管理	支持端口、MITM、路由、访问控制、MinIO、PublicIPs 和高级行为开关配置。
F-07	连接操作	支持关闭全部连接、关闭单条连接、按来源 IP 断开连接和清理离线用户流量记录。

从代码结构看，上述需求分别对应不同模块：CoreHttpServer 负责统一入口和处理链组织，Router 负责出站路径选择，AccessController 负责访问控制，GlobalUserTraffic 和全局流量计数器负责统计，WebSocketHub 负责实时推送，ConfigManager 负责配置持久化，MinIO 客户端负责请求体和响应体存储。这种对应关系保证需求能够在后续总体设计和详细实现章节中继续落到具体结构体、函数和数据流上。

3.1.3 非功能性需求

性能需求主要体现在大文件上传下载、高频日志推送和多连接并发场景。代理服务器不能为了统计或抓包而提前把完整请求体、响应体读入内存，否则会破坏流式转发语义并增加内存压力。因此系统要求流量统计和对象存储

捕获都以 Reader 包装方式完成，在主转发链路读取数据时同步累计字节数或写入上传管道。WebSocket 推送也需要批量化处理，避免大量日志或抓包记录逐条推送造成前端渲染压力。

稳定性需求要求系统在连接关闭、客户端中断、目标服务异常、MinIO 不可用和配置变更等情况下保持代理核心可用。代码中 ConnectionInfo 保存 OnClose 回调，便于用户从前端主动关闭连接；MinIO 初始化失败时不会使代理进程退出，而是让存储功能降级不可用；日志、拦截日志和抓包交换记录均采用带容量的通道和非阻塞发送策略，降低辅助功能反向阻塞代理主链路的风险。

可维护性需求要求模块边界清晰。代理核心只负责请求分发和处理链执行，路由、访问控制、流量统计、MinIO 捕获和 WebSocket 推送通过独立模块接入；前端页面按业务功能拆分为概览、连接、日志、MITM、路由、安全、存储和高级配置等视图。可扩展性需求要求未来能够继续增加规则类型、推送主题、页面视图和存储策略，而不需要重写代理入口。安全性需求则要求控制服务认证、MITM 开关、访问控制规则和公网 IP 防自环配置具备明确边界，其中当前控制服务使用简单 token 认证，后续仍需要进一步扩展账号、权限和审计能力。

3.2 后端功能需求

3.2.1 代理转发需求

后端代理转发需求是本系统的基础。系统需要识别进入代理端口的请求类型：当请求方法为 CONNECT 时进入隧道处理流程，当请求不是 CONNECT 时进入普通 HTTP 处理流程。普通 HTTP 代理应能够清理 Proxy-Connection、Proxy-Authenticate、Proxy-Authorization 等代理专用头部，必要时移除 Accept-Encoding，并通过 http.Transport 向目标服务发起请求。该需求对应代码中的 ServeHTTP、MyHttpHandle 和 RemoveProxyHeaders 等实现。

HTTPS CONNECT 隧道需求要求代理通过 Hijacker 接管底层连接，向目标主机建立 TCP 连接后返回 200 Connection established，并在客户端连接与目标连接之间双向复制字节流。CONNECT 是 HTTP 语义中专门用于建立隧道的机制^[1]，代理在透传模式下不能解析加密后的 HTTP 内容，因此系统只统计隧道整体上行和下行字节数，并把连接以 TUNNEL 或 HTTPS-Tunnel 形式登记到连接表中。

MITM 需求要求系统在授权场景下解析隧道内明文请求。系统需要根据配置选择 CONNECT 透传、HTTPS-MITM 或 HTTP-MITM 策略；在 MITM 模式下为目标主机生成证书，完成客户端侧 TLS 握手，再从解密后的连接中循环读取 HTTP/1.1 请求。由于同一条连接可能承载多个请求，需求中还要求支持长连接、分块响应、SSE、WebSocket 升级和 HEAD 响应等特殊场景，避免代理在流式响应或协议升级时错误捕获 Body。

3.2.2 规则路由需求

规则路由用于根据目标地址选择出站路径。系统默认应支持直连，同时允许用户配置多个二级 HTTP 代理节点。每个节点由名称和代理 URL 组成，路由规则通过 Action 字段引用节点名称。规则类型需要覆盖域名后缀、域名关键词和 IP 精确匹配三类常见场景，以满足访问外部服务、特定域名调试和固定地址转发等需求。

路由需求还包括热重载和连接池隔离。用户在前端保存路由配置后，后端应重建拨号器和规则列表，使后续请求立即使用新规则；已有连接不强制迁移，以避免破坏正在进行的传输。每个出站节点需要持有独立 http.Transport，防止不同代理节点之间错误复用连接池。与此同时，系统需要通过 PublicIPs 和监听端口识别代理自环，避免把请求重新转发回自身代理端口造成循环访问。

3.2.3 访问控制需求

访问控制需求包含目标访问控制和来源访问控制两部分。目标访问控制面向请求目的地，支持域名后缀、域名关键词和 IP 精确匹配；来源访问控制面向客户端，支持按来源 IP 封禁。两类规则都需要有独立启停开关和备注字段，以使用户在安全策略页面进行规则管理和演示说明。

访问控制必须覆盖 HTTP 与 CONNECT 两个入口。对于普通 HTTP 请求，访问控制命中后应直接返回 403 响应，并避免继续执行 MinIO 捕获等后续逻辑；对于 CONNECT 请求，访问控制命中后应拒绝建立隧道。系统还需要生成拦截日志并累计拦截次数，使前端概览页和安全策略页能够展示规则命中情况。来源 IP 封禁规则保存后，后端还应关闭当前已命中的活动连接，保证策略不仅影响后续请求，也能处理已经存在的异常连接。

3.2.4 流量监控需求

流量监控需要提供三个层级的数据。第一层是全局流量，包括进程级累计上传、累计下载以及按秒计算的上传下载速率；第二层是连接流量，包括单个连接的请求头、请求体、响应头、响应体和总流量；第三层是用户流量，包括来源 IP 到目标 Host 维度的上行、下行和总量统计。上述数据共同支撑概览仪表盘、连接列表和用户监控页面。

从并发角度看，流量计数属于高频写入路径，需求上必须尽量减少锁竞争。系统使用原子计数器保存全局流量，使用连接上下文中的 TrafficCounter 保存单连接统计，使用 GlobalUserTraffic 按 IP 和 Host 聚合用户流量。后端 WebSocketHub 每秒推送全局流量，每 500 毫秒推送连接快照，每 1.5 秒推送用户流量快照。Go 的并发原语为这类多连接统计和快照读取提供了基础支持^[6]。

3.2.5 抓包与对象存储需求

抓包需求只在 MITM 开启且用户授权的场景下生效。系统应为每个被解析出的明文请求生成一次 HttpExchange 记录，包含交换编号、子会话编号、父隧道编号、时间、请求快照、响应快照、耗时和错误信息。请求快照需要保存方法、URL、Host、头部和请求大小，响应快照需要保存状态码、状态文本、头部和响应大小。对于被本地访问控制拦截、WebSocket 升级或需要跳过的流式响应，系统不应错误生成完整抓包记录。

对象存储需求来源于大 Body 场景。请求体和响应体可能远大于头部元数据，不能直接通过 WebSocket 推送到浏览器。系统需要将正文保存到 MinIO，并在抓包记录中只保留对象 Key、大小、上传状态、Content-Type 和错误信息。MinIO Go Client SDK 提供了面向 S3 兼容对象存储的上传和下载能力^[5]，本系统基于该能力实现流式上传。对于已知 Content-Length 的数据，系统按已知长度上传；对于未知长度或 chunked 数据，系统先写入临时文件，获得实际大小后再上传，上传完成后清理临时文件。

3.2.6 配置管理需求

配置管理需求要求系统能够持久化保存代理端口、HTTPS 端口、MITM 开关、HTTP-MITM 行为、路由规则、代理节点、访问控制规则、来源 IP 封禁、PublicIPs、MinIO 连接信息和高级代理行为开关。配置采用 JSON 文件保存，程序启动时读取配置并补全默认值；如果配置文件不存在，则应写入默认配置，降低首次运行成本。

配置管理还要求线程安全和热更新。后端 ConfigManager 使用读写锁保护配置读写，并在保存时通过临时文件和重命名完成持久化，降低配置写入中断造成文件损坏的风险。前端通过 /api/config 获取和提交配置，后端在 POST 成功后热重载路由和访问控制。Docker 部署场景下，系统还需要识别持久化配置路径，并将历史配置中的

127.0.0.1:9000 MinIO 地址迁移为容器网络中的 minio:9000，避免配置迁移后服务不可达。

3.3 前端功能需求

3.3.1 登录与代理核心连接

前端登录页需要提供控制服务地址、密钥和备注输入能力，并支持保存多个代理核心连接信息。用户连接成功后，前端将 HTTP 基础地址保存为 `apiUrl`，同时根据协议生成 `ws://` 或 `wss://` 的 WebSocket 地址，连接 `/start?token=...` 控制入口。连接建立后，前端立即发送订阅消息，并调用统计接口补齐累计拦截次数等低频数据。

为了避免用户在未连接后端时进入控制台，前端路由需要具备访问守卫。`router.beforeEach` 通过 Pinia Store 中的 `isConnected` 判断连接状态，当目标页面不是登录页且 WebSocket 未连接时重定向到首页。该需求保证控制台页面始终有明确的数据来源，也避免用户误以为代理核心已经在线。

3.3.2 概览仪表盘

概览仪表盘承担系统首屏观测职责，需要展示总上传、总下载、实时上传速率、实时下载速率、安全策略摘要、路由策略摘要、用户流量摘要和活动连接简表。全局流量以最近 60 秒为展示窗口，前端 Store 每收到一次 `traffic` 消息就写入 `trafficHistory`，概览页据此更新折线图，使用户能够快速判断代理当前负载变化。

概览页还应提供从摘要到详情的跳转能力。活动连接简表只展示最关键的连接字段，例如会话 ID、方法、Host、URL、协议、上行和下行流量；当用户需要进一步筛选或关闭连接时，可以跳转到详细连接页。该设计使首屏保持简洁，同时保留进入深层操作页面的入口。

3.3.3 连接与历史连接管理

连接管理页需要展示活动连接的完整列表，并保留父子连接关系。对于 CONNECT 隧道，父连接表示隧道本身，MITM 模式下解析出的明文 HTTP 请求作为子连接展示；对于普通 HTTP 请求，则直接作为独立连接展示。页面需要支持按 ID、Host、URL、方法和状态筛选，支持按流量、开始时间等字段排序，并支持展开子连接查看明细。

连接操作需求包括关闭全部连接、关闭指定连接和按来源 IP 关闭连接。后端收到关闭消息后调用连接表中保存的 `onClose` 回调，释放对应请求上下文或底层 TCP 连接。历史连接页则保存连接从活动列表消失或状态变为 Closed 时的快照，便于用户回看短时间内结束的请求。由于连接数量可能较多，前端需要使用虚拟滚动降低 DOM 渲染压力。

3.3.4 日志管理

日志页面需要接收后端批量推送的 `log_batch` 消息，并提供等级筛选、分类筛选、正则搜索、清空和自动滚动开关。日志分类应覆盖路由匹配、访问控制、流量统计、请求响应、WebSocket、MITM/TLS、网络错误、系统和通用等类别，使用户能够快速定位问题来源。

日志管理还需要控制前端内存增长。Store 中对日志数量设置上限，当日志超过上限时只保留最近记录。后端也采用批量推送策略，按时间或条数聚合日志后再发送。该需求体现了实时控制台对可用性和性能的平衡：日志要足够及时，但不能因为高频输出影响浏览器交互。

3.3.5 MITM 抓包管理

MITM 页面面向授权调试场景，需要展示请求响应交换记录。列表字段包括交换编号、会话编号、父隧道编号、

方法、Host、URL、状态码、耗时、请求大小和响应大小；详情区域需要展示请求头、响应头、请求体上传状态、响应体上传状态、对象 Key、Content-Type 和异常信息。用户可以按会话 ID、父会话 ID、Host、URL 和方法筛选，也可以按状态码、耗时和大小排序。

抓包页面不能直接承载大 Body 内容，因此下载需求通过 /api/storage/download 完成。前端根据抓包记录中的对象 Key 构造下载请求，由后端返回代理下载响应或对象存储访问地址。页面还需要提供清空前端缓存的操作，避免长时间调试后浏览器端保存过多抓包记录。

3.3.6 路由、安全、存储和高级配置

配置类页面承担系统干预职责。路由配置页需要支持代理节点和路由规则的增删改查，规则类型包括域名后缀、域名关键词和 IP，目标动作对应直连或某个二级代理节点。安全策略页需要支持访问控制规则、来源 IP 封禁和实时拦截日志展示，并能够把用户监控页中的封禁操作同步为配置项。

用户监控页需要按来源 IP 聚合展示 Host 明细、上行流量、下行流量和在线状态，并提供断开用户连接、封禁用户和清理离线用户操作。存储配置页需要维护 MinIO endpoint、publicEndpoint、accessKey、secretKey、bucket、useSSL 和 enabled 等字段。高级设置页则负责代理端口、HTTPS 端口、PublicIPs、MITM 开关、HTTP-MITM 行为和请求头处理等全局参数。Vue 组件化模型适合将这些页面拆分为独立视图并复用同一个 Store 状态^[4]。

3.4 数据与接口需求

3.4.1 配置数据模型

配置数据模型以 ServerConfig 为核心。该模型包含代理监听端口、HTTPS 监听端口、日志开关、请求头处理开关、连接保持开关、MITM 开关、HTTP-MITM 开关、PublicIPs、路由配置、访问控制配置和 MinIO 配置。配置模型既要满足后端启动时读取，也要满足前端表单编辑，因此字段需要保持 JSON 可序列化，且新增字段应提供默认值以兼容旧配置。

表 3-2 配置数据模型需求表

数据结构	主要字段与需求
ServerConfig	保存端口、MITM、PublicIPs、路由、访问控制、MinIO 和高级行为开关，是配置接口的顶层模型。
ProxyNode	保存二级 HTTP 代理节点名称和 URL，供路由规则通过 Action 引用。
RouteRule	保存规则编号、匹配类型、匹配值、目标动作、启用状态和备注。
AccessRule	保存目标访问控制规则，支持域名后缀、域名关键词和 IP。
UserBlockRule	保存来源 IP 封禁规则，支持逗号分隔的多个 IP 和独立启停。

MinioConfig	保存对象存储 endpoint、publicEndpoint、凭证、bucket、useSSL 和 enabled。
-------------	--

配置接口采用 REST 风格。GET /api/config 返回当前配置，POST /api/config 接收完整配置对象并保存到磁盘；GET /api/stats 返回进程级累计拦截次数；GET /api/storage/download 根据对象 Key 下载或生成请求体、响应体访问地址。REST 接口适合这类低频、明确请求响应边界的操作，能够与 WebSocket 实时推送形成互补。

3.4.2 WebSocket 消息模型

WebSocket 消息模型采用主题订阅方式。前端连接后发送 subscribe 消息，其中 topics 数组声明需要接收的主题，后端根据订阅状态推送对应数据。WebSocket 作为双向通信协议，适合承载代理运行期持续变化的数据^[2]。系统需要支持的主要消息如表 3-3 所示。

表 3-3 WebSocket 消息需求表

消息类型	数据内容
traffic	推送每秒上传速率、下载速率、累计上传和累计下载。
connections	推送连接快照、连接总数、连接状态和实时流量引用值。
log_batch	批量推送运行日志，并按订阅的日志等级过滤。
mitm_exchange_batch	批量推送 MITM 请求响应交换记录。
intercept_log_batch	批量推送访问控制拦截日志和累计拦截次数。
user_traffic	推送来源 IP、在线状态、总流量和 Host 明细流量。

为了控制消息压力，后端需要对高频数据进行节流或批量化。全局流量按秒推送，连接快照按 500 毫秒推送，日志、拦截日志和 MITM 交换记录按条数阈值或时间阈值合并后推送。前端 Store 接收消息后写入响应式状态，并通过订阅回调通知页面刷新。该模型既满足实时性，也避免多个页面重复建立 WebSocket 连接。

3.4.3 存储数据模型

存储数据模型围绕 MITM 抓包记录展开。HttpExchange 表示一次明文请求响应交换，包含交换 ID、子会话 ID、父隧道 ID、时间戳、请求快照、响应快照、耗时和错误字段。RequestSnapshot 保存请求方法、URL、Host、Header、总大小和请求体对象信息；ResponseSnapshot 保存状态码、状态文本、Header、总大小和响应体对象信息。上述结构直接对应前端 MITM 页面列表和详情字段。

BodyCapture 负责描述对象存储状态，包括对象 Key、实际大小、是否上传成功、Content-Type 和错误信息。系统不要求 WebSocket 直接传输正文内容，而是通过对象 Key 将前端展示与正文下载解耦。这样既能满足抓包分析需求，也能降低代理进程和浏览器端的内存占用。

3.5 可行性分析

3.5.1 技术可行性

从后端看, Go 标准库提供 HTTP 服务端、客户端、TLS、TCP 连接和并发控制能力, 能够支撑代理监听、出站请求、CONNECT 隧道和连接统计等需求^[3]。项目中已经实现 CoreHttpServer、处理链、路由器、访问控制器、MinIO 客户端和 WebSocket 控制服务, 并配套了 HTTP 解析器、配置、访问控制和存储相关测试材料, 说明系统核心功能具备实现基础。

从前端看, Vue 3 的响应式和组件化能力适合构建多页面控制台^[4], Pinia 可以集中保存 WebSocket 状态、连接数据、日志、抓包记录和配置, Vue Router 能够组织登录页与控制台子页面。对象存储方面, MinIO 具备 S3 兼容接口, 能够承担请求体和响应体留存任务^[5]。因此, 本系统所需的主要技术均已成熟生态或项目代码支撑, 技术实现风险可控。

3.5.2 经济可行性

系统依赖的 Go、Vue、Vite、Pinia、Vue Router、Chart.js、TanStack Virtual、Gorilla WebSocket 和 MinIO Go Client 均可以基于开源生态完成开发。部署层面, 系统可以采用一个后端代理进程加一个 MinIO 服务的方式运行, 前端静态资源可嵌入后端程序, 不需要额外购买商业代理平台或专用可视化工具。

对于毕业设计而言, 系统运行主要面向本地或实验环境, 测试目标服务、代理端口、控制服务和对象存储均可在普通开发机器或容器环境中搭建。系统的经济成本主要是开发、测试和少量运行资源成本, 不涉及额外商业授权, 因此具有较好的经济可行性。

3.5.3 操作可行性

操作可行性体现在用户不需要直接修改配置文件, 也不需要长期观察命令行输出。用户通过登录页连接控制服务后, 可以在侧边栏进入概览、连接、历史连接、日志、MITM、路由配置、安全策略、用户监控、存储配置和高级设置等页面。常见操作如保存规则、关闭连接、封禁 IP、下载 Body 和清理缓存都在 Web 页面内完成。

部署后, 控制服务同时提供静态前端资源、REST 配置接口和 WebSocket 实时推送入口。用户只需访问一个控制服务地址即可完成观测和配置, 代理端口则用于浏览器或测试工具接入。由于系统保留默认配置并支持配置页面修改, 初次使用和后续调整成本都较低, 符合毕业设计演示和小规模调试使用场景。

4 系统总体设计

4.1 系统总体架构设计

4.1.1 整体架构

本系统面向代理转发、实时观测、策略配置和抓包留存等需求, 整体上采用“代理核心+控制服务+对象存储+前端控制台”的分层架构。代理核心负责接收客户端进入的 HTTP 请求和 HTTPS CONNECT 请求, 并根据请求类型选择普通转发、隧道透传或授权 MITM 处理; 控制服务负责配置接口、WebSocket 推送和前端静态资源服务; 对象存储用于保存 MITM 模式下可能较大的请求体和响应体; 前端控制台则把配置、连接、日志、抓包和用户流量组织为可操作的页面。该架构把高频转发链路与低频管理链路分离, 使系统既能满足 HTTP 代理的协议语义要求^[1], 也能支持网页端实时交互场景^[2]。

从运行形态看, 后端以 Go 程序为中心。CoreHttpServer 同时承担 HTTP 服务端入口和出站客户端转发的职责, 内部持有 http.Transport、请求处理器、响应处理器、CONNECT 处理器、路由器、访问控制器、连接表和 MinIO 客户端。Go 标准库 net/http 对服务端、客户端与传输层均提供了统一支持^[3], 因此代理核心可以在同一

套结构中完成接入、分发和转发。前端基于 Vue 3 构建，构建后的静态资源通过 Go embed 机制嵌入控制服务，部署后用户访问控制服务地址即可打开管理平台^{[4][12]}。

系统总体组成如表 4-1 所示。表中各层并不是彼此孤立的组件，而是围绕一次代理请求形成闭环：请求先进入代理接入层，经代理核心完成转发和统计，再由控制服务把运行数据推送到前端；当请求处于 MITM 抓包范围内时，正文数据由对象存储层保存，前端只接收元数据和下载入口，从而避免大对象直接进入 WebSocket 消息。

表 4-1 系统总体组成与职责划分

组成部分	主要代码位置	设计职责
代理接入层	main.go、mproxy/http.go、mproxy/https.go	监听 HTTP 代理端口和 HTTPS 代理端口，区分普通 HTTP 请求、CONNECT 隧道和 MITM 明文请求。
代理核心层	mproxy/core_proxy.go、mproxy/ctxt.go、mproxy/hooks.go	维护请求上下文、责任链、出站转发、连接生命周期和异常处理。
控制服务层	proxysocket/proxy.go、proxysocket/hub.go	提供配置 API、统计 API、下载 API、WebSocket 订阅推送和嵌入式静态资源服务。
对象存储层	myminio/config.go、myminio/minioUpload.go、myminio/storage_api.go	保存 MITM 请求体和响应体，生成对象下载地址或通过控制服务转发下载。
前端控制台	proxyui/src/router、proxyui/src/stores、proxyui/src/views	展示概览、连接、日志、抓包、路由、安全、存储和高级配置等页面。

4.1.2 前后端协作流程

前后端协作采用 REST 接口与 WebSocket 双通道设计。低频、明确的配置读写通过 REST 完成，例如前端调用 /api/config 获取或保存 ServerConfig，调用 /api/stats 获取进程级拦截统计，调用 /api/storage/download 获取抓包正文的下载信息。高频、连续变化的运行状态则通过 WebSocket 完成，包括全局流量、活动连接、运行日志、MITM 交换记录、拦截日志和用户流量快照。WebSocket 协议本身适合浏览器与服务端之间的双向通信^[2]，因此该设计能够减少前端轮询压力，也便于后端按照订阅主题精准推送数据。

用户进入系统时，前端登录页根据控制服务地址和密钥连接 /start?token=...，后端 loginHandler 完成简单 token 校验后升级为 WebSocket。连接建立后，Pinia Store 发送 subscribe 消息，声明当前需要订阅的主题和日志级别；后端 WebSocketHub 将连接映射到 Subscription，后续各类推送器只向订阅相应主题的客户端发送消息。Vue Router 负责登录页与控制台子页面切换，并通过路由守卫阻止未连接状态下直接访问控制台页面^{[8][9]}。

配置更新流程体现了前后端闭环。用户在路由、安全、存储或高级配置页面修改配置后，前端通过 POST /api/config 提交完整配置；后端 handleConfig 调用 ConfigManager.UpdateConfig 写入 JSON 文件，随后根据配置

内容热重载路由器和访问控制器。如果来源 IP 封禁规则发生变化，访问控制模块还会扫描当前连接表并关闭已命中规则的活动连接。这样，前端操作不仅改变配置文件，也会立即影响后续转发行为，并通过连接列表、拦截日志和运行日志向用户反馈结果。

4.1.3 运行时数据流

运行时数据流以代理请求为主线。客户端在浏览器、操作系统或测试工具中配置代理地址后，请求进入 HTTP 或 HTTPS 代理监听端口。ServeHTTP 根据方法进行第一次分发：非 CONNECT 请求进入 MyHttpHandle，CONNECT 请求进入 MyHttpsHandle。普通 HTTP 请求需要满足代理请求格式，即目标 URL 为绝对地址；CONNECT 请求则通过 Hijacker 接管底层连接，并由 CONNECT 处理链决定透传、拒绝、HTTPS-MITM 或 HTTP-MITM。

在一次请求内部，系统使用 Pcontext 贯穿请求阶段和响应阶段。请求阶段先创建连接记录和流量计数器，再按注册顺序执行请求责任链。当前主程序中路由、访问控制和流量监控按顺序挂载，其中访问控制命中时会返回 403 或拒绝 CONNECT，避免被拦截请求继续进入流量包装和 MinIO 捕获。若请求未被拦截，系统会移除代理专用头部，并通过默认 Transport 或路由感知的 RoundTripper 向目标服务发起请求。响应返回后，响应责任链继续完成响应头大小统计、响应体包装、抓包记录汇总和响应写回。

管理数据流与代理数据流并行存在。全局流量通过原子计数器累加，连接数据存放在 CoreHttpServer. Connections，日志、拦截日志和 MITM 交换记录分别进入带缓冲的通道。WebSocketHub 中的多个推送器按照不同频率读取这些数据：全局流量约每秒推送一次，连接快照约每 500 毫秒推送一次，用户流量约每 1.5 秒推送一次，日志和抓包记录采用批量方式发送。该设计使主转发链路只需完成轻量级写入或非阻塞发送，前端刷新压力则由推送器统一控制。

4.2 后端模块划分

后端模块划分遵循高内聚、低耦合原则：代理核心只保留请求分发、上下文传递和转发链控制，配置、路由、访问控制、流量统计、抓包存储和 WebSocket 推送分别由独立模块接入。各模块通过结构体字段、接口或 Hook 机制协作，既便于后续详细实现章节逐项展开，也便于系统在后续扩展新的规则类型、推送主题或存储策略时保持较小修改范围。

表 4-2 后端模块划分表

模块	核心对象	主要职责	对应需求
代理核心模块	CoreHttpServer、Pcontext、ReqHandler、RespHandler、HttpsHandler	统一入口分发、上下文传递、责任链执行、HTTP 转发和 CONNECT 连接接管。	F-01、N-01
配置管理模块	ConfigManager、ServerConfig	加载默认配置、读写 JSON 配置、并发安全访问、持久化和热更新触发。	F-06、N-03
	Router、OutboundDialer、DirectDialer、	按规则选择直连或二级代理节点，为普通请求、MITM 请求	

路由模块	HttpProxyDialer	和 CONNECT 隧道提供统一出站能力。	F-03
访问控制模块	AccessController、 AccessRule、UserBlockRule	拦截目标域名或 IP，封禁来源 IP，生成拦截日志并关闭命中连接。	F-04、N-04
流量与连接模块	TrafficCounter、 ConnectionInfo、 GlobalUserMonitor	统计全局、连接和用户维度流量，维护活动连接和历史连接快照。	F-02、F-07
抓包与存储模块	ExchangeCapture、 HttpExchange、 BodyCapture、MinIO Client	生成 MITM 请求响应元数据，流式保存请求体和响应体，并提供下载入口。	F-05
控制服务模块	WebsocketServer、 WebSocketHub、 Subscription	处理控制台登录、REST API、静态资源服务、WebSocket 订阅推送和连接操作命令。	F-02、F-06、F-07

4.2.1 代理核心模块

代理核心模块以 CoreHttpServer 为中心。该结构体对外实现 http.Handler 接口，对内持有 Transport、请求处理器数组、响应处理器数组、CONNECT 处理器数组、路由器、访问控制器、MinIO 客户端和连接表。ServeHTTP 只做请求类型分发，将普通 HTTP 请求和 CONNECT 请求交给不同处理函数，从而让入口逻辑保持清晰。具体转发过程中，普通 HTTP 请求通过 RoundTrip 获得目标响应，CONNECT 请求则在 Hijack 后使用底层 TCP 连接完成双向复制或进入 MITM 明文解析流程。

Pcontext 是代理核心内部的数据桥梁，保存当前请求、响应、客户端 IP、会话号、父上下文、流量计数器、拨号器、证书缓存和抓包状态等信息。请求过滤、响应过滤、路由选择、流量统计和抓包汇总都围绕同一个上下文展开，避免不同模块通过大量全局变量传递临时状态。Hook 机制将函数适配为请求处理器、响应处理器或 CONNECT 处理器，整体上体现了责任链模式的思想^[13]：每个处理器只处理自身关心的条件，主链路按注册顺序组织完整行为。

4.2.2 配置管理模块

配置管理模块负责把系统运行参数从代码中剥离出来。ServerConfig 记录 HTTP 代理端口、HTTPS 代理端口、日志开关、MITM 开关、HTTP-MITM 行为、路由规则、代理节点、访问控制规则、来源 IP 封禁规则、PublicIPs 和 MinIO 配置等字段。程序启动时从 PROXY_MAN_CONFIG_PATH 指定路径加载配置；如果配置文件不存在，则使用 DefaultConfig 创建默认配置并写入磁盘，降低首次运行成本。

ConfigManager 使用读写锁保护配置对象。读取配置时返回当前配置副本，更新配置时使用结构体覆盖内存配置并持久化到 JSON 文件。保存过程先写入临时文件，再通过重命名替换正式文件，降低进程中断导致配置文件损

坏的概率。对于 Docker 场景，模块还会把历史配置中的本机 MinIO 地址迁移为容器网络地址。该模块的设计目标是保证控制台配置修改、后端热重载和持久化文件三者保持一致。

4.2.3 路由模块

路由模块用于解决“同一代理入口如何选择不同出站路径”的问题。系统抽象了 OutboundDialer 接口，并提供 DirectDialer 与 HttpProxyDialer 两类实现。直连拨号器直接连接目标地址，二级代理拨号器先连接指定 HTTP 代理节点，再向该节点发起 CONNECT 请求。每个出站节点持有独立 http.Transport，这样不同代理节点之间不会错误复用连接池，能够避免 Keep-Alive 连接跨节点复用带来的路由混乱。

Router 内部维护规则列表和拨号器映射，规则类型包括域名后缀、域名关键词和 IP 精确匹配。对于 CONNECT 隧道，系统通过 ConnectWithReqDial 调用 Router.RouteDial 选择拨号器；对于 MITM 或普通 HTTP 出站请求，系统通过 Router.RoundTripper 使用命中的拨号器专属 Transport 发起请求。配置保存后，ReloadFromConfig 会在锁外构建新拨号器和新规则，并在锁内进行原子替换，减少热重载对转发线程的影响。

4.2.4 访问控制模块

访问控制模块面向目标地址和来源客户端两个维度设计。目标访问控制使用 AccessRule 描述域名后缀、域名关键词或 IP 规则；来源控制使用 UserBlockRule 描述需要封禁的客户端 IP。运行时，AccessController 将启用的规则编译为匹配器，并通过读写锁在请求线程与配置更新线程之间共享。规则值支持逗号分隔，便于前端一次配置多个目标。

该模块同时覆盖普通 HTTP 请求和 CONNECT 请求。普通 HTTP 请求命中规则时返回 403 响应，CONNECT 请求命中规则时返回拒绝动作并阻止隧道建立。拦截发生后，模块会写入 InterceptLogChan 并递增进程内拦截计数，前端安全策略页可以实时展示拦截记录。对于来源 IP 封禁，配置热更新后还会调用 CloseBlockedConnections，将当前已经处于活动状态且命中封禁规则的连接关闭，使策略不仅影响新请求，也能处理正在进行的异常访问。

4.2.5 流量监控与连接管理模块

流量监控模块按照全局、连接和用户三个层级组织数据。全局上传、下载流量由 atomic.Int64 累加；单连接流量由 TrafficCounter 记录请求头、请求体、响应头和响应体大小；用户流量由 GlobalUserTraffic 按客户端 IP 和 Host 聚合。Go 的并发同步原语适合这类多协程共享状态场景^[6]，系统在高频计数路径上优先使用原子操作，在快照和规则重载等低频路径上使用读写锁。

普通 HTTP 与 MITM 明文请求通过请求体、响应体 Reader 包装实现统计，只有在主转发链路实际读取数据时才累计字节数。CONNECT 隧道使用 tunnelTrafficClient 或 tunnelTrafficClientNoClosable 包装底层连接，分别统计客户端到目标方向和目标到客户端方向的字节数。连接管理模块以 ConnectionInfo 保存会话号、父会话号、Host、方法、URL、协议、开始时间、状态和关闭回调，WebSocket 推送器定期读取连接表，按活动优先、会话倒序生成前端快照。

4.2.6 MITM 抓包与存储模块

MITM 抓包模块只在授权调试场景和配置启用条件下工作。ExchangeCapture 记录一次明文请求的捕获状态，包括开始时间、父隧道会话、请求快照、请求体上传状态、响应体上传状态、错误信息和是否跳过发送。请求经过过滤链后生成 RequestSnapshot，响应体关闭时触发 SendExchange，汇总为 HttpExchange 并写入

GlobalExchangeChan。前端收到的是方法、URL、Host、头部、状态码、耗时、大小和对象 Key 等元数据，而不是完整正文。

正文存储由 MinIO 客户端模块承担。MinIO 提供兼容 S3 的对象存储能力^[5]，系统通过 BodyCapture 记录对象 Key、实际大小、上传状态、Content-Type 和错误信息。对于已知 Content-Length 的正文，系统使用管道直接流式上传；对于未知长度或 chunked 数据，系统先写入临时文件，获得实际大小后再上传并清理临时文件。对于 WebSocket、SSE 和其他流式响应，系统会跳过或特殊处理正文捕获，避免为了抓包破坏长连接语义。

4.2.7 WebSocket 控制模块

WebSocket 控制模块承担管理台入口功能。WebsocketServer 初始化 HTTP ServeMux，挂载 /start、/api /config、/api/stats、/api/storage/download 和静态文件服务；WebSocketHub 保存客户端连接与订阅信息，并启动多个推送器。每个客户端对应一个 Subscription，其中记录是否订阅流量、连接、日志、MITM 详情、拦截日志和用户流量，以及当前日志级别。

为了降低序列化和前端渲染压力，控制模块采用批量推送策略。日志推送器按日志级别预先序列化，避免多个客户端重复编码同一批日志；MITM 交换记录和拦截日志按数量阈值或定时器批量发送；连接推送器限制单次推送的连接数量，并为已关闭连接保留短暂“墓碑期”，让前端能够感知连接关闭状态。除此之外，WebSocket 还承载关闭全部连接、关闭单条连接、断开指定 IP 连接和清理离线用户等控制命令，使前端操作可以直接作用于后端连接表。

4.3 前端模块划分

前端模块围绕“连接控制服务、订阅实时数据、展示业务页面、提交配置变更”四类职责划分。Vue 3 提供组件化和响应式能力^[4]，Vue Router 负责页面组织^[8]，Pinia 负责全局状态共享^[9]。项目没有把所有业务逻辑堆叠在单个页面中，而是将登录、仪表盘布局、概览、连接、日志、抓包、路由配置、安全策略、用户监控、存储配置和高级设置拆分为独立视图，便于维护和扩展。

表 4-3 前端模块划分表

模块	主要文件	设计职责
路由与布局模块	src/router/index.js、src/views /dashboard.vue	维护登录页、控制台父路由和各业务子路由，并通过侧边栏组织页面入口。
状态管理模块	src/stores/websocket.js、src/api /api.js	管理 WebSocket 连接、订阅主题、实时数据缓存、配置加载与保存。
概览展示模块	src/views/Overview.vue	展示全局流量、核心状态、活动连接摘要、路由和访问控制状态。
连接管理模块	src/views/Connections.vue、HistoryConnections.vue	展示活动连接和历史连接，支持筛选、排序、父子连接展示和关闭操作。

日志与抓包模块	src/views/Logs.vue、MITM.vue	展示运行日志、MITM 交换记录、请求响应头部、正文对象下载和异常信息。
配置与安全模块	src/views/RouteConfig.vue、AccessControl.vue、UserMonitoring.vue、StorageConfig.vue、AdvancedConfig.vue	提供路由、安全、用户流量、MinIO 和高级行为开关等配置入口。

4.3.1 路由与布局模块

路由与布局模块由 Vue Router 统一管理。index.js 中定义根路径登录页和 /dashboard 控制台父路由，控制台台下继续挂载概览、连接、历史连接、日志、MITM、路由配置、高级设置、存储配置和安全策略等子页面。安全策略页面再包含访问控制和用户监控两个子路由，使安全相关功能在导航层级上保持集中。路由守卫会检查 Pinia Store 中的连接状态，未连接 WebSocket 时自动回到登录页，避免用户在没有控制服务连接的情况下进入空白控制台。

仪表盘布局模块负责把不同业务页面组织在统一界面中。侧边栏提供稳定导航入口，业务页面通过路由出口渲染。这样的布局方式使用户能够在查看运行状态、调整规则、观察拦截日志和下载抓包正文之间快速切换，也便于后续增加新的运维页面。由于控制台属于操作型系统，布局设计重点放在信息密度、状态可读性和操作闭环，而不是单纯展示型页面。

4.3.2 状态管理模块

状态管理模块集中在 websocket.js。该 Store 维护 WebSocket 实例、连接状态、订阅配置、API 地址、实时流量历史、活动连接、历史连接、日志、MITM 交换记录、拦截日志、拦截次数、用户流量列表和系统配置。前端页面不直接管理底层 WebSocket 协议细节，而是通过 Store 提供的连接、断开、订阅、清理、关闭连接、加载配置和保存配置等方法完成交互。

消息处理采用类型分发。后端推送 traffic 时，Store 将数据追加到最近 60 秒流量历史；推送 connections 时，Store 更新活动连接并把消失或关闭的连接转入历史连接集合；推送 log_batch、mitm_exchange_batch、intercept_log_batch 和 user_traffic 时，Store 分别更新对应页面所需的数据。模块还设置最大日志数、最大抓包记录数、最大拦截日志数和最大历史连接数，避免长时间运行后前端内存无限增长。

4.3.3 页面展示模块

页面展示模块按照业务目标拆分。概览页展示全局上传下载速率、运行状态、活动连接摘要和策略状态；连接页展示活动连接的父子关系、协议类型、来源 IP、目标 Host 和流量数值；历史连接页用于回看已关闭连接；日志页按级别和分类展示运行日志；MITM 页面展示交换记录、请求响应头部、状态码、耗时和正文下载入口；路由、安全、用户监控、存储和高级配置页面则提供低频配置操作入口。

展示层设计强调与后端数据模型对应。例如连接表中的 parentId 使前端能够呈现 CONNECT 隧道与 MITM 子请求的层级关系；抓包记录中的 reqBodyKey 与 respBodyKey 使 MITM 页面能够调用下载接口获取正文；拦截日志中

的 client_ip、target、rule_type 和 rule_value 可以直接映射到安全策略页面；用户流量快照中的 IP、在线状态和 Host 列表则支撑用户监控页面的聚合展示。由此，前端不是孤立的展示层，而是后端总体设计在用户界面上的直接表达。

4.4 关键数据结构设计

关键数据结构是系统总体设计落到代码层面的核心依据。本系统选择围绕请求上下文、连接记录、配置规则和抓包交换四类结构组织数据：请求上下文解决单次请求在多个模块之间传递状态的问题；连接记录解决前端实时展示和主动关闭的问题；配置规则解决路由、安全和存储等运行参数的持久化问题；抓包交换结构解决 MITM 元数据与大正文分离的问题。

表 4-4 关键数据结构设计表

结构名称	关键字段	设计作用
Pcontext	Req、Resp、RoundTripper、ClientIP、Session、parCtx、TrafficCounter、Dialer、exchangeCapture	贯穿请求处理链，连接请求、响应、路由、统计、抓包和异常处理。
ConnectionInfo	Session、ParentSess、Host、Method、URL、ClientIP、Protocol、Status、UploadRef、DownloadRef、OnClose	描述活动连接和历史连接，支撑前端实时展示、父子关系和主动关闭。
ServerConfig	Port、HTTPSPort、MitmEnabled、RouteEnable、ProxyNodes、Routes、AccessRules、UserBlockRules、MinioConfig、PublicIPs	集中保存系统运行配置，支撑配置持久化、热重载和前端配置页面。
HttpExchange	ID、SessionID、ParentID、Time、Request、Response、Duration、Error	描述一次 MITM 请求响应交换，作为前端抓包详情的数据来源。
BodyCapture	ObjectKey、Size、Uploaded、ContentType、Error	记录请求体或响应体对象存储结果，避免大正文直接进入 WebSocket 消息。

4.4.1 代理上下文 Pcontext

Pcontext 是代理核心最重要的运行时上下文。它保存当前请求 Req、响应 Resp、自定义 RoundTripper、客户端 IP、会话号、父上下文、流量计数器、拨号器和 MITM 捕获状态。对于普通 HTTP 请求，Pcontext 从请求进入代理开始创建，到响应体写回完成后结束；对于 MITM 场景，顶层 CONNECT 隧道拥有父上下文，每个明文子请求再创建子上下文，通过 parCtx 关联父隧道统计。

该结构的设计价值在于降低模块耦合。路由模块可以通过 RoundTripper 或 Dialer 改变出站路径，流量模块可以通过 TrafficCounter 累加字节数，抓包模块可以通过 exchangeCapture 汇总请求响应信息，日志模块可以通过 Session 输出带会话号的日志。各模块不需要互相持有对方的复杂状态，只需要围绕同一个上下文读取或写入自身负责的字段。

4.4.2 连接信息 ConnectionInfo

ConnectionInfo 用于描述一个可展示、可操作的连接会话。其字段包括会话号、父会话号、目标 Host、请求方法、URL、远端地址、客户端 IP、协议类型、开始时间、状态、结束时间、上传下载引用和关闭回调。HTTP 请求通常对应一条独立连接记录；CONNECT 透传对应一条隧道连接记录；MITM 模式下，父隧道与子请求通过 ParentSess 建立层级关系，前端可以据此展示“隧道-请求”的结构。

连接记录中的 UploadRef 和 DownloadRef 指向流量计数器中的数值，使连接推送器能够在不复制大量状态的情况下读取最新流量。OnClose 回调用于前端主动关闭连接，当用户关闭全部连接、关闭单条连接或断开指定来源 IP 时，控制服务最终都会调用对应回调释放底层连接。连接关闭后不会立即从连接表删除，而是先标记为 Closed 并保留短时间，便于前端历史连接列表捕获关闭状态。

4.4.3 规则配置结构

规则配置结构主要包括代理节点、路由规则、访问控制规则和来源 IP 封禁规则。ProxyNode 保存节点名称和代理 URL；RouteRule 保存规则 ID、规则类型、匹配值、目标节点、启用状态和备注；AccessRule 保存目标拦截规则；UserBlockRule 保存来源 IP 封禁规则。这些结构都通过 JSON 标签与配置文件和前端表单保持一致，使前端提交的配置能够直接反序列化为后端结构体。

从设计角度看，规则结构把“用户配置”和“运行时匹配器”分开。配置结构保留用户可理解的字段，例如规则类型、匹配值和备注；运行时模块在热重载时再将其转换为函数式匹配器、正则表达式或拨号器映射。这样既能保证配置文件具有可读性，也能让请求处理阶段只面对高效的匹配结构，减少每次请求重复解析配置的开销。

4.4.4 抓包交换结构

HttpExchange 描述一次 MITM 明文请求响应交换，字段包括交换 ID、子会话 ID、父隧道 ID、发生时间、请求快照、响应快照、耗时和错误信息。RequestSnapshot 保存方法、URL、Host、头部、总大小、请求体对象 Key、正文大小、上传状态和 Content-Type；ResponseSnapshot 保存状态码、状态文本、头部、总大小、响应体对象 Key、正文大小和上传状态。

该结构体现了“元数据实时推送、正文对象化存储”的设计思路。WebSocket 消息只携带抓包列表展示和详情查看所需的轻量字段，正文数据通过 MinIO 对象 Key 延迟下载。这样既能让前端快速展示大量抓包记录，也能避免大文件、二进制响应或未知长度响应占用浏览器内存。若捕获过程中发生错误，错误信息会写入对应的 Body 字段或交换级 Error 字段，便于前端展示异常原因。

4.5 系统安全设计

代理系统位于客户端与目标服务之间，既具备转发能力，也可能接触敏感的网络行为数据。因此总体设计中需要明确安全边界：控制服务访问需要基本身份校验；MITM 能力必须限定在授权调试环境；出站路由需要避免错误配置造成代理自环；抓包正文和配置文件也应避免暴露给无关用户。本系统当前更适合本地实验、课程演示和受控内

网环境，生产级安全能力仍需进一步扩展。

4.5.1 控制服务访问控制

控制服务当前使用 `token` 参数进行简单校验。用户在登录页填写控制服务地址和密钥，前端连接 `/start?token=...`，后端 `loginHandler` 校验通过后才允许升级 `WebSocket`。该设计实现成本较低，适合本课题的本地调试和课程演示场景，也能避免完全开放的控制台被随意连接。配置接口、统计接口和下载接口则由同一控制服务提供，前端在连接成功后即可完成管理操作。

需要说明的是，当前 `token` 校验并不等同于完整的生产级权限体系。代码中 `WebSocket` 升级允许跨源请求，控制服务默认也未提供多用户账号、角色权限、操作审计和强制 `HTTPS` 传输。因此在生产环境中，应在本设计基础上增加账号体系、细粒度权限、审计日志、`CSRF` 防护、`TLS` 终止或反向代理访问控制，避免配置接口和连接控制命令被未授权用户调用。

4.5.2 MITM 使用边界

`MITM` 模式会终止客户端侧 `TLS` 连接并读取明文 `HTTP` 内容，因此必须在论文和系统使用说明中明确其授权边界。系统默认配置中 `MitmEnabled` 为关闭状态，用户只有在开发调试、课程演示或内部测试场景中主动开启 `MITM`，并在客户端信任本地 `CA` 证书后，才能查看明文请求和响应。对于普通 `HTTPS` 访问，系统仍可以选择 `CONNECT` 透传模式，只统计隧道流量而不解析内容，这与 `HTTPS` 的端到端加密语义保持一致^[1]。

抓包设计也需要避免扩大数据暴露范围。系统对 `WebSocket`、`SSE` 和部分流式响应跳过正文捕获，防止长连接被错误缓存或打断；请求体和响应体保存到对象存储后，前端只通过对象 `Key` 或控制服务代理下载，避免大正文直接进入实时推送通道。后续若用于更严格环境，还应增加抓包保留周期、敏感字段脱敏、下载权限校验和对象存储访问控制。

4.5.3 代理自环防护

代理自环是部署代理系统时容易出现的风险。当二级代理节点、系统代理环境变量或路由规则错误指向当前代理自身时，请求可能在代理内部反复回到入口端口，造成连接堆积和资源耗尽。为降低该风险，系统设计了 `PublicIPs`、监听端口和本机 `IP` 检测组合：`proxyListenPorts` 提取 `HTTP` 与 `HTTPS` 代理端口，`isSelfLoop` 判断目标地址是否命中监听端口以及本机地址或配置的公网 `IP`。

自环检测被布置在多个出站路径上。默认 `Transport` 的 `DialContext` 会检查目标地址，`DirectDialer` 和 `HttpProxyDialer` 在拨号前也会执行检查，`CONNECT` 隧道底层拨号函数同样包含该逻辑。这样即使请求通过不同路径进入出站连接，也能在靠近拨号的位置阻止明显的回环目标。对于云服务器部署，用户应在控制台中配置真实公网 `IP`，使系统能够识别“通过公网地址访问自身代理端口”的情况，从而降低配置错误带来的循环代理风险。

5 系统详细设计与实现

本章在总体设计的基础上，对系统主要功能的代码实现进行展开说明。系统后端以 `Go` 程序为运行主体，通过 `main.go` 完成配置加载、对象存储初始化、路由与访问控制注册、流量监控注册、控制服务启动以及 `HTTP/HTTPS` 代理监听。代理核心以 `CoreHttpServer` 为入口，使用 `Pcontext` 保存单次请求在转发、统计、路由、访问控制和抓包过程中的上下文状态。前端以 `Vue 3` 页面和 `Pinia Store` 为核心，通过 `REST` 接口完成配置读写，通过 `WebSocket` 接收流量、连接、日志、`MITM` 抓包、拦截日志和用户流量等实时数据。

从实现链路看，本系统没有把所有逻辑集中在单个处理函数中，而是使用请求 Hook、响应 Hook 和 CONNECT Hook 将代理处理过程拆分为多个可组合步骤。普通 HTTP 请求、HTTP-MITM 子请求和 HTTPS-MITM 子请求最终都可以复用请求过滤、出站转发、响应处理、流量统计和对象存储捕获逻辑。CONNECT 隧道则在底层连接接管后，根据配置和处理器返回的动作进入透传、拒绝或 MITM 分支。这样的实现方式既对应了第 4 章中的模块划分，也为后续测试章节中的功能验证和压力测试提供了明确边界。

5.1 后端启动流程实现

5.1.1 主程序初始化

后端启动入口位于 `proxycore/main.go`。程序首先调用 `NewCoreHttpSever` 创建代理核心实例，然后读取环境变量 `PROXY_MAN_CONFIG_PATH` 作为配置文件路径。如果该变量未设置，程序会直接退出并给出日志提示，这说明当前实现要求部署环境显式指定持久化配置文件位置。配置管理器创建完成后，主程序通过 `GetConfig` 读取当前配置，并将配置对象挂载到 `CoreHttpServer` 中，后续路由、访问控制、MinIO 和监听端口都以该配置为基础。

初始化顺序体现了模块依赖关系。日志收集器先通过 `NewLogCollector` 接管代理核心日志输出，便于后续将运行日志推送到前端；`InitMinio` 根据配置决定是否初始化 MinIO 客户端，初始化失败时不会终止代理主程序；`AddRouter` 注册路由引擎；`AddAccessControl` 注册访问控制逻辑；`AddTrafficMonitor` 注册请求和响应流量统计逻辑。访问控制被安排在流量监控之前，能够使被拦截请求尽早返回，避免不必要的 Body 包装和对象存储捕获。最后，主程序创建 `WebsocketServer` 并启动控制服务，再创建 HTTP 与 HTTPS 两个代理监听服务。

`CoreHttpServer` 的构造函数还初始化了默认 Transport。该 Transport 设置 `TLSClientConfig`、连接池数量、空闲连接时间和 `DialContext`。`DialContext` 中包含代理自环检测逻辑，会根据当前配置中的监听端口和 `PublicIPs` 判断目标地址是否指向代理自身，若命中则返回错误。由此可见，自环防护不仅存在于路由拨号器中，也存在于默认出站路径中。

5.1.2 代理监听服务

主程序中分别创建 HTTP 代理服务 and HTTPS 代理服务。HTTP 服务监听配置中的 Port，处理器为同一个 `CoreHttpServer`；HTTPS 服务监听 `HTTPSPort`，同样使用 `CoreHttpServer` 作为 Handler，并通过 `BuildProxyListenerTLSConfig` 构造监听层 TLS 配置。由于 CONNECT 隧道依赖 Hijack 接管底层连接，HTTPS 监听服务显式设置 `TLSNextProto` 为空映射，避免 HTTP/2 自动接管影响 Hijack 行为。

两个代理服务通过 goroutine 并发启动，并共用 `errCh` 接收异常。HTTP 服务调用 `ListenAndServe`，HTTPS 服务调用 `ListenAndServeTLS`。只要任一服务出现非正常关闭错误，主程序就会将错误写入通道并终止运行。该方式使 HTTP 代理入口、HTTPS 代理入口和控制服务可以在同一进程内同时运行，便于本课题进行单程序部署和演示。

实际请求进入代理后，`CoreHttpServer` 的 `ServeHTTP` 根据请求方法进行分发。若请求方法为 CONNECT，则进入 `MyHttpsHandle`；否则进入 `MyHttpHandle`。入口分发逻辑保持简短，具体的协议处理、流量统计、路由选择和抓包留存都在后续处理函数和 Hook 链中完成。这种设计降低了入口函数复杂度，也让 HTTP 与 CONNECT 的处理边界更清晰。

5.1.3 前端资源嵌入

控制服务的前端资源嵌入位于 `proxycore/proxysocket/proxy.go`。文件中使用 `go:embed dist` 将构建后的前端资源嵌入 Go 二进制文件，初始化时通过 `fs.Sub` 获取 `dist` 子文件系统，并读取 `index.html` 作为单页应用回退内容。控制服务启动时挂载静态文件服务，根路径由 `handleStaticFiles` 处理。

`handleStaticFiles` 会先尝试从嵌入文件系统中打开请求路径。若请求的是 `index.html`，则设置 `no-store` 缓存头；若请求的是 `assets` 目录下的静态资源，则设置长期缓存。对于不存在的带扩展名资源，服务返回 404；对于不存在的前端路由路径，服务返回 `index.html`，由 `Vue Router` 在浏览器端继续接管。该处理方式适合 `Vue Router` 的 `history` 模式，用户刷新控制台子路由时不会因为服务端找不到真实文件而进入错误页面。

控制服务还挂载了 `/start`、`/api/config`、`/api/stats` 和 `/api/storage/download` 等接口。这样，单个控制服务同时承担前端页面访问、`WebSocket` 实时通信、配置读写、统计查询和对象下载入口。前端构建产物被嵌入后，部署时不需要额外启动独立 Web 服务，符合本系统“代理核心与控制台一体化”的设计目标。

5.2 HTTP 代理实现

5.2.1 普通 HTTP 转发流程

普通 HTTP 请求由 `mproxy/http.go` 中的 `MyHttpHandle` 处理。函数首先判断配置中的 `HttpMitmNoTunnel`，如果开启则进入基于 TCP 劫持的 HTTP-MITM 引擎；否则使用普通 HTTP 代理流程。普通流程要求请求 URL 是绝对地址，如果不是代理格式请求，则交给 `DirectHandler` 返回错误提示，避免代理服务被当作普通 Web 服务访问。

对于合法代理请求，`MyHttpHandle` 会基于请求上下文创建可取消的 `context`，并提取客户端 IP。随后创建 `Pcontext`，记录代理核心指针、请求对象、客户端 IP、流量计数器和全局递增的 `Session`。系统会立即向 `Connections` 写入一条连接记录，保存会话号、Host、方法、URL、`RemoteAddr`、`ClientIP`、协议、开始时间、状态、上传下载计数引用和 `OnClose` 回调。函数退出时调用 `MarkConnectionClosed`，使前端能够收到连接关闭状态。

请求进入出站转发前会执行 `filterRequest`。该函数顺序调用已注册的请求处理器，例如访问控制、路由注入和流量统计包装。如果请求处理器返回了本地响应，说明请求已被拦截或处理，后续不会再访问目标服务器；如果没有返回响应，则调用 `RemoveProxyHeaders` 清理代理相关头部，再通过 `Pcontext.RoundTrip` 发起出站请求。`RoundTrip` 会优先使用上下文中的自定义 `RoundTripper`，未设置时使用 `CoreHttpServer` 默认 `Transport`。

目标响应返回后，系统先保存原始响应体引用，再执行 `filterResponse`。响应链会完成响应头统计、响应体包装、MITM 交换记录发送回调等操作。随后处理函数复制响应头、写入状态码，并通过 `io.Copy` 将响应体写回客户端。若响应体在过滤链中被替换，函数会删除 `Content-Length`，使 Go 的响应写回机制按 `chunked` 方式处理，避免正文长度与头部声明不一致。

5.2.2 HTTP-MITM 引擎实现

当 `HttpMitmNoTunnel` 开启时，普通 HTTP 请求进入 `myHttpHandleWithEngine`。该模式会通过 `HiJack` 接管客户端连接，把原本由 `net/http` 管理的连接转为可手工读取和写入的 TCP 连接。函数先为该连接创建一个虚拟父隧道，协议标记为 `HTTP_MUX`，用于保存 `Keep-Alive` 连接级别的流量和生命周期；随后对每个 HTTP 请求创建子 `Pcontext` 和子连接记录，协议标记为 `HTTP-MITM`。

HTTP-MITM 引擎中的 `processRequest` 函数负责统一处理首个请求和后续请求。首个请求由 Go 标准库已经解

析完成；后续 Keep-Alive 请求则由 `httpParser.NewRequestReaderWithBufio` 从被劫持连接和缓冲区中继续读取。为了保证首个请求和后续请求行为一致，`processRequest` 内部统一完成 URL 补全、请求上下文创建、`StartCapture`、连接登记、`filterRequest`、`CaptureRequest`、`RoundTrip`、`filterResponse` 和响应写回。

该实现特别处理了被本地拦截的请求。如果 `filterRequest` 返回本地响应，系统会调用 `SetCaptureSkip`，并主动关闭已包装的请求体，避免 `MinIO` 管道等待后续数据。若请求被放行，系统会清理代理头部并调用 `RoundTrip`。响应写回阶段根据 `HEAD`、`1xx`、`204`、`304`、`WebSocket` 和 `Body` 是否被修改等情况处理 `Content-Length` 与 `Transfer-Encoding`。若响应体被过滤链修改，则使用 `newChunkedWriter` 按 `chunked` 格式写回客户端。

HTTP-MITM 引擎解决了普通 HTTP 长连接下“一个 TCP 连接承载多个请求”的观测问题。父连接保存整体流量，子连接保存单次请求响应流量和 MITM 抓包信息，前端可以通过 `parentId` 展示父子层级关系。这样，普通 HTTP Keep-Alive 场景也能获得类似 HTTPS-MITM 的会话结构。

5.2.3 WebSocket 与流式响应处理

`WebSocket` 和流式响应不能按普通响应体方式处理。`MyHttpHandle` 在 `filterResponse` 之后检测响应头是否为 `WebSocket` 握手。由于响应体经过过滤链包装后可能丢失写能力，系统保留了原始响应体 `oriBody`，并在 `WebSocket` 场景中使用 `oriBody` 作为 `io.ReadWriter` 与客户端连接进行双向转发。进入 `WebSocket` 分支后，系统调用 `SetCaptureSkip`，避免把长连接内容作为普通 `Body` 上传到对象存储。

对于 `SSE` 和 `chunked` 响应，普通响应写回使用 `flushWriter`。`flushWriter` 在每次写入后调用 `http.Flusher`，使事件流或分块响应能够及时发送给客户端，而不是被缓冲到较晚时间才输出。对于 HTTP-MITM 和 HTTPS-MITM 手工写回模式，系统在响应体被修改时使用 `newChunkedWriter` 写出标准 `chunked` 编码，并在结束时写入 `chunk` 结束符。

这种处理体现了代理实现中的协议边界：普通响应可以通过流式 `io.Copy` 直接转发；`WebSocket` 需要接管双向读写；`SSE` 和 `chunked` 响应需要及时刷新；被修改的响应体需要重新选择传输编码。系统在不同分支中保持转发语义正确，避免为了抓包或统计破坏长连接协议。

5.3 HTTPS 与 CONNECT 实现

5.3.1 CONNECT 隧道模式

HTTPS 代理请求首先以 `CONNECT` 方法进入 `MyHttpsHandle`。函数提取客户端 IP，创建顶层 `Pcontext`，并通过 `Hijack` 获取客户端底层连接。随后系统在 `Connections` 中登记一条隧道连接，方法为 `CONNECT`，协议初始标记为 `TUNNEL`，保存目标 `Host`、客户端地址、开始时间、流量计数引用和关闭回调。

`CONNECT` 分支通过 `httpsHandlers` 责任链选择处理动作。默认情况下，如果配置 `MitmEnabled` 未开启，则策略为 `ConnectAccept`；如果开启，则默认策略为 `ConnectMitm`。后续 `CONNECT` Hook 可以覆盖该策略，例如访问控制命中时返回 `RejectConnect`，或根据端口、域名返回 HTTP-MITM。该策略选择机制把 `CONNECT` 入口与具体处理动作解耦，使后续扩展其他 `CONNECT` 行为更容易。

在 `ConnectAccept` 透传模式下，系统补全目标端口，通过 `connectDial` 建立目标连接，并向客户端写入 `HTTP/1.0 200 Connection established`。随后代理在客户端连接和目标连接之间进行双向复制。若两端都支持半关闭接

口, 则使用 `copyAndClose` 处理 `CloseWrite` 和 `CloseRead`; 否则使用普通 `copyOrWarn` 处理。隧道流量通过 `tunnelTrafficClient` 或 `tunnelTrafficClientNoClosable` 包装客户端连接统计, 读取客户端数据计为上行, 写给客户端数据计为下行。

由于透传模式下数据多为 TLS 加密字节, 代理不解析内部 HTTP 请求, 只保存隧道级连接和总流量。该模式适合不需要明文抓包的 HTTPS 访问, 也符合 HTTPS 端到端加密的基本语义。前端连接页中看到的是一条 HTTPS-Tunnel 记录, 用户可以观察目标 Host、来源 IP、连接状态和累计流量。

5.3.2 MITM 模式

MITM 模式用于授权调试场景下解析 CONNECT 隧道内的 HTTP 请求。进入 `ConnectMitm` 或 `ConnectHTTPMitm` 分支后, 系统先向客户端写入 200 OK, 再通过首字节嗅探判断客户端后续流量是否为 TLS。代码中以 `TLS Record Handshake` 类型 `0x16` 作为 TLS 流量识别依据; 若识别为 TLS, 则执行服务端 TLS 握手; 若不是 TLS, 则按明文 HTTP 处理。

对于 HTTPS-MITM, 系统会基于目标 Host 动态生成 TLS 配置, 使用 `tls.Server` 与客户端完成握手。握手完成后, 隧道内数据变为可解析的明文 HTTP。系统使用 `http1parser.NewRequestReader` 从连接中连续读取 HTTP/1.1 请求, 每个请求都创建独立的子 `Pcontext`, 继承父上下文中的客户端 IP、父会话和自定义 `RoundTripper`, 并调用 `StartCapture` 初始化抓包状态。

每个 MITM 子请求的处理流程与 HTTP-MITM 引擎基本一致: 登记子连接、执行 `filterRequest`、调用 `CaptureRequest`、放行时执行 `RoundTrip`、执行 `filterResponse`, 再手工写回响应头和响应体。若出现本地拦截, 则设置跳过抓包并关闭请求体; 若出现 `WebSocket` 升级, 则使用原始响应体进行双向转发并跳过抓包; 若响应体被修改, 则使用 `chunked` 编码写回。该复用方式保证 HTTPS-MITM、HTTP-MITM 和普通 HTTP 请求在访问控制、路由、统计和存储行为上保持一致。

父隧道和子请求之间通过 `ParentSess` 建立关联。父隧道体现底层连接生命周期和隧道总流量, 子请求体现每次 HTTP 交换的 URL、方法、状态码、耗时、请求体对象 `Key` 和响应体对象 `Key`。前端 MITM 页面展示的是子请求交换记录, 连接页面则能呈现隧道与子请求之间的层级关系。

5.3.3 证书签发实现

MITM 证书签发逻辑位于 `TLSConfigFromCA`。函数接收本地 CA 证书, 返回一个根据 Host 生成 TLS 配置的函数。执行时先去除 Host 中的端口, 再克隆默认 TLS 配置; 若上下文中存在证书缓存 `certStore`, 则通过缓存获取证书, 否则调用 `signer.SignHost` 使用本地 CA 为目标域名动态签发证书。生成的站点证书被加入 TLS 配置后, 用于与客户端完成 TLS 握手。

该实现使代理可以针对不同目标 Host 生成匹配域名的证书, 从而让客户端在信任本地 CA 的情况下建立 TLS 连接。若客户端没有信任该 CA, 浏览器或测试工具会出现证书告警或握手失败。因此, 论文中需要明确 MITM 模式只能用于授权调试、课程演示和内部测试场景, 不能在未授权网络环境中使用。

证书签发与 MITM 开关共同构成安全边界。系统默认仍可选择 CONNECT 透传, 不要求所有 HTTPS 流量都被解析。只有当用户明确开启 `MitmEnabled`, 并在客户端安装或信任本地 CA 后, 系统才会生成明文请求响应记录和正文对象信息。该设计兼顾了调试能力与 HTTPS 加密访问的基本边界。

5.4 路由引擎实现

5.4.1 出站拨号器设计

路由引擎位于 `mproxy/router.go`，核心抽象是 `OutboundDialer`。该接口定义 `Dial`、`Name` 和 `GetTransport` 三个方法，用于统一直连和二级 HTTP 代理两类出站路径。`DirectDialer` 表示直接连接目标站点，`HttpProxyDialer` 表示先连接配置的二级 HTTP 代理，再通过 `CONNECT` 请求建立到目标地址的隧道。

每个出站拨号器都持有独立的 `http.Transport`。`createBaseTransport` 设置连接池、空闲连接时间、TLS 握手超时和 `Expect-Continue` 超时等参数。独立 `Transport` 的意义在于隔离不同出站节点的连接池，避免 `Keep-Alive` 连接在不同代理节点之间被错误复用。对于普通 HTTP 请求，`RouterRoundTripper` 会调用命中拨号器的 `GetTransport`；对于 `CONNECT` 隧道，`RouteDial` 会调用命中拨号器的 `Dial`。

`HttpProxyDialer` 内部复用 `CoreHttpServer.NewConnectDialToProxy`，根据代理节点 URL 构造二级代理拨号函数。该函数连接二级代理后写入 `CONNECT` 请求，并校验返回状态码是否为 200。这样，上层路由模块不需要关心二级代理的握手细节，只需要通过统一接口完成拨号。

5.4.2 路由规则匹配

`Router` 保存 `Dialers`、`Rules` 和 `Default` 三类运行时结构。`Rules` 中每条规则由匹配条件和目标拨号器名称组成。`MatchRoute` 会从请求中提取目标信息，按规则顺序依次执行条件判断；如果命中规则且目标节点存在，则返回该节点拨号器；如果目标节点不存在，则记录警告日志并回退到 `Direct`；如果没有任何规则命中，也返回默认直连拨号器。

路由条件来自配置中的 `Routes`，支持 `DomainSuffix`、`DomainKeyword` 和 `IP` 三类规则。`DomainSuffixRule` 用于域名后缀匹配，`DomainKeywordRule` 用于域名关键词正则匹配，`IPRule` 用于目标 IP 匹配。路由规则按配置顺序生效，因此用户可以通过调整规则顺序控制优先级。`RouteDial` 和 `RouterRoundTripper` 都会输出路由匹配日志，便于用户在日志页解释某个请求为何走直连或二级代理。

该设计覆盖了普通 HTTP、HTTPS 隧道和 MITM 子请求。普通 HTTP 和 MITM 明文请求经过 `RoundTrip`，因此由 `RouterRoundTripper` 接管出站；`CONNECT` 透传不经过 `RoundTrip`，因此通过 `ConnectWithReqDial` 调用 `RouteDial`。两条路径最终复用同一个 `MatchRoute`，保证路由规则在不同协议入口下行为一致。

5.4.3 路由热重载

配置保存后，控制服务的 `handleConfig` 会调用 `ConfigManager.UpdateConfig` 持久化配置；如果 `RouteEnable` 开启且 `Router` 存在，则调用 `Router.ReloadFromConfig`。`ReloadFromConfig` 的实现采用“锁外构建、锁内替换”的方式：先根据新配置创建 `DirectDialer` 和所有 `HttpProxyDialer`，再把启用的规则转换为运行时 `RoutingRule`，最后在持有写锁的短时间内替换 `Dialers`、`Rules` 和 `Default`。

这种热重载方式降低了运行时阻塞。创建拨号器、解析规则和编译匹配条件可能消耗一定时间，如果在写锁内完成，会阻塞正在匹配路由的请求。当前实现把耗时操作放在锁外，只在替换最终结构时加锁，使配置更新对代理转发的影响更小。

热重载只影响后续请求，不强制打断已经建立的连接。对于长连接或正在传输的大文件，保持原有出站路径可以避免配置保存导致传输中断。若用户希望立即终止旧连接，可通过前端连接管理页关闭全部连接、关闭指定连接

或断开某个来源 IP 的连接。

5.4.4 自环检测

代理自环是代理系统中的重要风险。若二级代理节点、路由规则或系统代理环境变量错误指向本机代理端口，请求可能反复回到自身入口，造成连接堆积和资源耗尽。本系统通过 `loop_check.go` 中的 `proxyListenPorts` 和 `isSelfLoop` 判断目标地址是否命中当前 HTTP/HTTPS 代理监听端口以及本机地址或 PublicIPs。

自环检测被布置在多个出站路径中。CoreHttpServer 默认 Transport 的 DialContext 会检查目标地址；DirectDialer 的 Dial 和 Transport DialContext 会检查目标地址；HttpProxyDialer 的 Dial 也会检查目标地址；MyHttpsHandle 中的底层 dial 函数同样会检查目标地址。多处检查可以覆盖普通 HTTP、CONNECT 透传和 MITM 子请求等不同转发路径。

PublicIPs 用于解决公网部署时的识别问题。如果代理部署在云服务器上，请求可能通过公网地址回到自身，而本机网卡地址不一定能覆盖该公网 IP。用户在高级配置中填写 PublicIPs 后，自环检测可以识别“公网地址加代理端口”的错误目标，从而提前阻止代理循环。

5.5 访问控制实现

5.5.1 目标访问控制

访问控制模块位于 `mproxy/actions.go`，核心对象为 `AccessController`。NewAccessController 创建控制器后会向请求 Hook 和 CONNECT Hook 各注册一个处理函数。普通 HTTP 请求进入请求 Hook 时，控制器根据请求 URL、Host 和客户端 IP 判断是否需要拦截；CONNECT 请求进入 DoConnectFunc 时，控制器根据隧道目标 Host 和客户端 IP 判断是否拒绝建立连接。

目标访问控制规则来自 ServerConfig 中的 AccessRules。ReloadFromConfig 会遍历启用的规则，将 DomainSuffix、DomainKeyword 和 IP 类型转换为运行时 ruleMatcher。DomainSuffix 通过小写后缀判断域名是否命中，DomainKeyword 通过正则表达式匹配，IP 类型通过 IP 集合判断目标是否为指定地址。请求处理阶段只读取已经构建好的 matcher，避免每次请求重复解析配置。

当普通 HTTP 请求命中目标规则时，系统返回 ForbiddenResponse，响应内容为 403 Access Denied；当 CONNECT 请求命中规则时，系统将 `ctx.Resp` 设置为 403 响应并返回 RejectConnect，隧道不会继续建立。该实现使访问控制同时覆盖明文请求和 HTTPS 隧道入口，满足系统对目标域名、关键词和 IP 的拦截需求。

5.5.2 来源 IP 封禁

来源 IP 封禁规则来自 ServerConfig 中的 UserBlockRules。ReloadFromConfig 会遍历启用规则，按逗号拆分多个 IP，并使用 NormalizeIP 进行标准化。合法 IP 被写入 blockedClients 映射，非法值会记录警告日志。请求处理时，控制器优先检查客户端 IP 是否在封禁集合中，如果命中则直接拦截。

客户端 IP 的提取逻辑优先读取 X-Forwarded-For 和 X-Real-IP，无法获取时回退到 RemoteAddr。这样既支持直接使用代理的本地客户端，也兼容前面存在反向代理或转发层的场景。ConnectionInfo 中也保存了 ClientIP，后续关闭用户连接和用户流量统计都复用同一来源 IP 语义。

来源 IP 封禁不仅影响新请求，还能作用于活动连接。CloseBlockedConnections 会遍历 CoreHttpServer.Connections，找到状态为 Active 且来源 IP 命中封禁集合的连接，并调用 CloseAndRemoveConnection。前端用

户监控页可以先观察某个 IP 的 Host 流量，再通过封禁操作写入配置并触发连接关闭，实现从观测到处置的闭环。

5.5.3 拦截日志与连接关闭

每次访问控制命中后，系统都会调用 `pushInterceptLog`。该函数先递增全局 `InterceptCount`，再尝试把 `InterceptLogMessage` 写入 `InterceptLogChan`。写入使用非阻塞 `select`，若通道暂时已满，则不会阻塞代理请求主流程。拦截日志记录客户端 IP、目标地址、规则类型、规则值和时间，供前端安全策略页展示。

WebSocket Hub 中的 `StartInterceptLogPusher` 每 500 毫秒批量发送拦截日志，或当批次达到阈值时立即推送。推送消息类型为 `intercept_log_batch`，同时携带当前累计拦截次数。前端 Store 接收后更新 `interceptLogs` 和 `interceptCount`，概览页与安全策略页可以同步展示安全策略运行效果。

连接关闭能力与拦截日志共同构成访问控制的可观测性和可操作性。拦截日志说明“为什么被拦截”，连接关闭说明“策略已经作用到现有连接”。这对于代理系统尤其重要，因为代理连接可能持续较长时间，单纯阻止新请求不能处理已经建立的隧道或长连接。

5.6 流量统计与连接管理实现

5.6.1 HTTP 明文流量统计

HTTP 明文流量统计由 `AddTrafficMonitor` 注入请求 Hook 和响应 Hook。请求阶段先通过 `GetHeaderSize` 估算请求头大小，写入 `TrafficCounter` 的 `req_header` 和 `req_sum`，同时累加 `GlobalTrafficUp`。若当前上下文存在父隧道，则父上下文的 `req_sum` 也会同步增加请求头大小。随后系统根据客户端 IP 和 Host 获取用户流量统计指针，并把请求头流量计入用户上行。

如果请求存在 Body，系统将原始 Body 包装为 `reqBodyReader`。该 Reader 在每次 Read 时累加 `req_body`、`req_sum` 和 `GlobalTrafficUp`，并同步更新父计数器和用户 Host 统计。响应阶段采用类似方式，先统计响应头并累加 `GlobalTrafficDown`，再把响应 Body 包装为 `respBodyReader`。`respBodyReader` 在读取时累加 `resp_body`、`resp_sum` 和用户下行流量，在 Close 时执行 `onClose` 回调，更新总量并触发 MITM Exchange 发送。

这种实现以实际读取字节数为准，而不是完全依赖 `Content-Length`。对于 chunked、未知长度、压缩响应或流式响应，`Content-Length` 可能不存在或不准确，Reader 包装能更真实地反映代理进程实际转发的数据规模。前端连接页展示的上传和下载数值正是通过 `ConnectionInfo` 中的 `UploadRef` 和 `DownloadRef` 指向这些计数器实现实时更新。

5.6.2 隧道流量统计

CONNECT 透传模式无法看到 TLS 内部的 HTTP 边界，因此系统按隧道方向统计整体字节流。`newTunnelTrafficClient` 针对支持半关闭的连接创建包装对象，Read 从客户端读取数据时累计 `nread` 并累加 `GlobalTrafficUp`，Write 向客户端写数据时累计 `nwrite` 并累加 `GlobalTrafficDown`。对于不支持半关闭的连接，系统使用 `tunnelTrafficClientNoClosable` 完成类似统计。

隧道流量也会写入用户 Host 统计。`MyHttpsHandle` 在透传分支中根据 CONNECT 目标提取 Host，并通过 `GlobalUserTraffic.GetOrCreateStats` 获取当前客户端 IP 和 Host 的统计对象。此后 `tunnelTrafficClient` 在 Read 和 Write 中分别更新用户上行与下行。这样，即使 HTTPS 透传不能解析明文请求，用户监控页仍能看到某个

来源 IP 访问某个 Host 的总体流量。

隧道关闭时，包装连接的 `onClose` 回调会记录上行、下行和总计，并调用 `MarkConnectionClosed`。前端连接推送器保留短时间已关闭连接，使历史连接页能够捕获隧道最终状态和流量。该设计使 CONNECT 透传模式在保护明文内容的同时，仍具备连接级观测能力。

5.6.3 连接生命周期管理

连接生命周期由 `ConnectionInfo` 和 `CoreHttpServer.Connections` 共同维护。`ConnectionInfo` 保存 `id`、`parentId`、`host`、`method`、`url`、`remote`、`clientId`、`protocol`、`startTime`、`status`、`endTime`、上传下载引用和 `OnClose` 回调。普通 HTTP 请求、HTTP-MITM 父连接、MITM 子请求、CONNECT 隧道都会登记连接记录，只是协议标签和父子关系不同。

`MarkConnectionClosed` 不立即删除连接，而是将状态改为 `Closed` 并记录结束时间。`WebSocket` 的 `StartConnectionPusher` 每 500 毫秒扫描连接表，保留短时间墓碑记录，超过保留时间后再物理删除。推送前，连接按 `Active` 优先、`Session` 倒序排序，并限制单次返回数量。这样前端既能看到活跃连接，又能在连接刚结束时捕获关闭状态。

主动关闭由控制服务处理。前端发送 `closeAllConnections` 时，后端遍历所有连接并调用 `OnClose`；发送 `closeConnection` 时，后端先关闭目标连接的子连接，再关闭目标连接本身；发送 `closeUserConnections` 时，后端按来源 IP 匹配连接并关闭。由于连接记录中保存了关闭回调，前端控制命令可以作用到底层 `context` 或 `net.Conn`，实现实际断开。

5.6.4 用户流量聚合

用户流量聚合由 `GlobalUserTraffic` 维护。该对象是一个全局用户监控器，内部以客户端 IP 为第一层键，以 Host 为第二层键，每个 IP 与 Host 组合对应 `UserHostStats`。`UserHostStats` 中的 `Up` 和 `Down` 使用 `atomic.Int64`，适合在高频读写路径上累加流量。

`Snapshot` 方法用于生成前端推送快照。`WebSocket Hub` 在 `StartUserTrafficPusher` 中每 1.5 秒扫描连接表，构建当前在线 IP 集合，再调用 `GlobalUserTraffic.Snapshot` 生成用户列表。快照中包含 IP、在线状态、总上行、总下行和 Host 明细。前端用户监控页据此展示用户维度的网络行为。

该聚合视角与连接列表互补。连接列表适合定位某一条请求或隧道，用户流量页适合分析某个来源 IP 的整体访问行为。当发现某个 IP 流量异常时，管理员可以从用户监控页直接断开该 IP 的所有连接，或写入来源 IP 封禁规则，形成“统计、定位、处置”的完整流程。

5.7 MITM 抓包与 MinIO 存储实现

5.7.1 抓包快照生成

MITM 抓包数据结构位于 `mproxy/mitm_exchange.go`。`HttpExchange` 表示推送给前端的一次请求响应交换，包含交换 ID、子会话 ID、父会话 ID、时间、请求快照、响应快照、耗时和错误信息。`RequestSnapshot` 保存方法、URL、Host、Header、总大小和请求体对象信息；`ResponseSnapshot` 保存状态码、状态文本、Header、总大小和响应体对象信息。

每个 MITM 子请求创建 `Pcontext` 后会调用 `StartCapture`，初始化 `ExchangeCapture` 并保存父会话 ID。请求

经过 `filterRequest` 后，系统调用 `CaptureRequest` 保存请求方法、URL、Host 和头部。响应体关闭时，`respBodyReader` 的 `onClose` 回调会调用 `SendExchange`，系统从 `TrafficCounter`、请求体 `BodyCapture` 和响应体 `BodyCapture` 中汇总数据，生成 `HttpExchange` 后写入 `GlobalExchangeChan`。

`SendExchange` 中设置了多个保护条件：`MitmEnabled` 关闭时不发送；捕获状态为空时不发送；`skipSend` 为 `true` 时不发送；已发送过的记录不会重复发送。被本地拦截、WebSocket 升级或不适合捕获的流式场景都会设置跳过标记。这样可以避免前端出现没有真实响应体或不完整的误导性抓包记录。

5.7.2 请求体和响应体捕获

正文捕获由 `myminio/minioUpload.go` 中的 `bodyCaptReader` 完成。`AddTrafficMonitor` 在请求体和响应体的流量统计包装外层，根据条件再套一层 `MinIO` 捕获 `Reader`。只有在 `exchangeCapture` 存在、`MitmEnabled` 开启且 `MinioClient` 可用时才启用捕获；否则只进行流量统计，不上传正文。

`BuildBodyReader` 首先判断 `MinIO` 是否启用以及 `Content-Type` 是否需要跳过。`text/event-stream`、`websocket`、`multipart/x-mixed-replace` 等流式类型会被跳过，避免无限流或长连接被错误上传。对于需要捕获的正文，函数创建 `io.Pipe`，并生成 `BodyCapture`，`BodyCapture` 中保存对象 `Key`、`Content-Type`、上传大小、上传状态和错误信息。随后启动上传协程 `uploadToMinIO`。

`bodyCaptReader` 的 `Read` 方法先从内层 `Reader` 读取数据，再把读取到的字节写入 `PipeWriter`。主转发链路仍然按原节奏读取请求体或响应体，上传协程则从 `PipeReader` 中获得同一份数据并写入 `MinIO`。`Close` 方法关闭 `PipeWriter`，等待上传协程完成，再关闭内层 `Reader`。这样正文捕获是转发链路的旁路，不需要把完整正文一次性读入内存。

5.7.3 大文件与 chunked 上传处理

`uploadToMinIO` 根据 `contentLength` 选择不同上传策略。若 `Content-Length` 已知且大于等于 0，系统直接调用 `PutObjectWithSize`，以指定大小将 `PipeReader` 中的数据流式上传到 `MinIO`。若 `Content-Length` 为未知值，例如 `chunked` 请求或响应，系统调用 `uploadViaTempFile` 先把 `PipeReader` 内容写入临时文件，得到实际大小后再上传。

临时文件路径位于 `myminio/tmp`。`uploadViaTempFile` 会创建临时目录和临时文件，使用 `io.Copy` 写入数据，随后将文件指针重置到开头，再调用 `PutObjectWithSize` 上传。函数通过 `defer` 保证先关闭临时文件，再删除临时文件，特别考虑了 Windows 下文件句柄未释放导致删除失败的问题。该策略在未知长度场景下避免了把大正文全部保存在内存中。

项目测试材料中包含 `TestUploadViaTempFileCleansTempFile`、`TestBodyCapture_LargeFile`、`BenchmarkUpload_Chunked` 和 `goroutine` 泄露检查等用例或基准。它们分别验证临时文件清理、大文件捕获、未知长度上传和并发关闭场景，支撑本章关于大文件与 `chunked` 上传可靠性的实现说明。

5.7.4 下载接口实现

对象下载接口由 `myminio/storage_api.go` 中的 `HandleDownload` 提供，并挂载到控制服务的 `/api/storage/download`。前端传入 `key` 参数后，接口首先检查 `Key` 是否为空，并根据对象 `Key` 解析建议文件名。若请求带有 `proxy=1`，接口进入控制服务代理下载分支；否则先检查对象是否存在，再生成浏览器可访问的下载 URL。

下载 URL 的生成优先使用 `PublicClient`。如果用户配置了 `PublicEndpoint`，系统生成对象存储直连的预签名 URL；如果未配置 `PublicEndpoint`，则回退到控制服务自身的代理下载 URL，并附带 `key` 和 `proxy=1` 参数。代理下载分支会调用 `StatObject` 获取对象信息，再调用 `GetObject` 读取对象内容，设置 `Content-Type`、`Content-Disposition`、`Content-Length` 和 `Cache-Control` 后通过 `io.Copy` 写回浏览器。

该接口实现了“实时推送元数据、按需下载正文”的抓包方式。`WebSocket` 消息只携带 `BodyKey`、`BodySize`、`BodyUploaded` 和 `BodyError` 等轻量字段；只有用户在 MITM 页面点击下载时，前端才请求正文。这样既降低了 `WebSocket` 消息体积，也避免浏览器在长时间抓包过程中保存大量二进制内容。

5.8 WebSocket 控制服务实现

5.8.1 连接认证与订阅

`WebSocket` 控制服务由 `WebsocketServer.StartControlServer` 启动。服务在 `/start` 路径挂载登录处理器，`loginHandler` 会读取 URL 中的 `token` 参数，并与服务端 `Secret` 比对。不一致时返回 `401 Unauthorized`，通过后才进入 `handleWebSocket` 完成协议升级。当前实现使用简单 `token` 适合本地实验和毕业设计演示，后续若用于生产环境还需要扩展账号、权限和 `HTTPS` 控制通道。

连接升级后，`handleWebSocket` 创建 `Subscription`，默认日志级别为 `INFO`，并把 `WebSocket` 连接和订阅对象保存到 `Hub` 的 `clients` 中。前端发送 `action` 为 `subscribe` 的消息后，`Hub.updateSubscription` 根据 `topics` 数组更新 `Traffic`、`Connections`、`Logs`、`MitmDetail`、`InterceptLogs` 和 `UserTraffic` 等布尔字段，同时保存 `LogLevel`。

订阅机制使后端可以按页面需要推送数据。概览页通常需要 `traffic` 和 `connections`；日志页需要 `logs`；MITM 页面需要 `mitm_detail`；安全策略页需要 `intercept_logs`；用户监控页需要 `user_traffic`。当前前端 `Store` 默认开启部分订阅，并在进入或离开用户监控、安全策略等页面时调整订阅状态，减少不必要的数据传输。

5.8.2 实时数据推送器

`WebSocket Hub` 启动后会同时启动多个推送器。`StartTrafficPusher` 每秒读取 `GlobalTrafficUp` 和 `GlobalTrafficDown`，计算与上一时刻的差值并归一化为 `bytes/s`，同时推送累计上传和累计下载。`StartConnectionPusher` 每 500 毫秒扫描连接表，生成活动连接和短时间关闭连接的快照，并推送给订阅 `connections` 的客户端。

`StartLogPusher` 从 `LogChan` 中批量收集日志，最多 200 条或每 300 毫秒发送一次。`StartMitmDetailPusher` 从 `GlobalExchangeChan` 中收集 `HttpExchange`，达到 100 条或每 500 毫秒发送一批。`StartInterceptLogPusher` 从 `InterceptLogChan` 中收集拦截日志，达到阈值或每 500 毫秒发送一批。`StartUserTrafficPusher` 每 1.5 秒生成用户流量快照并推送。

这些推送器的频率不同，体现了数据特征差异。流量速率适合按秒刷新；连接和拦截信息变化较快，采用 500 毫秒级推送；日志和抓包记录采用批量机制；用户流量聚合对实时性要求略低，采用 1.5 秒推送。通过分频处理，系统在保证可观测性的同时降低了前端渲染和网络传输压力。

5.8.3 批量发送与背压处理

批量发送是控制服务可靠性的关键。日志推送器不会每条日志单独编码发送，而是先构建批次，并根据客户端

日志等级预序列化不同级别的消息。例如 WARN 级别客户端只接收 WARN 和 ERROR，INFO 级别客户端接收 INFO、WARN 和 ERROR。这样可以避免多个客户端重复过滤和重复 JSON 编码同一批日志。

broadcastToTopic 在发送前先把消息序列化为字节数组，再遍历订阅了对应主题的客户端。真正写 WebSocket 时通过 Subscription.writeMu 加锁，并设置写超时，避免多个推送器同时写同一个连接造成帧交错。如果写入失败，Hub 会删除客户端并关闭连接。该处理避免慢客户端长期占用推送路径。

通道写入侧也尽量采用非阻塞或缓冲通道。例如 MITM Exchange 和拦截日志在写入通道时使用 select default，防止前端无人订阅或消费速度不足时阻塞代理转发主链路。这样的背压策略保证控制台是代理系统的观测与管理层，而不是影响代理转发的瓶颈。

5.9 前端控制台实现

5.9.1 登录页实现

前端登录页由 proxyui/src/views/loginView.vue 实现，核心连接逻辑在 src/stores/websocket.js。用户输入控制服务地址和密钥后，connect 函数根据 http 或 https 自动选择 ws 或 wss 协议，并拼接 /start?token=... 作为 WebSocket 地址。连接打开后，Store 将 isConnected 设置为 true，立即发送 subscribe 消息，并通过 loadStats 获取累计拦截次数。

WebSocket 关闭后，Store 将连接状态置为 false，并在 5 秒后尝试重连。错误事件也会更新连接状态。登录成功后页面进入控制台，连接地址保存在 apiUrl 中，后续配置接口和下载接口都以该地址作为基础。该实现使前端不需要写死控制服务地址，适合本地、局域网或远程代理核心切换。

路由守卫位于 src/router/index.js。除了登录页以外，访问其他路由时都会检查 Pinia Store 中的 isConnected。若未连接 WebSocket，则重定向到登录页。这样可以避免用户直接打开控制台子页面时看到空数据或错误操作入口，也使“先连接代理核心，再进入控制台”的使用流程更加明确。

5.9.2 仪表盘布局实现

控制台父布局由 dashboard.vue 实现。该页面提供左侧导航、代理核心状态和主内容区。导航项包括概览、详细连接、历史连接、日志、MITM 抓包、路由配置、安全策略、存储配置和高级设置，子页面通过 RouterView 渲染。侧边栏使用 lucide-vue-next 图标，状态区展示当前连接地址和在线离线状态。

dashboard.vue 中的 coreStatusText 根据 Store 的 isConnected 和 apiUrl 计算。刷新配置按钮调用 wsStore.loadConfig，用于从后端重新拉取当前配置。导航高亮通过当前 route.path 判断，安全策略相关子路由统一识别为安全分组。该布局把高频观测页面和低频配置页面集中在同一操作台中，符合系统可视化管理目标。

从页面组织上看，前端没有把所有功能堆叠在一个页面，而是按照业务模块拆分为独立视图。这样既便于用户按任务切换，也便于后续维护。连接、日志和 MITM 页面主要消费 WebSocket 实时数据；路由、安全、存储和高级设置页面主要消费 REST 配置接口；概览页则汇总多个数据源形成全局状态。

5.9.3 概览页实现

概览页由 Overview.vue 实现，数据主要来自 Store 中的 trafficHistory、connections、interceptCount、userTrafficList 和 config。Store 在收到 traffic 消息时，把速率和累计流量追加到最近 60 秒历史数组；概览页基于该数组展示上传和下载速率曲线。项目依赖中包含 Chart.js，用于绘制实时流量图表。

概览页还展示总上传、总下载、实时上传速率、实时下载速率、安全策略状态、路由策略状态和用户流量摘要。活动连接摘要来自 connections 消息，页面只展示部分连接并提供跳转详细连接页的入口。这样的设计使用户进入控制台第一屏时就能判断代理核心是否在线、当前是否有流量、策略是否启用以及是否存在异常活跃连接。

概览页并不承担所有详细操作，而是作为状态总览入口。用户需要查看单连接细节时进入连接页，需要查看请求响应明细时进入 MITM 页面，需要调整规则时进入路由或安全页面。这种分层设计减少了首页信息过载，也使不同页面职责更加明确。

5.9.4 连接页与历史连接页实现

连接页由 Connections.vue 实现，数据来自 Store 的 connections。后端推送的每条连接包含 id、parentId、host、method、url、remote、clientId、protocol、startTime、status、up 和 down 等字段。前端可以根据 parentId 构建父子关系，将 CONNECT 隧道与 MITM 子请求关联展示。页面支持筛选、排序、展开和关闭连接。

历史连接页由 HistoryConnections.vue 实现。Store 的 handleConnections 会比较上一次活动连接列表和本次推送列表，如果旧连接消失或新状态为 Closed，就将连接快照写入 historyConnections，并用 historyConnectionsSet 去重。历史连接最多保留固定数量，避免长时间运行导致前端内存不断增长。

详细连接和历史连接都适合使用虚拟滚动。项目依赖 @tanstack/vue-virtual，连接、历史连接、日志和 MITM 等大列表页面可以减少实际 DOM 节点数量，提高高频推送场景下的渲染性能。前端关闭连接时，会通过 WebSocket 发送 closeConnection 或 closeAllConnections，后端执行相应 OnClose 回调。

5.9.5 日志页实现

日志页由 Logs.vue 实现，数据来自 Store 的 logs。后端推送 log_batch 消息后，Store 的 handleLogBatch 会为每条日志补充分类和前端 id，再合并到日志数组。日志最多保留 MAX_LOGS 条，超过后只保留最新记录。该策略避免浏览器长时间运行后因日志积累过多而占用过大内存。

日志页支持日志等级、分类和消息内容搜索。后端日志分类包括路由匹配、访问控制、流量统计、请求响应、WebSocket、MITM/TLS、网络错误、系统和通用等类型。前端筛选和搜索主要在本地完成，适合用户快速定位某个会话号、Host、错误信息或规则命中记录。

日志页是系统调试的重要辅助页面。路由配置是否命中、访问控制为何拦截、MinIO 是否初始化失败、WebSocket 是否断开、TLS 握手是否失败等信息都可以通过日志观察。与概览页相比，日志页提供了更细粒度的运行解释能力。

5.9.6 MITM 页面实现

MITM 页面由 MITM.vue 实现，数据来自 Store 的 mitmExchanges。后端推送 mitm_exchange_batch 后，Store 的 handleMITMExchangeBatch 会把每条 HttpExchange 转换为扁平化对象。buildExchangeObject 会提取会话 ID、父会话 ID、时间、耗时、错误、请求方法、URL、Host、请求头、请求大小、响应状态码、响应头、响应大小，以及请求体和响应体的对象存储信息。

页面展示时，用户可以按会话、父会话、Host、URL、方法、状态码和耗时筛选或排序。展开记录后可以查看请求头和响应头，也可以看到 reqBodyKey、respBodyKey、BodySize、BodyUploaded 和 BodyError 等对象存储状

态。若正文上传成功，页面通过 `apiUrl` 调用 `/api/storage/download` 获取下载链接或代理下载内容。

MITM 页面体现了后端抓包设计的核心思想：实时列表只展示元数据，正文按需下载。这样即使系统处理大文件上传下载，前端也不会因为 `WebSocket` 携带二进制正文而卡顿。用户仍然可以在需要时下载请求体或响应体，用于进一步分析接口调试问题。

5.9.7 配置页面实现

配置页面包括 `RouteConfig.vue`、`AccessControl.vue`、`UserMonitoring.vue`、`StorageConfig.vue` 和 `AdvancedConfig.vue`。它们通过 `src/api/api.js` 调用 `/api/config` 完成读取与保存。`Store` 中的 `loadConfig`、`saveConfig` 封装了配置获取和提交，保存成功后会重新加载后端配置，保证页面表单与实际运行状态一致。

路由配置页维护 `RouteEnable`、代理节点和路由规则。用户可以新增或删除二级代理节点，设置域名后缀、域名关键词和 IP 规则，并指定目标节点。安全策略页维护 `AccessEnable`、目标访问控制规则和来源 IP 封禁规则，同时展示实时拦截日志。用户监控页基于 `user_traffic` 快照展示客户端 IP 与 Host 明细，并提供断开连接、封禁 IP 和清理离线用户等操作。

存储配置页维护 `MinIO endpoint`、`publicEndpoint`、`accessKey`、`secretKey`、`bucket` 和启用状态。高级设置页维护 HTTP 代理端口、HTTPS 代理端口、公网 IP、MITM 开关和代理行为开关。保存配置后，后端会持久化 JSON，并触发路由热重载、访问控制热重载和来源 IP 命中连接关闭，使前端配置能够直接影响代理运行行为。

5.10 异常处理与可靠性实现

5.10.1 连接关闭处理

普通 HTTP 请求使用 `context.WithCancel` 创建请求上下文，并把 `cancel` 函数放入 `ConnectionInfo`。`OnClose`。函数退出时通过 `defer` 调用 `cancel` 和 `MarkConnectionClosed`，保证请求结束后上下文可以释放。前端主动关闭连接时，后端调用对应 `OnClose`，使正在进行的请求尽快结束。

CONNECT 隧道关闭处理更复杂。透传模式下，系统根据连接是否支持半关闭选择 `copyAndClose` 或 `copyOrWarn`。支持半关闭时，一侧复制结束后调用 `CloseWrite` 和 `CloseRead`，使 TCP 连接能够较自然地完成关闭；不支持半关闭时，则使用普通关闭处理不可控的连接中断错误。`WebSocket` 连接也通过 `proxyWebsocket` 完成双向转发，并在连接结束时释放底层连接。

连接状态变更采用“标记关闭、延迟删除”的策略。`MarkConnectionClosed` 只更新状态和结束时间，`StartConnectionPusher` 在墓碑保留期后才删除记录。该策略保证前端可以看到连接关闭事件，并把关闭前的最终快照写入历史连接列表，避免连接刚关闭就从前端视图中突然消失。

5.10.2 请求体关闭处理

请求体和响应体包装链是本系统可靠性实现的重点。请求体可能同时被流量统计 `Reader` 和 `MinIO` 捕获 `Reader` 包装；响应体关闭时还会触发流量统计完成和 `SendExchange`。若请求被访问控制拦截，目标服务器不会读取请求体，此时如果不主动关闭包装后的 `req.Body`，`MinIO` 上传协程可能一直等待 `Pipe` 数据结束。

为解决该问题，HTTP-MITM 和 HTTPS-MITM 的本地拦截分支都会调用 `SetCaptureSkip`，并在 `req.Body` 不为空时主动关闭请求体。这样 `PipeWriter` 会关闭，上传协程能够退出，临时文件也能清理。响应体关闭则集中在 `respBodyReader.Close` 中处理，`onClose` 只执行一次，避免重复发送 `Exchange`。

项目中存在 goroutine 泄露检查相关测试，例如 test/gotest/goroutine_test.go 使用 go.uber.org/goleak 验证 Body Reader、真实网络已知长度上传和 chunked 上传场景下协程能够回收。测试材料还覆盖并发上传、并发关闭和 MinIO 临时文件清理，说明实现不仅关注功能成功，也关注长时间运行时资源是否可释放。

5.10.3 存储异常降级

MinIO 是抓包正文留存能力，不应成为代理转发的硬依赖。InitMinio 中如果配置未启用，会记录“MinIO 存储未启用”并保持 MinioClient 为空；如果创建客户端失败，也只记录错误，不使用 fatal 终止程序。后续 AddTrafficMonitor 在判断 MinioClient 为空时不会启用正文捕获，只保留流量统计和 MITM 元数据。

下载接口也包含降级处理。控制服务挂载 /api/storage/download 时会先判断 MinioClient 是否为空，若为空则返回 JSON，状态码语义为存储未启用。HandleDownload 内部还会处理缺少 key、对象不存在、生成下载链接失败和读取对象失败等情况，并给出明确消息。前端据此展示失败原因，而不是把错误伪装成成功下载。

该降级策略保证系统在未配置对象存储时仍能完成 HTTP/HTTPS 代理、CONNECT 透传、路由、访问控制、流量监控和前端控制台演示。当 MinIO 配置正确后，系统再提供请求体和响应体的完整保存与下载能力。这样既降低了首次运行门槛，也增强了系统在异常配置下的可用性。

5.11 设计模式在系统中的应用

5.11.1 观察者模式（发布-订阅模式）的应用

观察者模式在本系统中主要体现为 WebSocket 发布-订阅结构。后端的全局流量计数器、连接表、日志通道、MITM 交换通道、拦截日志通道和用户流量监控器都是被观察的数据源；WebSocket Hub 维护客户端连接和 Subscription，根据客户端订阅主题决定是否推送对应数据。

前端也存在一层观察者结构。Pinia Store 接收 WebSocket 消息后更新 trafficHistory、connections、logs、mitmExchanges、interceptLogs 和 userTrafficList；页面组件通过响应式状态或订阅回调刷新视图。后端不直接感知具体页面，页面也不直接读取底层代理结构，中间通过主题消息和 Store 状态解耦。

这种设计适合代理系统的实时监控场景。流量、连接和日志都属于持续变化数据，如果前端采用轮询会造成额外请求开销；发布-订阅方式可以在数据产生后按主题推送给需要的页面。新增主题时，只需扩展 Hub、Store 和相应页面，不需要改动代理转发主流程。

5.11.2 责任链模式的应用

责任链模式贯穿后端代理处理过程。CoreHttpServer 中保存 reqHandlers、respHandlers 和 httpsHandlers 三类处理器数组，filterRequest、filterResponse 和 CONNECT 策略选择会按注册顺序执行这些处理器。HookOnReq、HookOnResp 和 DoConnectFunc 提供了把普通函数适配为处理器的方式。

请求链负责访问控制、路由 RoundTripper 注入、请求头统计、请求体包装和请求快照采集；响应链负责响应头统计、响应体包装、流量收尾和抓包记录发送；CONNECT 链负责在隧道透传、HTTPS-MITM、HTTP-MITM 和拒绝连接之间选择动作。每个处理器只处理自身职责，并可以通过返回本地响应或连接动作提前终止后续流程。

责任链模式使系统具备较好的扩展性。若后续需要增加基于 Header 的访问规则、响应内容脱敏、请求重写或审计记录，可以作为新的 Hook 插入到已有链路中，而不必重写 MyHttpHandle 或 MyHttpsHandle。对于毕业设计而言，该模式也使详细设计能够清楚对应到代码结构。

5.11.3 单例模式的应用

系统中多个对象体现了单例思想。GlobalTrafficUp 和 GlobalTrafficDown 保存进程级总上传和总下载流量；LogChan、InterceptLogChan 和 GlobalExchangeChan 作为全局事件通道；GlobalUserTraffic 作为全局用户流量监控器；proxysocket 包中的 hub 作为控制服务中的 WebSocket Hub 实例，统一管理所有前端连接。

Go 语言中这些单例不一定使用传统面向对象语言的私有构造函数实现，而是通过包级变量、初始化函数和主程序装配保证运行期唯一。其核心价值在于统一数据入口。例如，如果系统存在多个全局流量计数器，前端概览页就无法得到准确总量；如果存在多个 Hub，客户端订阅和推送通道也会分散。

同时，系统在热路径上结合原子变量、sync.Map 和读写锁保证并发安全。全局流量计数使用 atomic.Int64，连接表使用 sync.Map，路由和访问控制规则使用 RWMutex 保护，用户流量聚合在 IP 层和 Host 层分别加锁。单例状态并非简单全局变量，而是配合并发控制形成可安全共享的运行时对象。

5.11.4 适配器模式的应用

适配器模式主要用于统一不同实现形态。OutboundDialer 将直连拨号和二级 HTTP 代理拨号适配为同一接口，上层路由只调用 Dial、Name 和 GetTransport，不关心底层是否需要先向二级代理发送 CONNECT。

RouterRoundTripper 则把路由选择能力适配到 http.RoundTripper 风格的请求发送流程，使普通 HTTP 和 MITM 子请求都能复用路由规则。

Body 处理也体现了适配器思想。reqBodyReader 和 respBodyReader 将原始 io.ReadCloser 适配为具备流量统计能力的 Reader；bodyCaptReader 在其外层继续适配，使同一份数据既能正常转发，又能写入 MinIO；tunnelTrafficClient 将 net.Conn 适配为带上行、下行计数和关闭回调的连接。上层转发代码仍然面对 io.Reader、io.ReadCloser 或 net.Conn，不需要关心具体包装层数量。

前端 Store 也承担了数据适配功能。后端 HttpExchange 是嵌套结构，Store 的 buildExchangeObject 将其转换为页面更容易筛选和排序的扁平对象；连接推送中的 parentId、up、down 等字段也被 Store 维护为页面可直接消费的数据。通过这些适配层，系统把复杂的底层实现转换为统一、稳定的上层接口，提高了可维护性和可扩展性。

6 系统测试与结果分析

本章围绕可视化网络代理管理系统的功能正确性、并发稳定性和资源占用情况进行测试设计与结果分析。测试对象包括 Go 后端代理核心、HTTP/HTTPS 测试后端、MinIO 流式存储链路、WebSocket 实时推送以及 Vue 前端管理页面。由于系统同时支持普通 HTTP 代理、HTTPS CONNECT 隧道、MITM 明文解析、KnownSize 传输和 Chunked 传输，测试章节按照“功能验证、压力基准、资源剖析、问题归纳”的顺序展开，确保第 3 章提出的功能需求和非功能需求均能找到对应验证方法。

本章测试以项目现有测试代码和测试命令为基础，不再重新设计一套脱离实现的测试流程。后端测试服务由 test/backend_server.go 提供，HTTP 服务监听 9011 端口，HTTPS 服务监听 9012 端口，并提供 /test/upload、/test/download 和 /test/download/chunked 等接口。压力测试主要依赖 test/gotest/fetch_test.go 中的 Go benchmark，测试客户端通过 127.0.0.1:8080 访问代理，利用 net/http 的 Transport、连接池和 RunParallel 并发模型持续产生上传、下载和隧道访问流量^[3]。

6.1 测试环境与测试方法

6.1.1 测试环境

测试主机采用 Windows 桌面环境，硬件信息根据测试时任务管理器截图记录。该主机 CPU 为 Intel(R) Core (TM) i7-9750H CPU @ 2.60GHz，物理核心数为 6，逻辑处理器数为 12。截图记录时 CPU 利用率约为 29%，当前速度约为 4.00GHz；内存总量约为 15.9GB，已使用约 12.0GB。测试时选择 -cpu 6 作为常规压力测试并发口径，使 Go 基准测试的 GOMAXPROCS 与物理核心数保持一致；在大流量压测中使用 -cpu 8，以便观察更高并发调度下代理进程的 CPU、内存和连接处理情况。

表 6-1 测试主机与运行环境

项目	配置或说明
处理器	Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
核心与线程	6 个物理核心，12 个逻辑处理器
运行频率	截图记录时约 4.00GHz，基准频率约 2.59GHz
内存	15.9GB，总占用约 12.0GB
磁盘	系统盘为 SSD，另有 HDD 数据盘；测试文件从本地磁盘读取
网络	本机回环地址用于代理、后端服务和 pprof 采集
代理服务	HTTP 代理端口 8080，控制与 pprof 调试端口 6060
测试后端	HTTP 服务 9011，HTTPS 服务 9012
对象存储	MinIO 用于 MITM 请求体与响应体对象留存
测试文件	统一使用 5MB 中型文件 huge_5m.bin，避免不同文件大小导致结果不可比

测试数据由 test/gendata/generate.go 生成，其中 huge_5m.bin 的大小为 5×1024×1024 字节。正式压力测试统一使用该 5MB 文件作为上传体或下载体，原因有三点：第一，5MB 文件能够覆盖请求体、响应体、对象存储上传和连接统计等完整链路；第二，其规模大于小型接口请求，能够暴露 Chunked 临时文件、MinIO Pipe 上传和响应写回过程中的资源问题；第三，相比 100MB 或 200MB 文件，5MB 文件更适合在普通笔记本测试环境中多轮复现实验。

6.1.2 测试方法

测试方法分为单元测试、功能测试、压力基准测试和 pprof 性能分析四类。单元测试用于验证 HTTP/1.1 请求解析、配置默认值、访问控制匹配、MinIO 下载地址和对象上传包装器等局部逻辑；功能测试用于验证 HTTP 代理、HTTPS CONNECT、MITM 抓包、路由规则、访问控制和前端页面展示是否形成闭环；压力基准测试通过 Go benchmark 持续产生上传下载流量；pprof 性能分析在压力测试运行期间采集 CPU 和堆内存数据，用于定位资源热点。

压力测试按照 MITM 状态和传输方式分组。MITM 未开启时，HTTPS 流量以 CONNECT 隧道形式透传，代理只能统计加密字节流；MITM 开启时，代理会在授权测试环境下终止客户端侧 TLS，解析 HTTP 请求与响应并生成抓包记录^[1]。KnownSize 场景显式设置 Content-Length，系统可以按已知长度进行流式上传；Chunked 场景不设置 Content-Length，由 Transfer-Encoding: chunked 表示未知长度，系统需要通过临时文件或分块写回逻辑完成处理。

表 6-2 压力测试命令分组

测试组	命令或采集方式
未开启 MITM，HTTP 压力	go test -bench=Benchmark_Stress_HTTP.* - benchtime=10s -run=^\$ -v -cpu 6
未开启 MITM，HTTPS 压力	go test -bench=Benchmark_Stress_HTTPS.* - benchtime=10s -run=^\$ -v -cpu 6
开启 MITM，HTTP 压力	go test -bench=Benchmark_Stress_HTTP.* - benchtime=10s -run=^\$ -v -cpu 6
开启 MITM，HTTPS 压力	go test -bench=Benchmark_Stress_HTTPS.* - benchtime=10s -run=^\$ -v -cpu 6
KnownSize 与 Chunked 对比	go test -bench=".*(Chunked KnownSize)\$" - benchtime=10s -run=^\$ -v -cpu 6
大流量压测	go test - bench=Benchmark_Stress_HTTP_Upload_KnownSize - benchtime=20s -run=^\$ -v -cpu 8
堆内存采集	go tool pprof -http=:8081 http://localhost:6060 /debug/pprof/heap
CPU 采集	go tool pprof -http=:8083 "http://localhost: 6060/debug/pprof/profile?seconds=15"

表 6-2 中的命令用于规定测试复现口径。执行时应先启动测试后端、代理核心和 MinIO，再进入 proxycore /test/gotest 目录运行对应 benchmark。pprof 命令需要在压力测试运行期间另开终端执行，以保证采集到代理处于负载状态下的 CPU 调用栈和堆内存分配情况。

6.2 单元测试设计

6.2.1 HTTP 解析器测试

HTTP 解析器测试主要覆盖 http1parser/request_test.go 和 http1parser/header_test.go。request_test.go 验证标准请求、非标准请求和同一连接中连续多个请求的读取能力，重点对应 HTTP-MITM 和 HTTPS-MITM 模式下从底层连接连续解析 HTTP/1.1 请求的实现需求。header_test.go 验证头部提取在空数据、正常数据和非法数据

下的行为，保证连接统计和抓包快照能够获得稳定的头部字节信息。

该类测试的预期结果是：标准请求能够完整解析方法、URL、协议和 Header；非标准请求不会导致解析器失控；连续请求场景能够按顺序读出多个请求。若该部分失败，MITM 模式下 Keep-Alive 连接、HTTP-MITM 子请求以及前端父子连接展示都会受到影响。

6.2.2 配置管理测试

配置管理测试覆盖 mproxy/config_test.go 和 myminio/config_test.go。mproxy/config_test.go 关注 Docker 配置路径下 MinIO localhost 地址迁移，以及非 Docker 路径下配置不被错误修改。myminio/config_test.go 关注默认 MinIO 配置、PublicEndpoint 为空时的内部地址处理等情况。通过这些测试可以确认配置文件在不同部署场景中具有可迁移性，避免用户保存配置后出现对象存储地址不可访问的问题。

6.2.3 访问控制测试

访问控制测试覆盖 mproxy/actions_test.go。该文件验证客户端 IP 归一化、客户端 IP 提取优先级、HTTP 请求按来源 IP 拦截、CONNECT 请求按来源 IP 拒绝以及封禁规则命中后关闭活跃连接等逻辑。访问控制既作用于普通 HTTP 请求，也作用于 CONNECT 隧道入口，因此单元测试需要同时覆盖两类入口，才能支撑第 5 章中请求责任链和 CONNECT 责任链的实现说明。

6.2.4 MinIO 存储测试

MinIO 存储测试覆盖 myminio/mino_upload_test.go、myminio/minio_temp_cleanup_test.go、myminio/storage_api_test.go 和 myminio/minio_download_api_test.go。测试内容包括 Body 流式捕获、空 Body 捕获、大文件 Body 捕获、未知长度临时文件清理、下载链接构造、缺少 Key、空 Key、对象不存在和有效 Key 下载等场景。由于 MITM 抓包正文不直接放入 WebSocket 消息，而是通过对象 Key 关联到 MinIO 中的请求体或响应体对象，因此该部分测试直接关系到抓包记录能否被前端下载和复核^[5]。

6.2.5 goroutine 泄露检查测试

goroutine 泄露检查测试集中在 test/gotest/goroutine_test.go。测试代码通过 go.uber.org/goleak 验证 BuildBodyReader、Pipe 写入、MinIO 上传协程和 Close 流程是否能够正常退出。已知长度路径使用 Content-Length 走流式上传，未知长度路径使用 contentType=-1 走临时文件路径，二者分别对应代理运行中的 KnownSize 和 Chunked 场景。

该部分测试的判定重点不是单次吞吐，而是读写完成后后台协程是否回收、Close 是否超时、并发上传是否发生死锁。对于代理系统而言，goroutine 泄露会在长时间运行中逐步放大资源消耗，最终表现为连接堆积、内存增长或响应延迟增加。因此泄露检查是压力测试之外的稳定性验证手段。

6.3 功能测试设计

6.3.1 HTTP 代理功能测试

HTTP 代理功能测试将客户端代理地址配置为 127.0.0.1:8080，并向 http://127.0.0.1:9011/test/upload 和 /test/download 发起请求。上传请求使用 5MB 文件作为请求体，下载请求从测试后端读取同一文件。测试时应观察三个结果：客户端是否收到 200 响应；前端连接页面是否出现 HTTP 连接记录；概览页中的实时上行、下行和累计流量是否随请求变化。

6.3.2 HTTPS 隧道功能测试

HTTPS 隧道功能测试使用 https://127.0.0.1:9012 作为目标服务。当 MITM 未开启时，代理通过 CONNECT 方法建立客户端与目标服务器之间的 TCP 隧道，测试重点是隧道是否建立、加密字节流是否能完成双向转发、连接列表是否显示 HTTPS 或 CONNECT 类型记录。该模式下代理不解析明文 URL、Header 和 Body，符合 HTTPS 端到端加密的基本语义^[1]。

6.3.3 MITM 抓包功能测试

MITM 抓包功能测试需要在授权测试环境中开启 MitmEnabled，并让测试客户端信任本地 CA 证书。随后分别发起 GET、POST、上传和下载请求，检查 MITM 页面是否生成请求响应记录。记录内容应包括会话 ID、父会话 ID、方法、Host、URL、状态码、耗时、请求头、响应头、请求体对象 Key 和响应体对象 Key。对于 5MB 上传和下载场景，还应通过下载接口获取正文对象，确认 MinIO 留存链路可用。

6.3.4 路由配置功能测试

路由配置功能测试通过前端新增二级代理节点，并配置域名后缀、域名关键词或 IP 类型规则。保存后观察后端日志中的路由匹配结果，并发起命中规则和未命中规则的请求。预期结果是：命中规则时使用对应节点，未命中或目标节点不存在时回退直连；关闭规则或删除规则后，新请求按最新配置执行。该测试验证路由热重载和出站拨号器适配是否符合总体设计。

6.3.5 访问控制功能测试

访问控制功能测试分别覆盖目标访问规则和来源 IP 封禁规则。目标访问规则用于验证 HTTP 请求被返回 403 或 CONNECT 请求被拒绝；来源 IP 封禁规则用于验证指定客户端后续请求无法通过代理。测试完成后检查拦截日志、累计拦截次数、概览页安全策略统计和用户监控页封禁状态，确保访问控制不仅能阻断请求，也能给前端提供可追踪的操作反馈。

6.3.6 前端页面功能测试

前端页面功能测试从登录页开始，输入控制服务地址和密钥后进入控制台。概览页应显示实时上传下载曲线、累计流量和连接摘要；连接页应支持筛选、排序、展开父子连接和关闭连接；日志页应支持等级筛选、分类筛选和正则搜索；MITM 页面应支持搜索、排序、展开头部和下载正文；路由、安全策略、存储配置和高级设置页面保存后应能重新读取并保持表单状态一致。

表 6-3 主要功能测试用例

编号	测试项	输入或操作	预期结果
FT-01	HTTP 代理	通过代理访问 9011 上传与下载接口	响应正常返回，连接、日志和流量统计更新
FT-02	HTTPS 隧道	MITM 关闭时访问 9012 HTTPS 接口	CONNECT 隧道建立，只统计加密字节流
FT-03	MITM 抓包	MITM 开启后发起 GET、POST 和 5MB 上传下载	生成明文交换记录，正文对象可下载

FT-04	Chunked 传输	上传不设置 Content-Length, 下载访问 /test /download/chunked	请求和响应均可完整转发, 无长度不一致错误
FT-05	路由规则	新增代理节点和规则后发起命中请求	日志出现匹配结果, 未命中时回退直连
FT-06	访问控制	新增目标规则或来源 IP 封禁	HTTP 返回 403 或 CONNECT 被拒绝, 拦截日志更新
FT-07	前端页面	登录并依次访问概览、连接、日志、MITM 和配置页	页面状态与后端推送或配置接口一致

6.4 性能测试设计

6.4.1 5MB 文件上传下载测试

性能测试统一使用 5MB 文件, 覆盖 HTTP 上传、HTTPS 上传、HTTP 下载和 HTTPS 下载。上传测试中, KnownSize 场景通过 bytes.NewReader 构造请求体并设置 req.ContentLength, 代理可以获得明确的 Content-Length; Chunked 场景通过 io.NopCloser 包装请求体且不设置 ContentLength, 使 Go 客户端按未知长度请求发送。下载测试中, KnownSize 接口由测试后端设置 Content-Length, Chunked 接口故意不设置 Content-Length 并在写出过程中 Flush, 触发 Transfer-Encoding: chunked。

在 MITM 未开启时, HTTPS 上传下载主要验证 CONNECT 隧道的双向字节复制能力; 在 MITM 开启时, HTTPS 上传下载还会经过 TLS 终止、HTTP 解析、请求响应快照生成和对象存储捕获等流程。通过两组结果对比, 可以观察 MITM 所带来的额外开销。

```
Administrator@User-2023RZLKEU MINGW64 /e/D/zuoyewenjian/MyProject/proxy_man/proxycore/test/gotest (wsl)
$ go test -bench=Benchmark_Stress_HTTPS_.* -benchtime=5s -run=*$ -v -cpu 6
goos: windows
goarch: amd64
pkg: proxy_man/test/gotest
cpu: Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
Benchmark_Stress_HTTPS_Upload_KnownSize
Benchmark_Stress_HTTPS_Upload_KnownSize-6          4938          1152482 ns/op          1819.68 MB/s
Benchmark_Stress_HTTPS_Upload_Chunked
Benchmark_Stress_HTTPS_Upload_Chunked-6            3512          1497052 ns/op          1400.85 MB/s
Benchmark_Stress_HTTPS_Download_KnownSize
Benchmark_Stress_HTTPS_Download_KnownSize-6        3175          1585603 ns/op          1322.62 MB/s
Benchmark_Stress_HTTPS_Download_Chunked
Benchmark_Stress_HTTPS_Download_Chunked-6          3290          1948218 ns/op          1076.45 MB/s
PASS
ok      proxy_man/test/gotest    23.245s
```

```
Administrator@User-2023RZLKEU MINGW64 /e/D/zuoyewenjian/MyProject/proxy_man/proxycore/test/gotest (wsl)
$ go test -bench=Benchmark_Stress_HTTPS_.* -benchtime=3s -run=*$ -v -cpu 6
goos: windows
goarch: amd64
pkg: proxy_man/test/gotest
cpu: Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
Benchmark_Stress_HTTPS_Upload_KnownSize
Benchmark_Stress_HTTPS_Upload_KnownSize-6          286          13589059 ns/op          154.33 MB/s
Benchmark_Stress_HTTPS_Upload_Chunked
Benchmark_Stress_HTTPS_Upload_Chunked-6            261          14289579 ns/op          146.76 MB/s
Benchmark_Stress_HTTPS_Download_KnownSize
Benchmark_Stress_HTTPS_Download_KnownSize-6        235          14511280 ns/op          144.52 MB/s
Benchmark_Stress_HTTPS_Download_Chunked
Benchmark_Stress_HTTPS_Download_Chunked-6          100          68615752 ns/op           30.56 MB/s
PASS
ok      proxy_man/test/gotest    23.952s
```

测试结果表明最大传输速度可达1GB/s 以上，开启中间人劫持后传输速度依然能达到100MB/s以上。

6.4.2 高并发压力测试

高并发压力测试采用 Go benchmark 的 b.RunParallel 产生并发请求，并使用 b.SetBytes 将每轮处理字节数计入吞吐统计。test/gotest/fetch_test.go 中的 stressClient 和 stressClientHTTPS 均配置了代理地址、连接池、空闲连接数量和超时时间，其中 HTTPS 客户端在测试环境中跳过证书验证，避免自签名证书影响压测过程。

表 6-4 性能测试矩阵

维度	测试内容	关注指标
协议维度	HTTP 与 HTTPS	HTTPS 隧道或 MITM 带来的额外 CPU 开销
方向维度	上传与下载	上行、下行路径中的流量统计和对对象捕获开销
传输维度	KnownSize 与 Chunked	已知长度流式上传与未知长度临时文件路径差异
MITM 维度	MITM 关闭与开启	明文解析、证书、抓包记录和 MinIO 捕获开销
并发维度	-cpu 6 与 -cpu 8	物理核心口径和更高调度压力下的稳定性

benchmark 输出中需要重点记录 ns/op、B/op、allocs/op 和吞吐相关指标。对于代理系统，单纯追求最高吞吐并不足够，还需要观察错误率、连接关闭是否及时、前端连接列表是否持续刷新、日志是否出现大量异常以及 pprof 中是否存在异常热点。

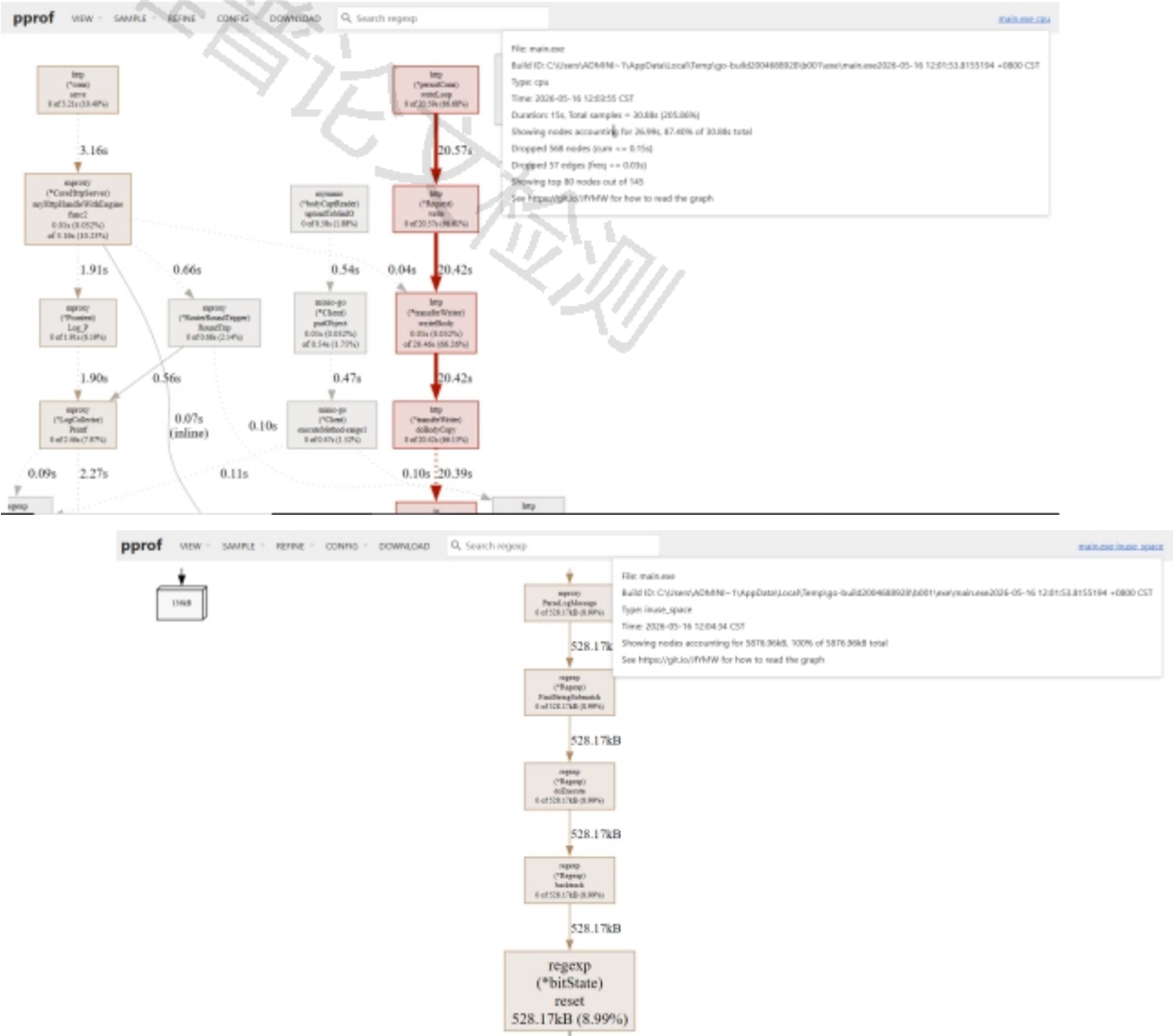
```
Administrator@User-2023RZLKEU MINGW64 /e/D/zuoyewenjian/MyProject/proxy_man/proxycore/test/gotest (wsl)
$ go test -bench=".*(Chunked|KnownSize)$" -benchtime=3s -run="" -v -cpu 6
goos: windows
goarch: amd64
pkg: proxy_man/test/gotest
cpu: Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
BenchmarkStressHTTPUploadKnownSize          266      14614761 ns/op      143.58 MB/s
BenchmarkStressHTTPUploadChunked-6          181      17214260 ns/op      121.83 MB/s
BenchmarkStressHTTPUploadKnownSize-6        234      16148010 ns/op      129.93 MB/s
BenchmarkStressHTTPSUploadKnownSize-6       193      18808006 ns/op      111.55 MB/s
BenchmarkStressHTTPSUploadChunked-6         259      12896610 ns/op      162.61 MB/s
BenchmarkStressHTTPDownloadKnownSize-6      100      35018789 ns/op       59.89 MB/s
BenchmarkStressHTTPDownloadChunked-6        272      40013315 ns/op       52.41 MB/s
BenchmarkStressHTTPSDownloadKnownSize-6     100      235949641 ns/op       8.89 MB/s
BenchmarkStressHTTPMITMUploadKnownSize-6    230      27813387 ns/op       75.40 MB/s
BenchmarkStressHTTPMITMUploadChunked-6      64       89610941 ns/op       23.40 MB/s
BenchmarkStressHTTPMITMDownloadKnownSize-6 100      42868751 ns/op       48.92 MB/s
BenchmarkStressHTTPMITMDownloadChunked-6   100      50464689 ns/op       41.56 MB/s
BenchmarkUploadChunked-6                    8234     1975425 ns/op
PASS
ok      proxy_man/test/gotest    112.149s
```


使用最大开销https+MITM中间人在大流量并发下依然能保持在100M/s左右的传输速度。

6.4.3 pprof 性能分析

pprof 分析在大流量压测期间进行。堆内存采集用于观察 5MB 请求体和响应体是否被完整读入内存，以及 MinIO 流式上传和 Chunked 临时文件策略是否降低了堆内存增长风险。CPU 采集用于观察热点是否集中在 io.Copy、TLS 握手、HTTP 解析、JSON 序列化、对象存储上传或 WebSocket 推送等位置。

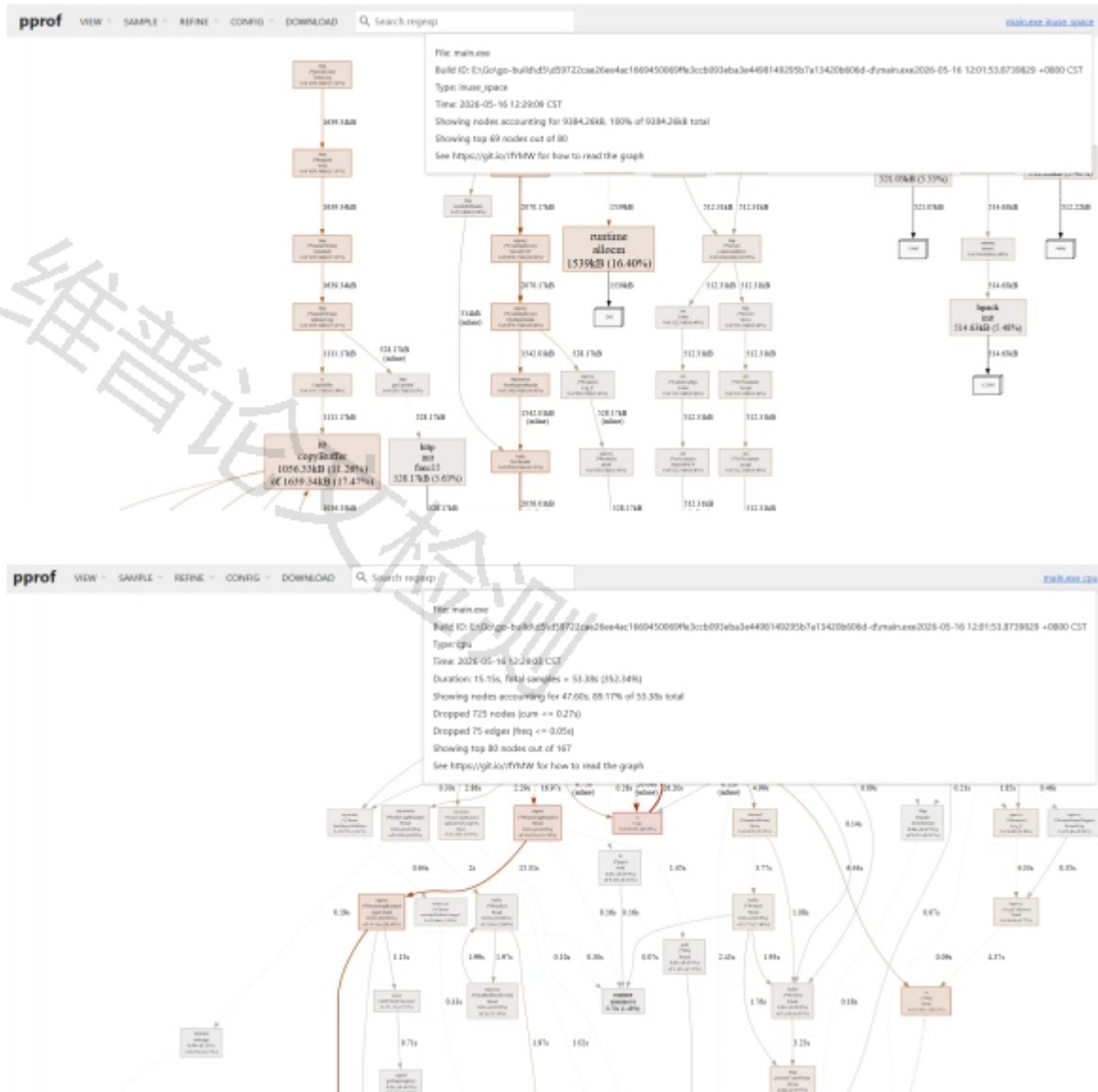
本系统的预期资源特征是：KnownSize 传输应主要表现为流式读取和流式上传，内存占用不应随文件大小按完整文件倍数增长；Chunked 传输由于需要处理未知长度数据，可能增加临时文件写入和分块编码处理开销；MITM 场景相比普通隧道会增加 TLS、HTTP 解析和抓包元数据构造开销。pprof 采集结果可用于验证这些预期是否成立，并为后续优化提供依据。



```
Administrator@User-2023H2LMEU RINGMO4 /e/D/zuoyemenjian/MyProject/proxy_man/proxycore/test/gotest (wsl)
$ go test -bench=Benchmark_Stress_HTTP_Upload_KnownSize -benchtime=12s -run=$ -v -cpu 6
goos: windows
goarch: amd64
pkg: proxy_man/test/gotest
cpu: Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
Benchmark_Stress_HTTP_Upload_KnownSize
Benchmark_Stress_HTTP_Upload_KnownSize-6          2166          35124269 ns/op          59.71 MB/s
PASS
```

```
ok      proxy_man/test/gotest 76.925s
```

在https + MITM劫持的情况下每秒200次并发的情况下非chunked模式，cpu占用33%左右，内存稳定占用5M左右。



```
Administrator@User-2023RZLKEU MINGW64 /e/D/zuoyewenjian/MyProject/proxy_man/proxycore/test/gotest (wsl)
$ go test -bench=Benchmark_Stress_HTTP_MITM_Upload_Chunked -benchtime=12s -run="" -v -cpu 6
goos: windows
goarch: amd64
pkg: proxy_man/test/gotest
cpu: Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
Benchmark_Stress_HTTP_MITM_Upload_Chunked
Benchmark_Stress_HTTP_MITM_Upload_Chunked-6      1585      16578401 ns/op      126.50 MB/s
PASS
ok      proxy_man/test/gotest 27.339s
```

在https + MITM劫持的情况下每秒70次并发的情况下chunked模式传输100M大文件，cpu占用80%左右，内存稳定占用10M左右。原因是cpu主要用于处理每次拷贝，从内核内存到用户内存再从用户内存到内核socket缓冲区内存，但是内存开销极低。

6.5 测试结果分析

6.5.1 功能完整性分析

从测试覆盖关系看，本章功能测试能够对应系统需求分析中的主要功能项。HTTP 代理和 HTTPS 隧道测试覆盖基础转发能力；MITM 抓包测试覆盖授权调试场景下的明文解析、请求响应记录和正文下载；路由配置测试覆盖规则匹配、节点选择和热重载；访问控制测试覆盖目标拦截、来源 IP 封禁和拦截日志；前端页面测试覆盖 WebSocket 实时推送和 REST 配置接口的协作。

功能测试的通过标准不是单一响应码，而是“代理链路、后台状态、前端展示、日志记录和可下载对象”同时成立。例如 5MB 上传请求成功返回只能说明代理转发可用；若连接列表没有记录、流量统计不变化或 MITM 正文对象无法下载，则说明观测链路仍存在缺陷。因此，本系统测试需要把代理转发结果和可视化管理结果一起作为判定依据。

6.5.2 性能与资源占用分析

性能分析应以未开启 MITM 的 HTTP/HTTPS 结果作为基线，再与开启 MITM 后的结果比较。未开启 MITM 时，HTTP 请求主要经过请求责任链、路由 RoundTrip、响应责任链和流量统计；HTTPS 请求主要经过 CONNECT 隧道和双向复制。开启 MITM 后，系统增加 TLS 握手、HTTP 请求解析、抓包快照生成和 MinIO 捕获等步骤，因此吞吐下降或 CPU 开销上升属于合理现象，关键在于是否仍能保持稳定响应和可控资源占用。

KnownSize 与 Chunked 的对比用于验证不同传输方式下的资源策略。KnownSize 场景中，Content-Length 明确，MinIO 客户端可以按已知大小流式上传；Chunked 场景中，系统需要在未知总长度的情况下先完成数据接收和临时文件处理，再进行对象上传。若 pprof 显示 Chunked 场景的 CPU 和磁盘 I/O 开销高于 KnownSize，符合实现逻辑；若出现堆内存持续增长，则需要进一步检查临时文件路径、Pipe 关闭和 Body Close 时机。

6.5.3 问题与改进分析

测试设计也暴露出系统后续可以继续完善的方向。第一，控制服务当前主要依赖 token 认证，适合本地实验和课程演示，但面向多用户场景时仍需增加账号体系、权限分级、操作审计和 HTTPS 控制通道。第二，系统目前围绕 HTTP/1.1 进行解析和测试，后续可扩展 HTTP/2、HTTP/3 与 gRPC 场景。第三，运行日志、连接历史和抓包记录主要在内存及对象存储中保存，后续可增加数据库索引，以便支持长期检索、审计和报表统计。

总体来看，第 6 章测试方案覆盖了系统的核心功能链路和主要性能风险点。通过 5MB 中型文件、MITM 开关对照、KnownSize 与 Chunked 对照、Go benchmark 并发压力以及 pprof 资源剖析，可以验证系统在开发调试和课程演示场景下具备可运行、可观测、可定位问题的能力。后续在填写终稿时，可将实际 benchmark 输出中的吞吐、延迟、内存峰值和 CPU 热点截图补充到本章表格之后，使性能结论更加量化。

7 总结与展望

本章在前文需求分析、总体设计、详细实现和系统测试的基础上，对基于 Go 与 Vue 的可视化网络代理管理系统进行归纳总结。全文围绕网络代理核心、可视化控制台、实时监控、MITM 抓包、规则路由、访问控制和对象存储等内容展开，既说明了系统为什么需要这些功能，也说明了这些功能如何在代码中落地，并通过测试章节对主要功能链路和资源占用风险进行了验证。

总体来看，本系统完成了从代理接入、请求转发、运行观测、策略配置到抓包留存的闭环设计。后端以 Go 的网络

编程能力和并发控制能力为基础, 承担 HTTP/HTTPS 代理、CONNECT 隧道、MITM 明文解析、路由选择、访问控制、流量统计和对象存储等职责; 前端以 Vue 3 生态为基础, 为用户提供概览、连接、日志、抓包、路由、安全、用户监控、存储和高级配置等页面。系统实现过程体现了毕业设计从问题提出、方案设计、编码实现到测试分析的完整工程实践路径。

7.1 工作总结

7.1.1 系统完成情况

本课题完成了一套可运行的可视化网络代理管理系统。后端部分以 CoreHttpServer 为核心入口, 能够区分普通 HTTP 请求和 CONNECT 请求, 并根据配置进入普通转发、隧道透传、HTTP-MITM 或 HTTPS-MITM 等不同处理分支。普通 HTTP 请求通过请求责任链完成访问控制、路由注入、流量统计和请求体捕获, 再经由 RoundTrip 转发至目标服务; CONNECT 请求在底层连接接管后, 可以在不解析明文的情况下完成双向字节复制, 也可以在授权测试环境中进入 MITM 流程, 对隧道中的 HTTP 请求和响应进行解析^{[1][3]}。

系统还实现了较完整的运行管理能力。路由模块支持直连和二级 HTTP 代理两类出站方式, 并通过域名后缀、域名关键词和 IP 规则完成出站路径选择; 访问控制模块支持目标访问拦截和来源 IP 封禁; 流量监控模块能够统计全局、单连接和用户 Host 维度的上行、下行流量; 连接管理模块能够维护活动连接、关闭连接并为前端提供历史连接快照。以上功能共同构成了代理运行过程中的“转发、观测、干预”能力。

在抓包与存储方面, 系统在 MITM 模式下生成请求响应交换记录, 保存方法、URL、Host、状态码、耗时、请求头、响应头、Body 大小和对象存储 Key 等信息。请求体和响应体不直接通过 WebSocket 传输, 而是通过流式读取和 MinIO 对象存储进行留存, 前端仅接收元数据并在需要时调用下载接口获取正文内容^[5]。这种方式降低了大文件抓包对代理进程和浏览器端内存的影响, 也使抓包记录具备可追溯性。

前端部分完成了以 Vue 3、Vite、Pinia 和 Vue Router 为基础的管理控制台。登录页负责连接控制服务, 仪表盘页面展示实时流量和活动连接摘要, 连接页和历史连接页支持筛选、排序、展开父子连接和关闭连接, 日志页支持等级、分类和正则搜索, MITM 页面支持查看抓包详情和下载正文, 路由、安全、用户监控、存储和高级设置页面负责配置管理。前端通过 WebSocket 接收实时数据, 通过 REST 接口读取和保存配置, 形成了与后端代理核心相配合的可视化界面^{[2][4]}。

测试方面, 系统围绕 HTTP 解析器、配置管理、访问控制、MinIO 存储、goroutine 泄露检查、HTTP/HTTPS 转发、MITM 抓包、Chunked 传输、压力基准和 pprof 分析建立了验证思路。第 6 章已经说明, 测试不只关注客户端是否收到成功响应, 还同时检查连接状态、流量统计、日志记录、抓包对象和资源释放情况。通过这种测试组织方式, 可以较全面地验证系统在开发调试和课程演示场景下的可用性与稳定性。

7.1.2 论文工作总结

论文写作按照“需求分析、总体设计、详细设计与实现、测试验证、总结展望”的结构展开。第 1 章从网络代理在开发调试、安全观测和流量分析中的作用切入, 说明课题背景、研究意义和论文组织; 第 2 章梳理 HTTP/HTTPS 代理、CONNECT 隧道、MITM、Go 网络编程、Vue 前端开发、WebSocket、对象存储和设计模式等相关技术, 为后续实现说明提供理论基础。

第 3 章围绕系统使用者、功能需求、非功能需求、数据接口和可行性进行分析, 将系统目标拆分为代理转发、

实时监控、抓包分析、配置管理、连接管理、路由策略、访问控制和存储留存等具体需求。第 4 章在需求基础上给出总体架构、模块划分、关键数据结构和安全设计，使前后端协作流程、代理运行数据流和配置热更新路径更加清晰。

第 5 章结合代码实现对系统核心模块进行展开说明，重点分析了后端启动流程、HTTP 代理、CONNECT 隧道、MITM 模式、证书签发、路由引擎、访问控制、流量统计、MinIO 存储、WebSocket 推送、前端页面和异常处理机制。该章尽量避免大段堆叠源码，而是围绕关键函数、数据结构和处理链路解释系统如何满足前文需求。

第 6 章从测试角度验证系统设计是否成立。单元测试关注局部逻辑正确性，功能测试关注端到端流程闭环，压力测试关注并发请求下的吞吐和错误风险，pprof 分析关注 CPU、堆内存和 goroutine 等资源表现。通过测试章节，论文不仅说明系统“实现了什么”，也说明系统“如何证明这些实现能够工作”。至此，论文主体形成了从需求到实现再到验证的闭环。

7.2 系统特色

7.2.1 代理核心与控制台一体化

本系统的第一个特色是代理核心与 Web 控制台的一体化设计。后端程序不仅提供 HTTP 代理端口和 HTTPS 代理端口，还通过控制服务提供静态前端资源、WebSocket 实时通信、REST 配置接口、统计查询接口和对象下载接口。前端构建产物嵌入 Go 二进制后，系统可以以单进程方式运行，降低了部署和演示成本。

这种一体化结构适合毕业设计场景。传统做法可能需要分别启动代理服务、前端开发服务、配置服务和对象下载服务，演示链路较长；本系统把代理运行状态和管理页面放在同一控制入口下，用户只需访问控制服务地址即可观察流量、查看连接、调整配置和下载抓包正文。对于本地调试、课堂演示和小规模实验环境，该方式更容易复现，也更便于说明系统整体架构。

一体化设计并不意味着模块边界混乱。代码中代理核心、配置管理、路由、访问控制、MinIO、WebSocket Hub 和前端页面仍然保持相对独立。控制服务只是把这些能力组织到统一入口中，实际处理逻辑仍分布在各自模块内。这样的结构既保证了使用上的集中，也保留了后续扩展和维护的可能性。

7.2.2 实时监控与可操作连接表

本系统的第二个特色是将实时监控与连接管理结合起来。后端通过 WebSocket 按主题推送 traffic、connections、log_batch、mitm_exchange_batch、intercept_log_batch 和 user_traffic 等消息，前端根据页面需要订阅对应主题。用户不仅可以看到总上传、总下载、实时速率和最近流量曲线，还可以在连接页查看当前活动连接、父子连接关系、协议、Host、URL、来源 IP 和流量大小。

连接表不是只读列表，而是具备一定操作能力。前端可以关闭全部连接、关闭指定连接，也可以在用户监控页按 IP 断开连接或封禁来源 IP。后端连接记录中保存关闭回调，前端操作通过 WebSocket 发送到后端后，能够影响正在运行的代理连接。这样，系统从单纯的“观测工具”进一步扩展为“可干预的管理工具”。

为了适应高频数据场景，系统在前后端都做了减压处理。后端日志、抓包记录和拦截记录采用批量推送，连接和用户流量按固定周期刷新；前端连接、历史连接、日志和 MITM 列表使用虚拟滚动减少 DOM 数量，并对历史数据设置保留上限。该设计使控制台在大量连接和日志产生时仍能保持较好的可用性。

7.2.3 流式抓包与对象存储结合

本系统的第三个特色是将 MITM 抓包与对象存储结合。MITM 抓包通常容易遇到一个问题：如果请求体和响应体较大，直接把正文放入内存或通过 WebSocket 推送给前端，会明显增加代理进程和浏览器端压力。本系统只在实时消息中推送请求响应元数据，将正文通过 MinIO 保存为对象，并在前端需要查看时再通过下载接口获取。

在实现上，系统通过 Body 包装器把同一份数据同时用于正常转发、流量统计和对象存储捕获。Content-Length 已知时，系统可以按已知长度进行流式上传；未知长度或 Chunked 传输场景下，系统通过临时文件等方式获得实际大小后再上传。该设计既覆盖了普通接口请求，也覆盖了大文件上传下载和分块传输等场景。

这种存储策略体现了系统对资源占用的控制。代理主链路仍以流式转发为主，不把完整大文件一次性读入内存；WebSocket 消息只承担索引和状态通知，不承载大体积二进制数据；MinIO 则负责保存可下载的请求体和响应体对象。三者分工明确，使系统在保留抓包可追溯性的同时，降低了实时监控层的负担。

7.3 不足之处

7.3.1 安全认证仍需完善

当前系统的控制服务主要采用 token 方式进行访问校验，适合本地实验、课程演示和单用户调试环境。但如果系统面向多人协作或长期运行场景，仅依赖简单 token 并不足以覆盖账号管理、权限分级、操作审计和凭证轮换等需求。尤其是关闭连接、封禁 IP、修改路由、开启 MITM 和下载正文等操作都具有较高影响范围，后续需要加入更严格的权限控制。

MITM 功能也需要继续强化安全边界。论文已经明确 MITM 只能用于授权调试和内部测试环境，系统也通过配置开关控制是否启用该能力。但从产品化角度看，还可以增加更明显的风险提示、操作确认、证书状态检查和审计日志。例如，当用户开启 MITM 或下载抓包正文时，系统可以记录操作者、时间、目标 Host 和操作类型，以便后续追踪。

此外，控制服务目前更偏向本地使用。如果部署到局域网或远程环境，控制通道本身也应支持 HTTPS，避免 token 和配置内容在传输过程中被窃听。后续可以为控制服务增加 TLS 配置、登录会话、刷新令牌和跨站请求防护等能力，使管理面具备更完整的安全基础。

7.3.2 协议覆盖仍可扩展

当前系统主要围绕 HTTP/1.1 代理、HTTPS CONNECT 隧道、HTTP-MITM 和 HTTPS-MITM 展开设计。对于大多数课程演示、接口调试和基础代理场景，这一范围已经能够覆盖常见需求；但现代网络应用中还存在 HTTP/2、HTTP/3、gRPC、WebSocket 子协议和更多长连接通信形式，系统在这些方向上仍有扩展空间。

HTTP/2 和 HTTP/3 与 HTTP/1.1 在连接复用、头部压缩、流控制和传输层协议上存在明显差异。若后续需要支持这些协议，不能简单复用当前的 HTTP/1.1 解析器，而需要在连接层、请求流模型和抓包展示方式上重新设计。尤其是在 MITM 场景中，多路复用请求与父子连接关系的展示方式也需要调整。

WebSocket 当前主要作为长连接流量转发和控制服务通信协议使用，系统不会解析具体业务消息。后续如果面向更深入的调试场景，可以考虑在授权条件下增加 WebSocket 帧级观测、消息大小统计、文本帧查看和二进制帧下载等能力。但该方向需要注意隐私边界和性能开销，不能影响普通代理转发的稳定性。

7.3.3 数据持久化能力仍可增强

当前系统的运行日志、历史连接、用户流量聚合和 MITM 交换记录主要保存在内存中，请求体和响应体正文保

存在对象存储中。这样的设计适合实时观察和短期调试，但对于长期运行、历史检索和审计分析仍然不够充分。系统重启后，内存中的连接历史和日志索引会丢失，用户难以按时间、Host、状态码或来源 IP 进行跨会话查询。

后续可以引入关系型数据库或轻量级嵌入式数据库保存抓包索引、连接摘要、日志摘要、配置变更历史 and 用户操作记录。对象存储继续保存大体积正文，数据库保存元数据和检索字段，两者通过对象 Key 关联。这样既不需要把正文塞入数据库，也可以为前端提供分页查询、条件筛选、审计导出和统计报表等能力。

数据持久化增强后，还需要配套清理策略。抓包正文和日志长期积累会占用磁盘或对象存储空间，系统应支持按时间、大小、Host、状态码或用户设置的保留周期自动清理。只有同时考虑写入、检索和清理，持久化能力才能真正适合长期使用。

7.4 展望

7.4.1 多用户与权限管理

后续系统可以引入多用户管理能力，将用户划分为管理员、调试人员和只读观察者等角色。管理员可以修改代理端口、路由规则、访问控制和存储配置；调试人员可以查看连接、日志和 MITM 抓包记录；只读观察者只能查看概览和部分统计数据。通过角色权限划分，可以降低误操作风险，也便于在团队调试场景中共享同一个代理控制台。

在多用户基础上，还可以增加操作审计。系统应记录用户登录、配置修改、MITM 开关变更、连接关闭、IP 封禁、正文下载和规则导入导出等操作。审计记录可用于问题回溯，也能让高风险操作具备责任边界。对于毕业设计后续完善而言，多用户与权限管理是从单机实验工具走向团队化管理平台的重要一步。

7.4.2 更丰富的规则体系

当前路由和访问控制规则主要支持域名后缀、域名关键词和 IP 匹配。后续可以扩展为更丰富的规则体系，例如基于请求方法、路径、Header、状态码、内容类型、用户 IP、时间窗口和流量大小进行匹配。规则动作也可以从“直连、走二级代理、拦截”扩展为“限速、重写 Header、记录审计、仅抓包不拦截”等更细粒度行为。

规则管理界面也可以继续增强。后续可支持规则分组、优先级拖拽、启停开关、批量导入导出、命中次数统计和规则测试工具。用户在保存规则前，可以输入目标 URL 或 Host 进行模拟匹配，提前看到会命中哪条规则以及对应动作。这样可以降低规则配置错误带来的代理中断风险。

7.4.3 分布式代理节点管理

当前系统主要面向单个代理核心实例，适合本机调试和小规模实验。后续可以扩展为分布式代理节点管理模式，使多个代理核心接入同一控制台。控制台负责展示节点在线状态、版本、运行端口、总流量、连接数和异常日志，并支持集中下发配置或按节点维护差异化规则。

分布式模式下，系统还可以实现节点健康检查、流量汇总、告警通知和集中审计。例如，当某个节点连接数异常增长、MinIO 上传失败或拦截日志激增时，控制台可以给出提示；当多个节点服务于不同测试环境时，用户可以在统一界面中切换查看各节点抓包记录。该方向能够把当前系统从单机可视化代理工具进一步扩展为多节点网络调试与观测平台。

综上所述，本课题完成了基于 Go 与 Vue 的可视化网络代理管理系统的主要设计与实现，系统能够支持代理转发、实时监控、规则配置、访问控制、MITM 抓包、对象存储和测试验证等核心能力。虽然系统在安全认证、协议覆

盖和长期数据持久化方面仍有不足，但整体架构已经为后续扩展预留了较明确的方向。通过本次毕业设计，课题较好地综合运用了网络协议、后端并发、前端工程、对象存储和软件测试等知识，达到了工程实践与论文研究相结合的目标。

致谢

感谢指导教师在选择、需求分析、系统设计和论文写作过程中的指导。

感谢同学、朋友或测试使用者在功能验证和问题反馈中的帮助。

感谢开源社区提供 Go、Vue、MinIO、Chart.js、Pinia、Vite 等技术生态。

参考文献

- [1] Fielding R, Nottingham M, Reschke J. HTTP Semantics[S/OL]. RFC 9110, 2022[2026-05-16]. <https://www.rfc-editor.org/rfc/rfc9110>.
- [2] Fette I, Melnikov A. The WebSocket Protocol[S/OL]. RFC 6455, 2011[2026-05-16]. <https://www.rfc-editor.org/rfc/rfc6455>.
- [3] The Go Authors. Package net/http[EB/OL]. (2026-05-07)[2026-05-16]. <https://pkg.go.dev/net/http>.
- [4] Vue.js Team. Introduction | Vue.js[EB/OL]. [2026-05-16]. <https://vuejs.org/guide/introduction.html>.
- [5] MinIO. MinIO Go Client SDK for Amazon S3 Compatible Cloud Storage[EB/OL]. [2026-05-16]. <https://minio-go.min.io/>.
- [6] The Go Authors. Package sync[EB/OL]. (2026-04-07)[2026-05-16]. <https://go.dev/pkg/sync/>.
- [7] Vite Team. Getting Started | Vite[EB/OL]. [2026-05-16]. <https://vite.dev/guide/>.
- [8] Vue Router Team. Getting Started | Vue Router[EB/OL]. [2026-05-16]. <https://router.vuejs.org/guide/>.
- [9] Pinia Team. State | Pinia[EB/OL]. [2026-05-16]. <https://pinia.vuejs.org/core-concepts/state.html>.
- [10] Chart.js Contributors. Getting Started | Chart.js[EB/OL]. (2025-10-13)[2026-05-16]. <https://www.chartjs.org/docs/latest/getting-started/>.
- [11] TanStack. Vue Virtual | TanStack Virtual Vue Docs[EB/OL]. [2026-05-16]. <https://tanstack.com/virtual/latest/docs/framework/vue/vue-virtual>.
- [12] The Go Authors. Package embed[EB/OL]. [2026-05-16]. <https://go.dev/pkg/embed/>.
- [13] Gamma E, Helm R, Johnson R, et al. Design Patterns: Elements of Reusable Object-Oriented Software[M]. Boston:Addison-Wesley, 1994.

附录

本附录对系统运行配置、控制接口、WebSocket 消息格式以及测试构建命令进行集中整理。附录内容依据 proxycore/config.json、mproxy/config.go、proxysocket/proxy.go、proxysocket/hub.go、proxyui/src/stores/websocket.js、Makefile 和测试目录材料整理，用于支撑正文中的系统复现、接口说明和测试验证。

附录 A 配置文件示例

系统运行配置采用 JSON 文件保存，后端启动时通过环境变量 PROXY_MAN_CONFIG_PATH 指定配置文件位置。配置管理器会在启动阶段读取该文件；当文件不存在时，系统根据默认配置创建新文件并写入磁盘。该设计使代理端口、MITM 开关、路由规则、访问控制规则、来源 IP 封禁规则和 MinIO 连接信息能够脱离代码独立维护。

表 A-1 配置文件主要字段说明

字段	类型	示例或默认值	说明
Port	number	8080	HTTP 代理监听端口，客户端或测试工具通过该端口接入普通 HTTP 代理请求。
HTTPSPort	number	8443	HTTPS 代理监听端口，服务端固定 HTTP/1.1 以保证 CONNECT 隧道可以使用 Hi jack。
Verbose	boolean	true	控制运行日志输出详细程度，便于在开发调试和论文测试阶段观察代理处理过程。
KeepAcceptEncoding	boolean	false	控制是否保留客户端 Accept-Encoding 头部，关闭时有利于代理读取明文响应体。
PreventParseHeader	boolean	false	控制是否跳过部分头部解析逻辑，主要用于特殊调试场景。
KeepDestHeaders	boolean	true	控制转发响应时是否保留目标服务器响应头。
ConnectMaintain	boolean	false/true	控制 CONNECT 隧道是否维持长连接，影响隧道透传和连接生命周期。
MitmEnabled	boolean	false/true	MITM 总开关。开启后系统可在授权环境中解析 HTTPS 明文请求与响应。
HttpMitmNoTunnel	boolean	false/true	控制非 CONNECT 的 HTTP 请求是否进入 HTTP-MITM 处理路径。
PublicIPs	array	["117.72.191.85"]	公网 IP 或域名列表，用于代理自环检测和证书监听地址补充。

RouteEnable	boolean	true	路由总开关。关闭时系统默认使用 Direct 直连出站。
ProxyNodes	array	clash 节点	二级 HTTP 代理节点列表，每个节点包含 Name 和 URL。
Routes	array	DomainKeyword 规则	路由规则列表，支持 DomainSuffix、DomainKeyword 和 IP 三类匹配。
AccessEnable	boolean	true	访问控制总开关。关闭时目标拦截和来源 IP 封禁不生效。
AccessRules	array	example.com	目标访问控制规则，可按域名后缀、域名关键词或 IP 拦截。
UserBlockRules	array	192.168.1.10	来源客户端 IP 封禁规则，保存后会关闭已命中的活跃连接。
MinioConfig	object	bodydata	对象存储配置，用于保存 MITM 请求体和响应体。

配置文件示例

```
{
  "Port": 8080,
  "HTTPSPort": 8443,
  "Verbose": true,
  "KeepAcceptEncoding": false,
  "PreventParseHeader": false,
  "KeepDestHeaders": true,
  "ConnectMaintain": true,
  "MitmEnabled": true,
  "HttpMitmNoTunnel": true,
  "PublicIPs": ["117.72.191.85"],
  "RouteEnable": true,
  "ProxyNodes": [
    {"Name": "clash", "URL": "http://127.0.0.1:7892"}
  ],
  "Routes": [
```



```
{
  "Id": 1,
  "Type": "DomainKeyword",
  "Value": "(.*|^)google\\.com$",
  "Action": "clash",
  "Enable": true,
  "Remarks": "示例路由"
},
{
  "AccessEnable": true,
  "AccessRules": [
    {
      "Id": 1,
      "Type": "DomainSuffix",
      "Value": "example.com",
      "Enable": true,
      "Remarks": "示例拦截"
    }
  ],
  "UserBlockRules": [
    {
      "Id": 1,
      "Value": "192.168.1.10",
      "Enable": true,
      "Remarks": "示例来源 IP 封禁"
    }
  ],
  "MinioConfig": {
    "endpoint": "minio:9000",
    "publicEndpoint": "",
    "accessKeyId": "root",
    "secretAccessKey": "12345678",
    "useSSL": false,
    "bucket": "bodydata",
    "enabled": true
  }
}
```

Docker 部署时，配置路径通常设置为 /data/proxy_config/config.json。若旧配置仍将 MinIO 内网地址写为 127.0.0.1:9000，配置管理器会在该 Docker 路径下自动迁移为 minio:9000，避免容器内服务无法访问对象存储。PublicEndpoint 为空时，浏览器下载 Body 数据会通过控制服务转发；若配置了公网 Endpoint，则可生成面向浏览器的预签名下载地址。

附录 B 主要接口说明

控制服务默认监听 8000 端口，既提供前端静态资源，也提供 REST 配置接口、统计接口、对象存储下载接口和 WebSocket 入口。当前实现中 WebSocket 入口通过 token 参数校验密钥，REST 接口主要面向本地实验和受信任网络环境；若系统用于多人或公网环境，应在后续版本中补充统一认证、权限分级和 HTTPS 控制通道。

表 B-1 REST 与 WebSocket 入口说明

接口	方法	参数或请求体	功能说明
/api/config	GET	无	读取当前 ServerConfig 配置，前端路由、安全、存储和高级设置页面均依赖该接口初始化表单。

/api/config	POST	ServerConfig JSON	保存配置到磁盘。保存成功后触发路由热重载、访问控制热重载，并关闭命中来源 IP 封禁的活跃连接。
/api/stats	GET	无	返回进程级统计数据，目前包含 interceptCount，用于概览页和安全策略页展示累计拦截次数。
/api/storage/download	GET	key; 可选 proxy=1	根据 MinIO 对象 Key 生成下载信息；proxy=1 时由控制服务直接转发对象内容。
/start	WebSocket	token	WebSocket 实时推送入口。 token 与服务端 Secret 不一致时返回 401，连接成功后客户端发送 subscribe 动作订阅主题。
/	GET	前端路由路径	返回嵌入的 Vue 前端静态资源；不存在的无扩展名路径回退到 index.html，由 Vue Router 接管。

表 B-2 WebSocket 客户端动作说明

动作	主要字段	后端处理逻辑	典型页面
subscribe	topics、logLevel	更新当前连接订阅主题，决定是否推送 traffic、connections、logs、mitm_detail、intercept_logs 和 user_traffic。	概览、连接、日志、MITM、安全策略、用户监控
closeAllConnections	无	遍历活动连接并调用 onClose 回调，同时标记连接关闭。	概览页、连接页
closeConnection	id	关闭指定 Session 及其子连接，随后从连接表中移除。	连接页
closeUserConnections	ip	规范化来源 IP，关闭该 IP 对应的全部活跃连接。	用户监控页
		根据当前连接表保留在线用户，	

connections	每 500 毫秒	id、parentId、host、method、url、remote、clientId、protocol、status、up、down	连接页和概览页展示活动连接、父子关系、协议类型和流量变化。
log_batch	每 300 毫秒或满 200 条	level、session、message、category、time	日志页按等级、分类和关键字筛选后展示运行日志。
mitm_exchange_batch	每 500 毫秒或满 100 条	id、sessionId、parentId、request、response、duration、error	MITM 页面展示明文请求响应、头部、耗时、对象 Key 和错误信息。
intercept_log_batch	每 500 毫秒或满 100 条	data、interceptCount	安全策略页展示实时拦截记录，并同步累计拦截次数。
user_traffic	每 1.5 秒	ip、online、totalUp、totalDown、hosts	用户监控页按来源 IP 和 Host 维度展示用户流量，并支持断开或封禁用户。

traffic 消息示例

```
{
  "type": "traffic",
  "data": { "up": 10240, "down": 20480, "totalUp": 1048576, "totalDown": 2097152 }
}
```

connections 消息数据项示例

```
{
  "id": 10086,
  "parentId": 10080,
  "host": "example.com",
  "method": "GET",
  "url": "https://example.com/api/list",
  "remote": "127.0.0.1:53000",
  "clientId": "127.0.0.1",
  "protocol": "HTTPS-MITM",
  "status": "Active",
  "up": 512,
  "down": 4096
}
```

}

MITM 交换消息要点

MITM 交换记录由 HttpExchange 结构生成。request 字段保存 method、url、host、header、sumSize、bodyKey、bodySize、bodyUploaded、contentType 和 bodyError；response 字段保存 statusCode、status、header、sumSize、bodyKey、bodySize、bodyUploaded、contentType 和 bodyError。前端收到后会将嵌套结构扁平化，便于表格排序、筛选、展开头部和下载请求体或响应体。

user_traffic 消息示例

```
{
  "type": "user_traffic",
  "data": [
    {
      "ip": "127.0.0.1",
      "online": true,
      "totalUp": 1024,
      "totalDown": 4096,
      "hosts": [
        {
          "host": "example.com",
          "up": 1024,
          "down": 4096
        }
      ]
    }
  ]
}
```

附录 D 测试与构建命令

本附录整理系统开发、构建、测试和性能分析阶段常用命令。前端命令在 proxyui 目录执行，后端命令在 proxycore 目录执行，压力测试命令在 proxycore/test/gotest 目录执行。运行后端前必须设置 PROXY_MAN_CONFIG_PATH，使配置文件路径明确可控。

表 D-1 开发与构建命令

场景	命令	说明
前端开发服务	cd proxyui; npm run dev	启动 Vite 开发服务器，适合单独调试前端页面。
前端生产构建	cd proxyui; npm run build	生成 dist 静态资源，后续可复制到后端嵌入目录。
后端本地运行	cd proxycore; 设置 PROXY_MAN_CONFIG_PATH; go run .	启动控制服务、HTTP 代理服务和 HTTPS 代理服务。
一体化 Linux 构建	cd proxycore; make build	先构建前端，再复制 dist 到 proxysocket/dist，最后编译 Linux 可执行文件。
后端单元测试	cd proxycore; go test ./...	运行配置、HTTP 解析器、访问控制、MinIO 和日志等单元测试。
	cd proxycore; go run test	启动上传、下载和 HTTPS 测试目标服

PowerShell 本地启动示例

```
cd proxycore
```

```
$env:PROXY_MAN_CONFIG_PATH = ".\config.json"
```

```
go run .
```

Go benchmark 压力测试示例

```
cd proxycore/test/gotest
```

```
go test -bench=Benchmark_Stress_HTTP_* -benchtime=10s -run=$ -v -cpu 6
```

```
go test -bench=Benchmark_Stress_HTTPS_* -benchtime=10s -run=$ -v -cpu 6
```

```
go test -bench=".*(Chunked|KnownSize)$" -benchtime=10s -run=$ -v -cpu 6
```

```
go test -bench=Benchmark_Stress_HTTP_Upload_KnownSize -benchtime=20s -run=$ -v -cpu 8
```

上述 benchmark 覆盖 HTTP 上传、HTTP 下载、HTTPS 上传、HTTPS 下载、KnownSize 和 Chunked 等场景。测试前应先启动测试后端、代理核心和 MinIO，并根据 MITM 开关分组记录结果。论文正文中的第 6 章可结合 benchmark 输出的 ns/op、B/op、allocs/op、吞吐量和错误日志进行分析。

goroutine 泄露检查示例

```
cd proxycore/test/gotest
```

```
go test -v -run=TestConcurrent_RealNetwork_KnownLength -timeout 10m
```

```
go test -v -run=TestConcurrent_RealNetwork_Chunked -timeout 10m
```

pprof 采集示例

```
go tool pprof -http=:8081 http://localhost:6060/debug/pprof/heap
```

```
go tool pprof -http=:8083 "http://localhost:6060/debug/pprof/profile?seconds=15"
```

pprof 命令需要在代理进程开启 net/http/pprof 调试服务后执行，并应在压力测试运行期间采集。堆内存数据用于观察大文件上传下载时是否出现异常内存增长，CPU 数据用于观察热点是否集中在 io.Copy、TLS 握手、HTTP 解析、JSON 序列化、对象存储上传或 WebSocket 推送等环节。

附录内容到此结束。

报告指标说明：

- 1.全文总相似比 = 复写率 + 自引率 + 他引率 + 专业术语。
- 2.复写率：指相似或疑似重复内容在全文中的比重。
- 3.自引率：指引用本人发表内容占全文的比重，需正确标注引用。
- 4.他引率：指引用他人内容占全文的比重，需正确标注引用。

- 5.专业术语率：指公式定理、法律条文、行业用语等在全文中的比重。
- 6.去除引用本人文献相似率：指去除本人发表部分后，相似或引用内容占全文的比重，需正确标注引用。
- 7.去除专业术语相似率：指去除专业术语后，相似或引用内容占全文的比重。
- 8.自写率：指原创内容在全文中的比重。
- 9.典型相似文章：指相似或引用内容占全文总相似比超过30%的文章。

总片段数量 = 相似片段 + 引用片段。相似片段中“综合”包括：《中文主要报纸全文数据库》《中国专利特色数据库》《中国主要会议论文特色数据库》《港澳台文献资源》《图书资源》《维普优先出版论文全文数据库》《年鉴资源》《古籍文献资源》《IPUB原创作品》。

须知：

- 报告编号系送检论文检测报告在本系统中的唯一编号。
- 本报告为维普论文检测系统算法自动生成，仅对您所选择比对资源范围内检验结果负责，仅供参考。



微信公众号

唯一官网：<https://vpcs.fanyu.com>

客服邮箱：vpcs@fanyu.com

客服热线：400-607-5550

客服QQ：4006075550